Dealing with Higher Dimensions: The Well-Separated Pair Decomposition and Its Applications

Paul B. Callahan

A dissertation submitted to The Johns Hopkins University in conformity with the requirement for the degree of Doctor of Philosophy.

Baltimore, Maryland

1995

Abstract

Many problems of great practical importance and theoretical interest can be posed as computations on finite point sets in fixed-dimensional Euclidean space. Often, these problems are expressed most naturally in terms of the set of all distinct pairs of points, along with the Euclidean distance between each such pair. Such problems include finding the nearest neighbor of each point and constructing a Euclidean minimum spanning tree. Though it is not purely geometric, computing the electrostatic interaction between point charges can also be posed in these terms, and has applications to particle simulation, a fundamental technique in computational physics. Because there are quadratically many distinct pairs, one would like to avoid enumerating these pairs explicitly.

This dissertation develops a data structure, called the *well-separated pair decomposition*, that organizes the pairs of points into pairs of clusters, or well-separated pairs, in a way that preserves approximate distance. This structure has linear size and can be computed efficiently. Hence, it leads to substantial improvements over algorithms that enumerate all distinct pairs of points.

The well-separated pair decomposition uses several ideas that have been exploited in previous algorithms. However, the development presented here has led to many simplifications and refinements to these approaches. Additionally, a more flexible treatment of tree construction has led to efficient parallel and dynamic algorithms that could not be developed using previous techniques.

This dissertation presents sequential, parallel, and dynamic algorithms for computing the well-separated pair decomposition, along with applications to specific problems posed on point sets. Some empirical testing is also performed on a C implementation in order to estimate the geometric constants in the complexity.

Acknowledgements

This work could not have been completed without the help and guidance of my advisor, Rao Kosaraju. I thank him for allowing me much freedom in setting the scope and direction of my research, and yet also for being there at those times when my individual efforts brought me only to the brink of an unbridgeable gulf, or to the center of a maze of blind alleys. It is a privilege to discuss problems with Rao Kosaraju. He has the uncanny ability to see instantly the core ideas behind initial, vague attempts at a formulation, and, just as importantly, to see and to point out frankly the lack of any promising ideas.

In reflecting on my stay at Hopkins, I cannot neglect the strong influence of Mike Goodrich, who motivates us all through his enthusiasm for algorithm design, his boundless energy, and his exciting, lucid explanations of all the key strategies. It was through his course in algorithms that I first truly understood many things I had seen already and thought I knew. It was through his course in computational geometry that I developed a focused interest where there had been only a vague curiosity.

I acknowledge two others from a time and place that now seems remote, but which left no less of an impact on my life. I wish to thank Georg Schnitger, who when I was twenty, and knew something about programming, first let me in on what computer science was all about. While never failing to communicate his expertise, it always seemed the true motive of his teaching was to infect one with his singular zeal and sense of vocation. I also wish to thank Ian Parberry, whose steady, understated encouragement, and useful advice helped me far more than he may realize.

At the base of it all, I wish to acknowledge the influence of my father, Francis Callahan, who in addition to giving me all the love one could ever ask from a parent, taught me from an early age to see the beauty he saw in mathematics. To a child who saw little beauty and quite a bit of frustration in the math taught at school, such tutelage made all the difference in the world.

My friends in the Computer Science department, particularly Dave Heath,

Lewis Stiller, Jack Eifrig, and Dwight Wilson, provided welcome distractions throughout the completion of my dissertation research. Such distractions were essential in renewing my concentration.

This dissertation was supported by NSF/DARPA grant CCR-8908092, NSF grant CCR-9107293, and by an ARCS Foundation Scholarship. My first year of graduate study at Johns Hopkins was supported by an Abel Wolman Graduate Fellowship from The Whiting School of Engineering. Without such financial support, this work would not have been possible.

Contents

Table of Contents v

List of Figures viii

1 Introduction 1
- 1.1 Point-set problems . . . 2
  - 1.1.1 Planar point sets and the Voronoi diagram . . . 3
  - 1.1.2 Multi-dimensional divide-and-conquer . . . 5
  - 1.1.3 The well-separated pair decomposition . . . 5
- 1.2 Computational Framework . . . 6
  - 1.2.1 The algebraic model . . . 6
  - 1.2.2 Parallel and dynamic algorithms . . . 8
- 1.3 Overview of related techniques . . . 11
  - 1.3.1 Quadtrees and other hierachical decompositions . . . 11
  - 1.3.2 Well-separated sets . . . 14
- 1.4 Applications . . . 16
  - 1.4.1 Proximity problems . . . 16
  - 1.4.2 Spanning graphs . . . 17
  - 1.4.3 Potential computation . . . 18
- 1.5 Dissertation outline . . . 20

2 Sequential algorithms 23
- 2.1 Definitions . . . 23
- 2.2 Computing a fair split tree of a point set . . . 26
  - 2.2.1 The splitting phase . . . 29
  - 2.2.2 The main algorithm . . . 31
- 2.3 Computing a well-separated realization . . . 32
- 2.4 The canonical realization . . . 35

2.5 Procedure **findpairs** computes the canonical realization 36
2.6 Size of the canonical realization 38

3 Applications **41**
3.1 k-nearest neighbors 43
3.2 n-body potential fields 47
3.2.1 Computation of arbitrary force functions 51
3.3 Approximate Euclidean minimum spanning trees 53
3.3.1 First approximation algorithm 56
3.3.2 Refining the bound on ϵ 57
3.3.3 Final approximation algorithm 61
3.4 Approximate complete Euclidean graph 62
3.5 Exact EMST 66
3.6 Enumerating k smallest distances 70

4 Combinatorial properties **72**
4.1 Minimum depth for linear size realizations 72
4.2 Properties of the contracted tree 75
4.3 The level pairs in a restricted multi-level partition 78
4.4 Realizations for log-depth trees 79
4.5 One-to-many realizations 81
4.6 Distance classes 83

5 Parallel algorithms **86**
5.1 Parallel fair split tree construction 87
5.2 Computing the pairs 91
5.3 Optimal parallel fair split tree construction 94
5.3.1 The constructible rectangles 95
5.3.2 The production graph 99
5.3.3 Extracting T_c 101
5.3.4 Constructing the partial fair split tree 102
5.3.5 Correctness of the splitting phase 105
5.3.6 The final algorithm 106
5.4 Applications 106
5.4.1 Parallel computation of n-body potential fields 107
5.4.2 Computing k-nearest-neighbors in parallel 109
5.4.3 Other applications 113

6 Dynamic algorithms **115**
6.1 Maintaining the fair split tree 116
6.2 Point location 121
6.3 Maintaining the pairs 123
6.3.1 Dummy points 124
6.3.2 Point insertions and deletions 125
6.4 Abstract tree-pair computations 126
6.5 Directing the pairs 129
6.5.1 A data structure for detecting movement 133
6.5.2 Detecting movement for $d > 1$. 136
6.5.3 Maintaining pair directions 137
6.6 Applications 143
6.6.1 Maintenance of attractor sets 143
6.6.2 Dynamic closest pair 145
6.6.3 Fast multipole method 146

7 Empirical tests **148**
7.1 Implementation considerations 148
7.1.1 Specifics of the decomposition 149
7.2 The number of well-separated pairs 151
7.2.1 Varying n 152
7.2.2 Varying s and d 155
7.3 Tradeoffs and hybrid approaches 156

8 Conclusions **162**
8.1 Future directions 163

Bibliography **165**

List of Figures

1.1 The Voronoi diagram of a planar point set. 4
1.2 A quadtree in the plane. 12

2.1 Well-separated sets in the plane 24
2.2 Two examples of fair splits 27
2.3 A canonical pair (A, B). 36

3.1 Points a and a' 45
3.2 Approximate distance between points in well-separated sets 56
3.3 Approximate direction u of a well-separated pair 59

4.1 Node contractions in a restricted multi-level partition 77

5.1 Choosing the rectangle R. 88
5.2 A δ-rendering of the cube C 93
5.3 Case 1(a) 97
5.4 Case 1(b) 98
5.5 Case 2(a) 99
5.6 Case 2(b) 100
5.7 Case 3 101

7.1 Uniform distribution of points in a cube ($d = 3$, $s = 2.0$) 153
7.2 Uniform distribution of points in a periodic cube ($d = 3$, $s = 2.0$) . . 154
7.3 The pair constant c for varying d ($n = 10000$) 156
7.4 The pair constant c for varying s ($n = 10000$) 157
7.5 Speed-ups in hybrid and direct approaches ($d = 3$, $s = 2.0$, $n = 100000$) 159
7.6 Speed-ups for hybrid approach ($d = 3$, $s = 2.0$) 160

Chapter 1

Introduction

One of the most exciting consequences of the rapid growth in computational power has been the ability to solve problems posed on ever more detailed models of physical systems. Such computation furthers both the understanding of scientific principles and the development of technology that uses these principles. It is not surprising that geometry plays an important role in the solution to physically-motivated problems, and a branch of algorithmic theory, called *computational geometry* [52] has arisen to study geometric problems and to develop efficient algorithms to solve them. In roughly two decades since its inception, computational geometry has developed into a mature field, yet algorithmic issues remain open regarding some of its most fundamental problems.

The simplest object of geometric interest is the point, which represents a locus in space. It is not surprising, then, that sets containing a finite number of points are among the most basic objects considered in computational geometry. Point sets, understood here to be finite unless otherwise noted, play a role analogous to strings in pattern-matching algorithms or graphs in certain combinatorial algorithms, in that they provide a natural representation of instances from a wide range of problems.

While the idea of a point set is simple to understand, this simplicity belies a richness of structure that allows many practical problems to be expressed in terms of point sets. Much of this structure can be deduced by considering each point

and its distance to every other point. The analysis of such distance relationships allows us, among other things, to determine the neighborhood of a point in several natural ways, to construct minimal networks connecting all points, and to compute force interactions between particles in a dynamic system. Almost any collection of data has some meaningful representation as a point set, and the solution to point-set problems has such diverse applications as the extraction of clusters from a set of observations, the design of communications networks connecting geographic sites, and the dynamic simulation of a large-scale physical system, such as a collection of molecules or astronomical bodies.

This dissertation develops a general-purpose technique that can be used to develop efficient algorithms to solve problems posed on point sets. In addition to algorithms that make efficient use of the conventional resources of time and space on a one-processor machine, we develop massively parallel algorithms that can exploit the increase in computational resources when the number of processors is allowed to grow with the size of the input. We also develop a dynamic version of our technique, in which the effect of a small change to the input to an algorithm can be computed without having to repeat all the work of solving the original problem.

1.1 Point-set problems

We will focus our attention on problems posed on the set of all distinct pairs from a set P of n points in d-dimensional space in which each pair is labeled by Euclidean distance. For some problems, it will be useful to think of this set of pairs as the edges in a complete graph on n vertices corresponding to points in P. This graph, with edges weighted by Euclidean distance, is called the *complete Euclidean graph.*

Some important problems in this class include determining the nearest neighbor of each point, and constructing the *Euclidean minimum spanning tree*, or the minimum spanning tree of the complete Euclidean graph. While not a purely geometric problem, the problem of n-body potential computation also fits into this framework.

Because there are $\binom{n}{2}$ pairs of points, any algorithm that explicitly constructs the set of interpoint distances requires $\Omega(n^2)$ time. However, the point set itself provides an implicit representation of size n, so it is interesting to ask whether one can improve upon the above bound by employing more sophisticated algorithmic techniques. We will see that the $\Omega(n^2)$ bound is often too pessimistic, although the extent to which we can do better depends upon the problem to be solved.

1.1.1 Planar point sets and the Voronoi diagram

Considerable success has already been attained in the design of efficient algorithms for planar point set problems. Often such algorithms are based on the very powerful technique of constructing a *Voronoi diagram* of the point set. This is a subdivision of the plane into convex polygonal regions each of which is the locus of those points in the plane that are closer to some point $p \in P$ than to any other point. Assuming no degeneracies, the boundary between two such polygons is a line segment representing the set of points equidistant to two distinct closest points in P, and these line segments intersect at vertices corresponding to those points in the plane that have exactly three closest points in P. The collection of regions, line segments, and vertices is a partition of the plane. It can be represented as a planar graph, and hence its size is linear in the number of polygonal regions, which is n. An example is shown in figure 1.1.

The Voronoi diagram concisely captures a great deal of useful information about the distance relationships of points in P. For example, given a point in the plane, we can reduce the problem of finding its nearest neighbor in P to that of locating its place in a planar subdivision. In addition, the planar dual of the Voronoi diagram, called the Delaunay triangulation, has useful properties in its own right. Among other things, it contains the Euclidean minimum spanning tree as a subgraph, allowing us to compute this tree very efficiently, once we have constructed the Delaunay triangulation. A thorough treatment of such properties and their applications is provided in the textbook of Preparata and Shamos [52].

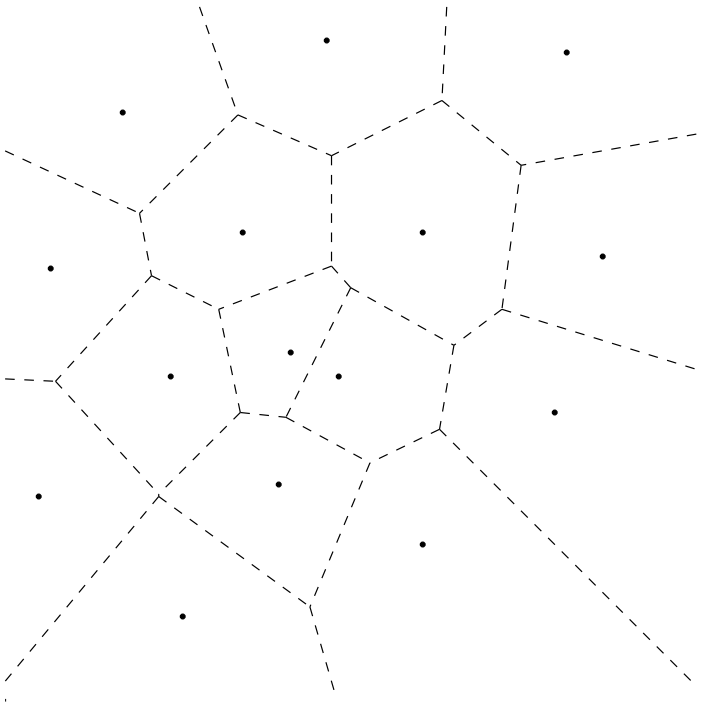


Figure 1.1: The Voronoi diagram of a planar point set.

The Voronoi diagram of a planar point set can be computed in $O(n \log n)$ time [64], which is clearly far less than that required to construct the set of all distances between pairs of points. The fact that such a useful structure can be computed efficiently has made it the mainstay of a whole class of algorithms to solve planar point set problems. Aurenhammer [12] has written an extensive survey of the Voronoi diagram, which he calls "one of the most fundamental data structures in computational geometry." In it, he discusses the history of this data structure, as well as numerous algorithms, applications, and extensions, for which he provides a comprehensive bibliography.

For many applications, it will suffice to consider points in the plane, but for others, higher dimensions are needed. Three dimensional point sets, for example, are particularly important in modeling systems residing in physical space. Point sets of

dimension $d \geq 4$ can also arise quite naturally any time we are considering a data set consisting of d-tuples. The Voronoi diagram can be extended to $d > 2$, and algorithms can be modified to use this higher dimensional structure. However, the worst-case size of a Voronoi diagram is $\Omega(n^{\lceil d/2 \rceil})$ for d-dimensional point sets [52]. Hence, algorithms that are optimal for planar point sets often become inefficient when $d \geq 3$.

1.1.2 Multi-dimensional divide-and-conquer

Many higher-dimensional point set problems can be solved efficiently using the general-purpose technique of *multi-dimensional divide-and-conquer*, introduced by Bentley [15]. The idea behind this approach is as follows. We divide a d-dimensional point set into two point sets of roughly equal size lying on either side of an axis-parallel hyperplane. As in standard divide-and-conquer recurrences, this results in two subproblems like the original, but smaller by a constant factor, and we solve these recursively.

The remaining problem is to combine the two point sets at the boundary. To apply multi-dimensional divide-and-conquer, we must be able to pose the combining step as a $(d-1)$-dimensional problem. This gives us another multi-dimensional divide-and-conquer problem, but of a lower order. Thus, eventually the problem is reduced to a low-dimensional case that is easy to solve.

Multi-dimensional divide-and-conquer is of great importance because it leads to $O(n \text{ polylog } n)$ algorithms for many d-dimensional point-set problems, where *polylog* denotes a function of the form $\log^k n$ for some constant k. Bentley used this technique to derive the first subquadratic algorithm for the all-nearest-neighbors problem when $d \geq 3$. However, this technique suffers from the significant drawback that the exponent in the polylog factor is dependent on d (usually between $d-2$ and d).

1.1.3 The well-separated pair decomposition

In this dissertation, we will present a data structure, called the *well-separated pair decomposition*, that appears better suited to higher dimensional geometric problems

than the Voronoi diagram, and, when applicable, leads to faster algorithms than those attainable using multi-dimensional divide-and-conquer. This structure was initially motivated by the fast multipole method of Greengard and Rokhlin [37, 38] for computing n-body potential, and it can be used to extend this technique by eliminating the need for certain assumptions about the bit precision of the input.

The well-separated pair decomposition of a point set P consists of a binary tree T whose leaves are points in P, with internal nodes corresponding to subsets of P in the natural way, and a list of pairs of subsets of P, each of which is a node in T. Every pair of subsets in the list must be "well-separated," a notion that we will later define rigorously. Roughly speaking, two point sets are well-separated if each represents a cluster of points such that the distance between these two clusters is significant in comparison to the size of the larger of the two. The list of pairs is a decomposition of the set of distinct pairs of points in P in the sense that every pair $\{a, b\}$ $(a, b \in P)$ must satisfy $a \in A$, $b \in B$ for exactly one pair of subsets $\{A, B\}$ in the list. Often, we will be able to perform a single operation on the pair $\{A, B\}$ in place of $|A||B|$ separate operations on the corresponding pairs of points. In doing so, we will greatly improve the efficiency of the resulting algorithms.

For fixed d, such a decomposition can always be constructed using $O(n)$ pairs of subsets. This leads to fast sequential, parallel, and dynamic algorithms for a number of point-set problems that will be examined in this dissertation.

1.2 Computational Framework

1.2.1 The algebraic model

In computational geometry, one typically assumes that points are defined by vectors of real numbers. This is natural because geometry places no other restrictions on locations in space. However, a real number can contain an infinite amount of information in its decimal expansion, so one must be especially careful in defining the set

of operations in the computational model used for geometric problems. It is standard to restrict the model to the arithmetic operations $+$, $-$, $\times$, and $/$, each of which is assumed to require unit time. In this model, the value of a variable is allowed to affect the control flow of an algorithm only through comparisons, such as $x \leq y$, which return a boolean value "true" or "false" depending on the relative values of the real variables being compared. Thus, even though the input to the algorithm can be varied continuously, the algorithm "sees" this input only through a sequence of discrete decision steps. We will refer to this set of operations as the *algebraic model*.

For some algorithms, it may seem advantageous to augment this set of operations, for example, by allowing certain bit operations, or the floor function. Often, a knowledge of how the input is represented makes an augmented operation set quite useful. Moreover, many such operations are implemented quite efficiently on existing architectures. There are two reasons that one should always ask, before proceeding, whether such augmentation is absolutely necessary. First, it seems natural that the algebraic operations given above should suffice to solve most geometric problems. For example, it is clear that they suffice to allow the most naive approach to finding the nearest neighbor of each point, since we may find the nearest neighbor of a point x by minimizing the squares of the distances to all other points. Any faster algorithm that depends on additional operations is not really comparable to the original. Second, if we restrict our algorithms to the algebraic model, we can often prove $\Omega(n \log n)$ lower bounds using the *algebraic decision tree*.

An algebraic decision tree [30, 53, 55] is a loop-free program in which branches are taken based on the comparison of some polynomial function $f(x_1, \ldots, x_n)$ with 0, where $x_1, \ldots, x_n$ are the real variables of the program. For any fixed value of n, one can imagine expanding an instance of an algorithm in the algebraic model into an algebraic decision tree. Using the algebraic decision tree, one can prove an $\Omega(n \log n)$ lower bound for any algorithm in the algebraic model that solves the *element uniqueness problem* in which we are given a set of n real numbers and must decide if any two are equal. Because the element uniqueness problem can be reduced

to many problems in computational geometry, this result is very useful for proving superlinear lower bounds on a variety of geometric problems, such as the closest-pair problem. Further discussion of this lower-bound technique is provided in [52].

1.2.2 Parallel and dynamic algorithms

When presented with a problem, the first algorithmic task is to develop a fast sequential method of solving it. This, however, is rarely the final word on the topic, because we may need to solve the problem in certain contexts for which the sequential algorithm is not well suited. Given a problem for which a fast sequential algorithm is known, two questions in particular remain. First, can we design an algorithm that takes advantage of many processors running simultaneously? Second, can we design an algorithm that does not merely accept one set of input and solve a single instance, but can instead maintain the solution to a problem with respect to small changes in the input?

Parallel algorithms

To answer the first question, we must design a parallel algorithm for the problem. For such an algorithm to be completely general, it should apply to a number of processors that grows with the input size, and should provide a speed-up that is roughly commensurate with the number of processors dedicated to the problem. Ideally, we would like to be able to get a speed-up factor of m by using m processors in parallel. This does not seem to be possible for all problems, so for a particular problem, we would like to see how close we can come to attaining this goal.

The parallel algorithms presented in this dissertation will usually require n processors working in parallel, where n is the input size. While such a high degree of parallelism is rarely attainable on existing architectures, the same algorithm can be run on fewer processors by using each processor to simulate the work of several processors. In contrast to the problem of increasing the number of processors, we can

always decrease the number of processors at a cost of a strictly proportional reduction in speed. Consequently, it is to our advantage to present parallel algorithms that use as many processors as possible, while guaranteeing that the total work (total number of time steps performed on all processors) is as close to the sequential time as possible.

As an additional simplification, we assume our algorithms are designed to run on a *parallel random access machine* (PRAM). This model consists of a set of standard random access machines that share a global memory space. This model is not considered realistic, because it allows constant-time communication between an arbitrary pair of processors. However, it is a useful abstraction that allows us to put aside issues such as the routing of information, while focusing on the problem of algorithm design. PRAMs are typically grouped into categories depending on whether access to a memory address is allowed to be *concurrent* (two or more processors have simultaneous access) or *exclusive* (only one processor at a time may have access). This classification is further refined by distinguishing read access from write access. In this dissertation, we will prefer the EREW (exclusive read/exclusive write) model because it requires the weakest assumptions. We will occasionally use the analogous CREW model when concurrent reads help in reducing the complexity. None of our algorithms will require concurrent writes.

An algorithm is considered highly parallelizable if it can be made to run in polylog time using a polynomial number of processors. For some problems, particularly those that seem to involve many sequential dependencies, it is interesting to ask whether such a speed-up is attainable even if the work (number of time steps totaled over all processors) we perform is significantly greater than the time for the best sequential algorithm. For the class of problems called P-Complete (see, for example [42]) it is conjectured that such a speed-up is impossible using any polynomial number of processors, and such algorithms are considered inherently sequential. Thus, for example, even if a given problem can be solved in linear time sequentially, it may give us considerable insight to discover that we can solve the same problem in $O(\log^3 n)$ time using $O(n^6)$ processors.

In this dissertation, however, we limit our scope to parallel algorithms whose work is within a constant, or at most a polylog factor of the best sequential algorithm, since these are the most likely to be useful in practice. Often, the techniques presented here are superior to previous techniques precisely because they permit the development of efficient parallel algorithms.

Dynamic algorithms

To answer the second question posed above, we must design a *dynamic algorithm*, or an algorithm that allows us to compute the effect of small changes to the input in much less time than it would take to recompute the problem from scratch. A very simple example of a problem that admits a fast dynamic algorithm is the *minimum-element* problem in which we must find the minimum of a totally ordered set of n elements. Given such a set, we must clearly examine every element before we can determine the element of minimum value, so the obvious linear-time algorithm for this problem is easily seen to be optimal.

The minimum-element problem becomes more interesting when we consider the dynamic version. In a dynamic algorithm, we must be able to *update* the input by modifying it in some minor way, and we must also be able to process *queries*, in which we compute the answer to some problem posed on the set. In the present case, an update consists of inserting or deleting a single element, and a query computes the minimum of the set. The dynamic version of this problem can be solved in $O(\log n)$ time per query and update, by using some form of *balanced search tree*. A standard source of such classical data structures is the textbook of Aho, Hopcroft, and Ullman [7].

There are two salient points to be made about the dynamic algorithm for maintaining the minimum element. First, the worst-case time for performing queries and updates, $O(\log n)$, is far less than the time it takes to recompute the minimum by examining all elements of the set. This is because much of the previous computation on the set is still valid; the dynamic algorithm computes only the effect of a small

change. Second, the time it takes to solve the problem by building up the set from the empty set using insertions is $O(n \log n)$. Hence, the time it takes to solve an instance of the problem using the dynamic algorithm is not too much greater than the time it would take to solve an instance of the problem as a batch, which is $O(n)$. Both of these properties are desirable in dynamic algorithms, although they are not attainable for all problems.

In this dissertation we consider the extension of our technique to permit the development of dynamic algorithms. Because our problems are geometric in nature, the problems we solve are more complex than maintaining the minimum element of a set, and the techniques we apply are necessarily more intricate. However, the algorithms we present will be efficient in both of the senses used above. That is, a single update and query can be performed with a low worst-case complexity, and as a result, the total work to solve an instance of a problem using dynamic operations will be within a polylog factor of the time it takes to solve the problem sequentially.

Another notion that appears in the analysis of dynamic algorithms is that of *amortized* complexity, in which we do not necessarily guarantee that any single update is fast, but must still bound the average time per operation over an entire sequence of updates and queries. This is a very useful concept, especially in cases for which worst-case complexity cannot be guaranteed. However, in this dissertation all the dynamic algorithms will be analyzed in terms of worst-case bounds.

1.3 Overview of related techniques

1.3.1 Quadtrees and other hierachical decompositions

The well-separated pair decomposition makes use of a hierarchical tree structure sharing many of the properties of the *quadtree*, a data structure with a long history in geometric computation and computer graphics. A survey of this structure and its uses can be found in Samet [61].

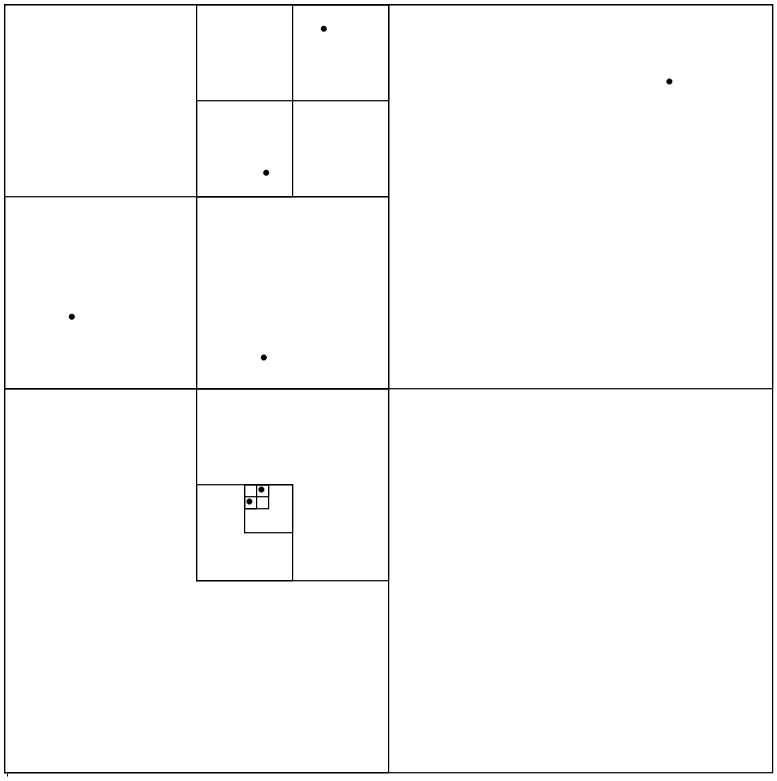


Figure 1.2: A quadtree in the plane.

The quadtree gets its name from its original application to planar point sets, although it generalizes naturally to higher dimensions. In the simplest case, we consider a point set lying in the unit square $[0,1] \times [0,1]$. To construct a quadtree of such a point set, we begin with a root node labeled by this square. The quadtree is then defined recursively as follows. For each node containing more than one point, we subdivide its square into four equal regions (hence the name quadtree). For the root node, these squares are $[0,\frac{1}{2}] \times [0,\frac{1}{2}]$, $[0,\frac{1}{2}] \times [\frac{1}{2},1]$, $[\frac{1}{2},1] \times [0,\frac{1}{2}]$, and $[\frac{1}{2},1] \times [\frac{1}{2},1]$. Any non-empty squares that result become the children of the subdivided node. An example is shown in figure 1.2.

The related three-dimensional subdivision in which cubes are split into eight regions is sometimes called an "octtree". Here, however, we will use "quadtree" to

denote an analogous tree in any fixed dimension d. While the quadtree provides a very natural approach to spatial decomposition, it suffers from the drawback that it may be arbitrarily large with respect to the point set for which it is constructed. That is, there may be a pair of very close points that can only be separated by constructing a long chain of squares, each having only one non-empty child. Note that in figure 1.2, this can be observed in the lower left quadrant of the root square. To develop efficient algorithms, we will need to define a structure that shares useful properties with the quadtree, but whose size is linear in terms of the number of points.

Clarkson [25] used the quadtree as the basis for his all-nearest-neighbors algorithm, which was the first algorithm to provide a substantial improvement over the complexity obtained by multi-dimensional divide-and-conquer [15]. In fact, our algorithm using the well-separated pair decomposition is similar to his in many ways. However, the potentially large size of quadtrees, noted above, made the time complexity of Clarkson's deterministic algorithm dependent on the ratio of the point-set diameter and the distance between the closest pair of points. He was able to eliminate this dependency only by adding randomization and non-standard operations, such as the floor function.

By moving away from a strict quadtree approach, Vaidya [67, 69] was able to develop an optimal $\Theta(n \log n)$ deterministic algorithm for all-nearest-neighbors using only the algebraic operations that are standard in computational geometry. Vaidya's refinement corresponds, in two dimensions, to shrinking the square that labels any node of the tree until it is the smallest square containing all points at that node. This insures that when a node is subdivided, it must result in more than one non-empty child. This procedure yields a structure much like a quadtree, but whose size is guaranteed to be linear in the total number of points.

For our decomposition, we relax the requirements of our tree still further. This will not be essential for sequential algorithms, which can be solved efficiently by constructing a tree like that of Vaidya's. However, the advantages of our approach will become apparent when we discuss parallel and dynamic versions of our technique.

An alternative method of decomposing d-dimensional point sets is the k-d tree [14]. In a k-d tree, a node is subdivided at each step by a hyperplane chosen to guarantee that there is roughly an equal number of points on either side of the subdivision. Thus, the number of points corresponding to a node is decreased by a constant factor from its parent, and the tree depth must be $O(\log n)$. By contrast, the depth of quadtree-like structures cannot be bounded below linear even when they have been modified to guarantee overall linear size. However, k-d trees lack an essential property of quadtree-like structures, which is that the geometric size of the square that bounds the points at some node of a quadtree tends to shrink rapidly with increasing depth. This property will be essential to the well-separated pair decomposition. In fact, one of the results of Chapter 4 will imply that there is no way to guarantee a well-separated pair decomposition of linear size using the k-d tree, or in fact, any tree of polylog depth.

1.3.2 Well-separated sets

The term "well-separated" as used in this dissertation is taken from Greengard and Rokhlin [37, 38, 56], who used it in their work on the fast multipole method. Two point sets are said to be well-separated if the diameter of each set is relatively small compared to the distance between these sets. Intuitively, this provides us with a very strong criterion by which two point sets can be regarded as distinct clusters. A more formal definition will be presented in the next chapter.

Essentially the same notion was used by Agarwal et al. [4] previously in the context of constructing Euclidean minimum spanning trees, though these authors use the term "strongly separated." The notion is itself quite natural, and has presumably appeared in many other contexts as well.

For example, Salowe uses the notion of a well-separated pair implicitly in Lemma 1 of [58], when discussing sufficient conditions for a set of edges to be a spanner of the complete Euclidean graph of a point set. This is a graph in which the

length of a path between any two points is approximately equal to Euclidean distance. For all u and v in a point set V, Salowe's condition (ii) states that there must exist an edge $\{x, y\}$ in the spanner such that

$$d(u, x) + d(y, v) \leq \gamma d(x, y), \;\; 0 < \gamma < \frac{1}{2}.$$

It follows that for a given $\{x, y\}$, the pair of sets A and B consisting of all such u, and all such v, respectively, are well-separated. We will later present a type of spanner that is similar, but defined more explicitly as the set obtained by introducing, for each well-separated pair $\{A, B\}$ in the decomposition, one edge $\{x, y\}$ such that $x \in A$ and $y \in B$.

Often, it is very helpful to know that sets A and B are well-separated when computing some function of the pairs $\{a, b\}$ such that $a \in A$ and $b \in B$. For example, one can show that an approximation of the potential field due to electrostatic charges placed at points in A has a small error bound for points in B. Also, it can be shown that for any $a \in A$, the nearest neighbor of a in $A \cup B$ cannot lie in B unless A is a singleton set, and that the Euclidean minimum spanning tree of any superset of $A \cup B$ has at most one edge $\{a, b\}$ such that $a \in A$ and $b \in B$.

Earlier, it was noted that the all-nearest-neighbors algorithms of Clarkson [25] and Vaidya [67, 69] bear similarity to the sequential algorithm that will be presented in Chapter 2. As with much other previous work, these algorithms can be viewed as making implicit use of several properties of well-separated sets. Nevertheless, the explicit definition of a well-separated pair decomposition seems quite useful as a technique for developing such algorithms. Often, it allows us to put aside the details of a particular algorithm and focus on more pertinent geometric properties. This may lead to a more refined analysis of the problem, ultimately giving us a faster algorithm, as, for example, in the algorithm we present for computing approximate Euclidean minimum spanning trees.

1.4 Applications

The well-separated pair decomposition provides us with a versatile technique for solving many problems posed on multi-dimensional point sets. The problems treated in this dissertation fall into several natural classes that are described below.

1.4.1 Proximity problems

In a proximity problem, we are interested in computing neighbors of points in P, as determined by Euclidean distance. The most natural problem of this kind is the all-nearest-neighbors problem, in which we must compute, for each $a \in P$, that $b \in P - \{a\}$ that is nearest to a. This is often generalized to the k-nearest-neighbors problem in which we must compute the k nearest points in $P - \{a\}$ for some $k \leq n - 1$. A simpler variant is the closest-pair problem. In this problem, we must report only the pair of points $\{a, b\}$ that minimizes Euclidean distance over all pairs of distinct points. While it is clearly reducible to the all-nearest-neighbors problem, this problem will be useful for showing lower bounds on the time to construct our decomposition in the algebraic model. It will also be of interest when we develop dynamic algorithms using the well-separated pair decomposition. Other proximity problems include enumerating the m smallest interpoint distances, and the related problem of computing neighbors according to a fixed cutoff radius r. We will show how the well-separated pair decomposition can be used to develop efficient algorithms for these problems.

The first subquadratic algorithm for solving the all-nearest-neighbors problem was presented by Bentley [15] and used multi-dimensional divide-and-conquer. As noted in the preceding section, Clarkson later presented a faster algorithm to solve the all-nearest-neighbors problem using a quadtree-like structure. This was refined by Vaidya [67, 69], who presented an optimal sequential algorithm for the all-nearest-neighbors problem in the algebraic model. Fast sequential algorithms for enumerating the m smallest interpoint distances have been presented by Salowe [60], and Lenhof

and Smid [47].

One important problem left open was to compute all nearest neighbors of a point set in parallel. While multi-dimensional divide-and-conquer lends itself to a fairly direct parallelization and can thus be performed with n processors in polylog time, the exponent depends on d. Approaches using quadtrees, by contrast, suffer from the fact that the tree itself can be very deep, making a straightforward parallelization impossible. In the plane, optimal parallel algorithms have been developed using the Voronoi diagram [27, 28]. Using approaches quite different from any discussed here, Frieze, Miller and Teng [33, 48] presented the first randomized parallelization of the all-nearest-neighbors problem in which the exponent in the log factor was independent of d, and ultimately presented an optimal $O(\log n)$ time randomized algorithm in a somewhat non-standard PRAM model. An important contribution of this dissertation, presented earlier in [18, 19], is an optimal $O(\log n)$ time deterministic CREW PRAM algorithm for the all-nearest-neighbors problem. This algorithm employs an approach similar to Vaidya's, but defined in terms of the well-separated pair decomposition. Because a tree of up to linear depth must be constructed in logarithmic time, the parallelization is non-trivial.

1.4.2 Spanning graphs

In the bichromatic-closest-pair problem, we are given a set of points colored red and blue, and we must report the closest pair of differently-colored points. Surprisingly, the techniques used to solve the proximity problems mentioned above do not appear to lend themselves to this new problem. In fact, it has been shown [4] that this problem is closely related to computing the Euclidean minimum spanning tree (EMST) of a point set. The latter is a natural instance of a class of problems in which we must find a spanning graph of the complete Euclidean graph satisfying certain properties. Another such problem is to construct a *spanner*, or approximate complete Euclidean graph: a sparse graph in which the distance between points is a close approximation

to Euclidean distance. Despite the increased difficulty of this class of problems, the well-separated pair decomposition can be applied. Unlike its application to proximity problems, the total complexity will in general be dependent on an approximation factor ϵ.

The connection between EMST and other problems involving point set interdistances (i.e., those distances determined by pairs of distinct points) was first presented by Yao in his seminal paper [71] on the topic. Yao's paper also contains the first $o(n^2)$ algorithm for constructing the EMST of point sets in three or more dimensions. The EMST can be constructed optimally for planar point sets using the Voronoi diagram, but the lower bound on the complexity is not known for $d \geq 3$.

Vaidya [66, 68] presented the first fast algorithm to compute an approximate minimum spanning tree. The sequential time complexity for Vaidya's algorithm is $O(\epsilon^{-d} n \log n)$. By posing this problem in terms of the well-separated pair decomposition, we are able to refine the analysis considerably, and we present an algorithm with complexity $O(n \log n + (\epsilon^{-d/2} \log \frac{1}{\epsilon})n)$.

The problem of computing a Euclidean spanner has been considered by many researchers [24, 31, 44, 57, 58, 59, 70]. In [57], Ruppert and Seidel present an $O(\epsilon^{-d+1} n \log^{d-1} n)$ algorithm for the problem of computing a spanner with $O(\epsilon^{-d+1} n)$ edges. Salowe [58, 59] and Vaidya [70] have presented algorithms to compute somewhat larger spanners, but in less time. Using the well-separated pair decomposition, it is possible to compute a spanner that is comparable in size to that of Ruppert and Seidel [57] but in time $O(n \log n + (\epsilon^{-d} \log \frac{1}{\epsilon})n)$, which is with an $\epsilon^{-1} \log \frac{1}{\epsilon}$ factor of the optimal complexity.

1.4.3 Potential computation

A final problem that can be treated using the well-separated pair decomposition is that of potential computation. Unlike the previous classes, this problem is not purely geometric in nature. In its most general form, each point a represents the position of

a particle, and we are given a potential function $\phi_a(x)$ that associates a real number with every position x in the space around a, decreasing with distance from a. Any set of particles generates a potential field that is computed as the sum of the potential functions of each particle. We wish to evaluate, for each a, the potential field at a due to all $b \in P - \{a\}$. This can clearly be done in quadratic time by computing such sums directly.

We will show how the well-separated pair decomposition can be used to implement the fast multipole method of Greengard and Rokhlin [37, 38, 56], which approximates such potentials to high precision. Other methods for solving this problem using clustering techniques have been presented in [8, 13], and such techniques fall under the general classification of hierarchial n-body methods [40].

All such methods rely on the fact that the effect of a small cluster of charges on a point whose distance is high compared to diameter of the cluster (and therefore well-separated) can be approximated very accurately by a function whose complexity is independent of the number of charges in the cluster. The simplest such approximation is the *monopole* approximation, in which the effect of a cluster is approximated by a single point charge of the same total charge as those within the cluster. Appel [8] implemented a useful many-body simulation using only the monopole approximation. For his algorithm, he observed that the error term can be reduced substantially by positioning the one-charge approximation at the center of mass of the cluster.

The multipole approximation generalizes the monopole approximation so that the effect of the cluster can be approximated to arbitrarily high precision by increasing the number of terms in a truncated series. The precision depends on the number of terms, but is independent of the number of charges. These higher order approximations can be used in conjunction with the Barnes and Hut algorithm [13].

In the Barnes and Hut algorithm, a box decomposition like the quadtree is constructed for the set of charges, and a function is computed for each box to approximate its effect on points sufficiently far from this box. To approximate the potential at a point charge, one computes the total effect of the field due to a set

of boxes covering all other point charges. These boxes are allowed to grow with increasing distance from the point, and under assumptions about the distribution of charges that are reasonable in practice, one can show that the number of boxes needed to compute the potential at any given point is $O(\log n)$. This approach is often put forward as an $O(n \log n)$ algorithm for potential computation, although the analysis is distribution dependent.

The Barnes and Hut algorithm is asymptotically slower than the fast multipole method by a $\log n$ factor. In fact, the fast multipole method is essentially the same algorithm but with the computation factored out more carefully. The Barnes and Hut algorithm is, however, easier to implement, and is still considered of practical significance.

1.5 Dissertation outline

In Chapter 2, we give a precise definition of the well-separated pair decomposition and present optimal sequential algorithms for constructing both the tree and the set of well-separated pairs. We also prove that the number of well-separated pairs is linear in n, a fact that will be crucial to all applications. Later results will build upon the techniques of this chapter, and it is essential to the understanding of this dissertation.

In Chapter 3, we consider the application of the well-separated pair decomposition to a diverse set of problems. These applications cover all three of the categories described in the preceding section. In addition to providing efficient algorithms, this chapter illustrates a general methodology for applying the well-separated pair decomposition to any problem that can be posed in terms of the interpoint distances of a point set. All of these applications have received much attention from other researchers, and we show how the well-separated pair decomposition leads to faster algorithms than those obtained previously.

In Chapter 4, we study the decomposition itself more closely and derive

a number of combinatorial properties, several of which are directly applicable to the dynamic versions of our algorithms. These results shed light on the structure inherent in the well-separated pair decomposition, and justify its formulation both as a hierarchical decomposition of a point set, and as a partition of the set of distinct pairs into geometric classes.

In Chapter 5, we consider the problem of constructing the decomposition in parallel, for which we provide an optimal CREW algorithm. We begin with an efficient, but not optimal, algorithm for tree construction. We then develop the optimal algorithm as a refinement of the first technique. These algorithms require us to construct nodes occurring deep in the tree without reference to their ancestors, demonstrating the advantage of the flexible characterization of our tree. We also use the decomposition to improve known parallel time and work bounds for a number of important applications.

In Chapter 6, we make use of additional properties of the well-separated pair decomposition to develop methods of maintaining the decomposition dynamically along with computations expressed in terms of it. These methods demonstrate the full power of our approach by requiring us to exploit more of the structure of the well-separated pair decomposition than in any other contexts considered. We apply this method to the efficient maintenance of n-body potential calculations, as well as to the fundamental geometric problem of maintaining the closest pair of a point set under insertions and deletions.

Finally, in Chapter 7, we present the results of empirically testing a C implementation of our sequential algorithm to compute the decomposition. By performing such tests, we can more accurately assess the geometric constants in the complexity of our algorithms, for which our theoretical bounds are too pessimistic to be useful. In addition, we consider some of the tradeoffs between naive quadratic algorithms and algorithms that use the well-separated pair decomposition. In practice, the fastest implementations will use both of these approaches, depending on the size of the subproblem to be solved. We present an application-independent formulation of this

trade-off, and empirically determine parameters needed for tuning an implementation to optimize its performance.

Chapter 2

Sequential algorithms

In this chapter, we present sequential algorithms for constructing the well-separated pair decomposition. These algorithms were originally presented in [19, 20]. Compared to the parallel and dynamic extensions we will later present, these algorithms have the great advantage of simplicity with respect to both analysis and implementation. Subsequent variations of the algorithms will require the results of this chapter, so it is essential to the understanding of the dissertation. This chapter also contains a proof of the linear size of the decomposition, which is necessary to guarantee efficiency.

2.1 Definitions

Let P be a set of points in $\Re^d$, where d is a constant denoting the dimension. We define the *bounding rectangle* of P, denoted by $R(P)$, to be the smallest rectangle that encloses all points in P, where the word "rectangle" denotes any Cartesian product $R = [x_1, x_1'] \times [x_2, x_2'] \times \cdots \times [x_d, x_d']$ in $\Re^d$. We use the term *open rectangle* to denote the product of open intervals.

We denote the length of R in the ith dimension by $l_i(R) = x_i' - x_i$. We denote the maximum and minimum lengths by $l_{\max}(R)$ and $l_{\min}(R)$. When all $l_i(R)$ are equal, we say that R is a *d-cube*, and denote its length by $l(R) = l_{\max}(R) = l_{\min}(R)$. We write

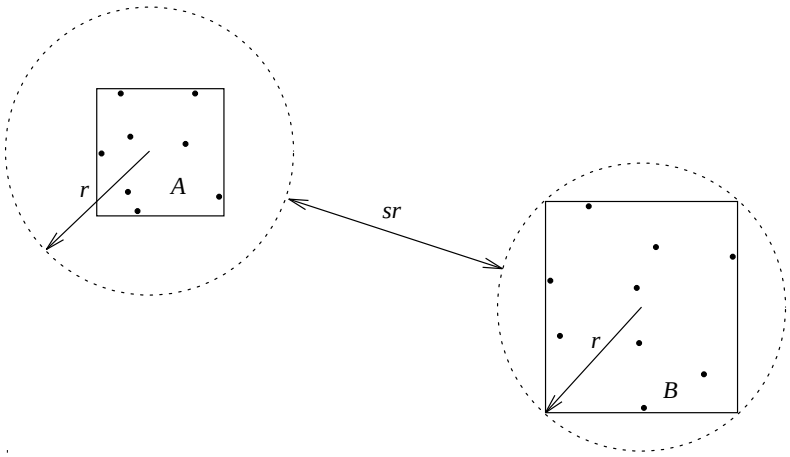


Figure 2.1: Well-separated sets in the plane

$l_i(P)$, $l_{\min}(P)$, and $l_{\max}(P)$ as shorthand for $l_i(R(P))$, $l_{\min}(R(P))$, and $l_{\max}(R(P))$, respectively. We define a d-ball of radius r to be the set of all points in d dimensions at a distance less than or equal to r from some point, referred to as the *center* of the d-ball.

We use $d(x, y)$ to denote the Euclidean distance between points x and y. We extend this to point sets X and Y so that $d(X, Y)$ refers to the distance between two points $x \in X$ and $y \in Y$ for which $d(x, y)$ is minimized. X and Y need not be finite point sets, but they will always be defined in a manner (for example, with closed boundaries) to insure that such a minimum distance exists.

We say that point sets A and B are *well-separated* if $R(A)$ and $R(B)$ can each be contained in d-balls of radius r whose minimum distance is at least sr, where s is the *separation*, assumed to be fixed throughout our discussion at a value strictly greater than 0 (see figure 2.1). While we can define this notion in a somewhat more natural way without using bounding rectangles, this definition will make subsequent computation easier.

We will often assume that P has a binary tree T associated with it, where each leaf of T is labeled by a singleton set containing one of the points in P, and

each internal node is labeled by the union of all sets labeling the leaves of its subtree. We will refer to each node in T by the name of the set labeling it. Since leaves are labeled by singleton sets, we may also refer to them by the names of points.

We define the *interaction product*, denoted $\otimes$, between any two point sets A and B as follows:

$$A \otimes B = \{\{p, p'\} | p \in A, p' \in B, \text{and } p \neq p'\}.$$

Note that $P \otimes P$ is the set of all distinct unordered pairs of points in P. The interaction product is essentially the Cartesian product adapted to unordered pairs.

The set $\{\{A_1, B_1\}, \ldots, \{A_k, B_k\}\}$ is said to be a *realization* of $A \otimes B$ if

r1. $A_i \subseteq A$ and $B_i \subseteq B$ for all $i = 1, \ldots, k$.

r2. $A_i \cap B_i = \emptyset$ for all $i = 1, \ldots, k$.

r3. $(A_i \otimes B_i) \cap (A_j \otimes B_j) = \emptyset$ for all i, j such that $1 \leq i < j \leq k$.

r4. $A \otimes B = \bigcup_{i=1}^{k} A_i \otimes B_i$.

The realization is said to be *well-separated* if it satisfies the additional property

r5. A_i and B_i are well-separated for all $i = 1, \ldots, k$.

Let T be a binary tree associated with P. For $A, B \subseteq P$, we say that a realization of $A \otimes B$ *uses* T if all A_i and B_i in the realization are nodes in T. We define a *well-separated pair decomposition* of P to be a structure consisting of a binary tree T associated with P, and a well-separated realization of $P \otimes P$ that uses T.

Agarwal et al. [4, 5] have also used the notion of partitioning the set of pairs of a point set into pairs of sets satisfying certain geometric criteria. In their formulation, the sets must lie in cones, each of some small angle α, whose axes are parallel and which intersect only at the apex. This decomposition does not guarantee that each pair in $P \otimes P$ is *uniquely* represented by a pair in the decomposition, and

does not incorporate the notion of separation of clusters based on distance.[1] Both of these properties are essential for the application to the fast multipole method. This is because the influence of each charge must be taken into account exactly once in computing the potential at some point, and the convergence of a truncated multipole expansion depends on distance-based separation. These properties appear less critical with respect to purely geometric problems, but they may prove useful in future applications.

2.2 Computing a fair split tree of a point set

Given a point set P of size n, we would like to construct a binary tree T, such that there exists a linear size well-separated realization of $P \otimes P$ that uses T. In this section, we define a type of tree that, we will later show, permits an $O(n)$ size well-separated realization. We show how to construct such a tree in $O(n \log n)$ time sequentially.

We define a *split* of P to be its partition into two non-empty point sets lying on either side of a hyperplane (called the *splitting hyperplane*) perpendicular to one of the coordinate axes, and not intersecting any points in P.

We define a *split tree* of P to be a binary tree, constructed recursively as follows. If $|P| = 1$, its unique split tree consists of the node P. Otherwise a split tree is any tree with root P and two subtrees that are split trees of the subsets formed by a split of P. For any node A in the tree, we denote its parent (if it exists) by $p(A)$.

[1] However, they use this decomposition to construct a set of well-separated pairs for which each edge in the Euclidean minimum spanning tree has a (not necessarily unique) representative.

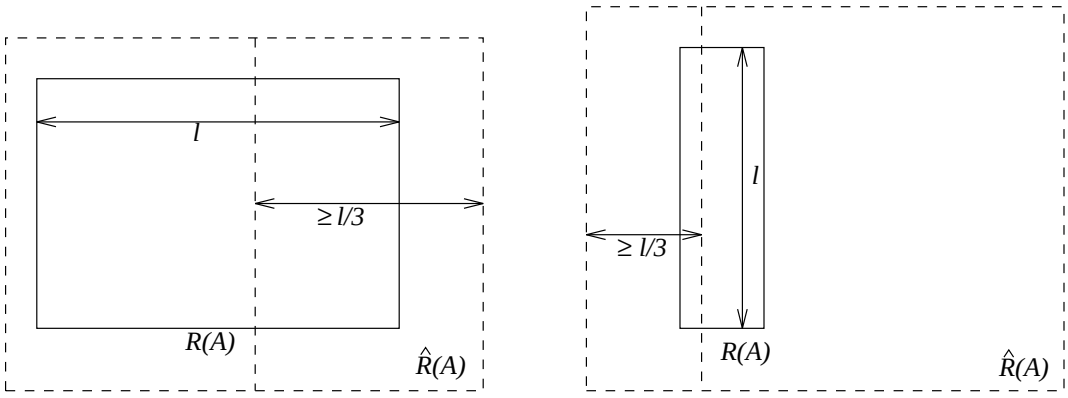


Figure 2.2: Two examples of fair splits

We define the *outer rectangle* of A, denoted $\hat{R}(A)$, for each node A top down as follows:

- For the root P, let $\hat{R}(P)$ be an open d-cube centered at the center of $R(P)$, with $l(\hat{R}(P)) = l_{\max}(P)$.
- For all other A, the splitting hyperplane used for the split of $p(A)$ divides $\hat{R}(p(A))$ into two open rectangles. Let $\hat{R}(A)$ be the one that contains A.

We define a *fair split* of A to be a split of A in which the splitting hyperplane is at a distance of at least $l_{\max}(A)/3$ from each of the two boundaries of $\hat{R}(A)$ parallel to it (see figure 2.2). A split tree formed using only fair splits is called a *fair split tree*.

We present an algorithm to find a fair split tree for any point set. This algorithm will run in time $O(n \log n)$, which is optimal, as we will later prove.

We first present a high level description of the algorithm, and then show how to implement it efficiently. Let P be the given point set. If $|P| = 1$, return a tree consisting of a single node labeled P. Otherwise, consider a hyperplane perpendicular to the longest dimension of $R(P)$ that divides it into two geometrically equal halves.[2]

[2] We make the non-degeneracy assumption that no points lie *on* this hyperplane. It is easy to

This hyperplane lies at a distance of $l_{\max}(A)/2$ from the nearest parallel boundaries of $R(A)$. Hence, it defines a fair split of P into two non-empty sets P_1 and P_2. We find fair split trees for these sets, and return a tree with root P, having two subtrees that are the split trees of P_1 and P_2.

Note that the fair splits defined above satisfy a stronger condition than we require. The relaxed condition will become important when we present our parallel algorithm, so we will use it in all our analysis. However, the constants in the size of the realization can be improved by assuming the stronger condition.

Unfortunately, each split can result in all but one point going to one side, so a naive implementation could have quadratic complexity. We can improve the complexity, if, instead of computing a single fair split at each recursive call, we compute an entire sequence of splits. Let a *partial fair split tree* be a tree satisfying all the conditions of a fair split tree, except that the point sets corresponding to its leaves need not be singletons. We will show how in linear time, assuming the points have been presorted, we can construct a partial fair split tree in which each leaf corresponds to a point set of no more than half the size of the point set at the root.

Vaidya [67, 69] has previously shown how to construct a fair split tree in the same time complexity as we will attain. Our algorithm is, however, simpler both in implementation and analysis. First, it requires no auxiliary structures other than doubly linked lists, whereas Vaidya's algorithm maintains a binary search tree to represent the point set at a given node. In addition, the main step of Vaidya's algorithm is to split an individual node in the tree into its children, while maintaining sorted order on the resulting point sets. The time to perform such an operation for a given node depends upon the structure of the fair split tree, and the overall bound on complexity follows from a tree weighting lemma. In contrast, we do not attempt to bound the time for splitting an individual node; instead, we develop a splitting phase in which an entire batch of splits is performed efficiently. The result is a simplified analysis that does not require a tree weighting lemma.

---

adapt the algorithm to eliminate the need for this assumption.

2.2.1 The splitting phase

We present a splitting phase that will be used as a subroutine by our main divide-and-conquer algorithm. The splitting phase takes as input a point set sorted in every dimension, and outputs a partial fair split tree of the point set, along with a partition of the point set into sorted subsets corresponding to the leaves of the partial fair split tree.

Let P_0 be a point set for which we are to perform a sequence of splits. For $i = 1, \ldots, d$, let L_i^0 denote a doubly-linked list of all points in P_0, sorted in increasing order by coordinate i. Assume that for every point, we have cross-references allowing us to retrieve its place in each of the lists. We make copies of $L_1^0, \ldots, L_d^0$ for later reference, since these lists will be modified when computing the sequence of splits, and will be needed again in the final step of the splitting phase.

Our method will be to perform a sequence of fair splits, each as given in the high level description presented earlier. After each split, we will obtain two non-empty point sets, and we will continue to split the set containing the most points, breaking ties arbitrarily. Let the point set that results after j splits be denoted P_j and let the corresponding sorted lists be denoted $L_1^j, \ldots, L_d^j$. We will show how to maintain these lists in linear time during the splitting phase.

We consider how to perform the jth fair split. Let $i_{\max}$ denote the longest dimension of $R(P_{j-1})$. Clearly, the jth fair split divides the list $L_{i_{\max}}^{j-1}$ into two contiguous pieces. As in Vaidya's tree-construction algorithm [67, 69], we determine the split location in $L_{i_{\max}}^{j-1}$ using a linear search that alternates between both ends of the list, heading towards the middle. The search time is proportional to the size of the smaller set that results from the split.

We now delete all of the points in the smaller set from all of the lists $L_1^{j-1}, \ldots, L_d^{j-1}$. These points can be obtained by traversing $L_{i_{\max}}^{j-1}$ beginning at the end closest to the split location. The lists resulting from these deletions are the sorted lists $L_1^j, \ldots, L_d^j$, containing the points in the larger set, P_j, that results from the split.

The time for deletion, like the search time, is proportional to the size of the smaller set.

The deleted points must not be discarded, since we will need to consider them in a later splitting phase. At the completion of the splitting phase, we must output lists $\hat{L}_1^j, \ldots, \hat{L}_d^j$ containing all of the points deleted after the jth split. It will not be efficient to construct these lists yet, so instead we initialize empty lists, and mark each deleted point with a pointer back to each list $\hat{L}_1^j, \ldots, \hat{L}_d^j$ without inserting it. We will show how, as a final step in the splitting phase, we can insert the deleted points into these lists so that they are sorted.

We repeat the above splitting process on $L_1^j, \ldots, L_d^j$ until $|P_j| \leq |P_0|/2$. Let k denote the final value of j. As we perform the sequence of splits, we construct a partial fair split tree consisting of a chain starting with a node corresponding to P_0 and ending with a node corresponding to P_k. For $j = 1, \ldots, k$, we insert a sibling to P_j that corresponds to the smaller set of points (i.e., $P_{j-1} - P_j$) that resulted from the jth split. One can easily verify that the resulting tree is a partial fair split tree. Note that the total amount of work is proportional to the number of points deleted, and hence is linear.

After completing the above sequence of splits, we must insert points into all of the lists $\hat{L}_1^j, \ldots, \hat{L}_d^j$ for $j = 1, \ldots, k$ so that these lists are in sorted order. We do this in d passes as follows. For each coordinate i, we return to the original list L_i^0, which we copied prior to the deletions, and traverse it in increasing order. For each point we encounter, we append it to the end of the list given by the pointer that was set at the time of deletion. After the ith pass, $\hat{L}_i^j$ contains all the points in $P_{j-1} - P_j$ in the same order in which they appeared in L_i^0, guaranteeing that $\hat{L}_i^j$ sorted. Because d is constant, this phase takes linear time.

One can see that the above splitting phase takes as input a set of sorted lists and computes a partial fair split tree for this set, as well as an explicit partition of the original point set into sorted lists representing the sets at each of the leaves. We use this phase as a step in the divide-and-conquer algorithm for computing a fair

split tree.

2.2.2 The main algorithm

We are now ready to describe the main algorithm. This algorithm is defined recursively, but before we invoke the recursive procedure for the first time, we must presort the points in increasing order by each coordinate, and construct the sorted lists that we will pass to the first invocation of the splitting phase. Because the splitting phase maintains sorted order in the partition it computes, no explicit sorting will be necessary within the recursive procedure.

For the recursive procedure, we assume we are given a set of sorted lists $L_1^0, \ldots, L_d^0$ representing a point set P for we which must return a fair split tree. We perform a splitting phase on these lists, and obtain a partial fair split tree, and a set of sorted lists for each leaf in the partial fair split tree. For each leaf in the partial fair split tree that does not correspond to a singleton, we use the sorted lists at that leaf recursively to compute a fair split tree of the point set at that leaf, and replace that leaf with the corresponding tree. The result is a fair split tree of P.

Theorem 2.2.1: *Given a set P of n points in $\Re^d$, a fair split tree of P can be constructed in $O(n \log n)$ time.*

Proof. First note that the presorting step can be performed in $O(n \log n)$ time. To analyze the recursive part of the algorithm, we group the invocations of the procedure according to the *level* of recursion. That is, the level of the first invocation is defined to be 0, and the level of any recursive invocation at level i is $i+1$. One can see that the maximum level of recursion is $O(\log n)$, because the size of the point set decreases by a factor of at least two with each recursive call.

We now consider the total work performed over all invocations at level i. It suffices to consider the work performed during all splitting phases that occur at this level, because the only additional work involves replacing leaves of the partial fair split tree with fair split trees.

Note that the point sets occurring in the recursive calls of an invocation of the recursive procedure form a partition of the point set passed to that invocation. Hence, the point sets occurring over all splitting phases at level i form a partition of P. We have shown above that the time of a splitting phase is linear in the number of points. Hence the amount of time for invocations at level i is $O(n)$. It follows that the total amount of time for invocations at all levels is $O(n \log n)$. □

Note that as we generate each node A in the split tree T we can compute $R(A)$ in $O(d)$ steps, using the sorted lists we maintain. This property will hold for the parallel algorithm as well, so we assume that any solution of this problem includes a copy of $R(A)$ at each node A in T. For this reason, we will assume in subsequent sections that in $O(1)$ time, we can construct the smallest d-ball containing $R(A)$ for any A in T. This will allow us to test whether two nodes are well-separated in constant time.

2.3 Computing a well-separated realization

Suppose we have already computed a fair split tree T for the point set $P = \{p_1, \ldots, p_n\}$. We present an algorithm to compute a well-separated realization of $P \otimes P$, denoted by $\{\{A_1, B_1\}, \ldots, \{A_k, B_k\}\}$, that uses T. The algorithm will run in $O(k)$ steps, which we will later prove is linear with respect to n. Note that $\binom{n}{2}$ is a trivial upper bound on k, since $P \otimes P$ has the well-separated realization $\{\{\{p_i\}, \{p_j\}\} | 1 \leq i < j \leq n\}$.

To make our algorithm precise, it will be useful to define the following ordering relation between nodes in T. Let $\prec$ denote a relation between nodes A and B in T such that $A \prec B$ if either (i) $l_{\max}(A) < l_{\max}(B)$, or (ii) $l_{\max}(A) = l_{\max}(B)$ and A precedes B in postorder (we assume some lexical ordering on the children of each node in T so that there is a unique postorder traversal of T). Note that either $A \prec B$ or $B \prec A$ unless A and B are identical. Moreover, $A \prec p(A)$ for all nodes A in T. The relation $\preceq$ is defined analogously, so that $A \preceq B$ if $A \prec B$ or $A = B$.

Note that every internal node of T has exactly two children, which we will

call A_0 and B_0 for purposes of the algorithm. For each node, we will construct a well-separated realization $\{\{A_1, B_1\}, \ldots, \{A_l, B_l\}\}$ of $A_0 \otimes B_0$. We can easily prove that the union of all such realizations is a well-separated realization of $P \otimes P$. We use the following procedure to construct each such realization.

Let A and B be nodes in T (initially $A = A_0$ and $B = B_0$). If A and B are well-separated, we return $\{\{A, B\}\}$. Otherwise, we swap A and B if necessary to insure that $B \prec A$. It must be the case that $|A| > 1$, because if not, $l_{\max}(A) = l_{\max}(B) = 0$, and A and B are well-separated. Hence, node A has two children, which we call A_1 and A_2. We observe that the subtrees rooted at these nodes are fair split trees of A_1 and A_2 and we associate them with the sets. We recursively construct well-separated realizations of $A_1 \otimes B$ and $A_2 \otimes B$, and return the union as our result.

Termination of this procedure follows from the fact that $|A_1||B|, |A_2||B| < |A||B|$. If $|A||B| = 1$, then $|A| = |B| = 1$, so A and B must be well-separated. Hence, we must reach the termination condition at some finite recursive depth. Note, however, that $|A||B| = 1$ is not necessary for termination, and in fact our algorithms will benefit most from well-separated pairs $\{A, B\}$ such that $|A||B|$ is large.

We use the fact that $A_1 \otimes B \cup A_2 \otimes B = A \otimes B$ to show that the procedure above outputs a well-separated realization of $A \otimes B$. As a base case, note that if A and B are well-separated, then $\{A, B\}$ is a trivial well-separated realization of $A \otimes B$. Otherwise, A and B are not well separated, and we can assume inductively that the recursive calls produce well-separated realizations of $A_1 \otimes B$ and $A_2 \otimes B$. It follows that the union of these realizations is a well-separated realization of $A \otimes B$. Let **findpairs** denote the algorithm in which we perform this procedure on all pairs of siblings, and return the union of all the realizations computed.

Lemma 2.3.1: *Procedure* **findpairs** *computes a well-separated realization of* $P \otimes P$.

Proof. First, one can easily verify that the set of pairs computed satisfies conditions r1, r2, and r5. To show that conditions r3 and r4 are satisfied, we must prove that every pair of points in P is represented by exactly one well-separated pair.

To see this, consider any pair of distinct points $\{a, b\}$ such that $a, b \in P$. There is a unique pair of siblings A_0 and B_0 in T such that $a \in A_0$ and $b \in B_0$. That is, A_0 and B_0 must be the children of the lowest common ancestor of the leaves $\{a\}$ and $\{b\}$.

Consider the set of well-separated pairs computed by **findpairs**. We have shown above that the procedure invoked for A_0 and B_0 computes a realization of $A_0 \otimes B_0$. Hence, there is a unique pair $\{A, B\}$ in this realization such that $a \in A$ and $b \in B$. Similarly, one can verify that there is no such pair in the realization computed for any other pair of siblings. Condition r4 is equivalent to stating that every pair of distinct points is represented by some well-separated pair, and condition r3 is equivalent to stating that this representation is unique. Therefore, we have shown that r3 and r4 are satisfied, guaranteeing that **findpairs** computes a realization. □

If we unravel each instance of the procedure into a computation tree, we can bound the time complexity of **findpairs** as the sum of the sizes of trees over all invocations of the procedure. We formalize this notion as follows:

Consider an invocation of the procedure to find a well-separated realization of $A_0 \otimes B_0$, where A_0 and B_0 are siblings in T. We define the *computation tree* of (A_0, B_0) to be a tree with root labeled (A_0, B_0) that satisfies the following. For any node labeled (A, B), let A' and B' be the unique pair of nodes that satisfies $\{A', B'\} = \{A, B\}$ and $B' \prec A'$, and let A'_1 and A'_2 denote the children of A' in T if they exist. Then if A and B are not well-separated, (A, B) has two children (A'_1, B') and (A'_2, B') in the computation tree. Otherwise, (A, B) is a leaf. It is easy to verify that this computation tree correctly captures all recursive invocations of the procedure to compute the realization of $A_0 \otimes B_0$.

Then, the time complexity of **findpairs** is linear in the total number of leaves of all such computation trees, corresponding to the number of pairs output. Thus, we have the following observation.

Observation 2.3.2: *The algorithm* **findpairs** *runs in time linear in the size of the realization it computes.*

2.4 The canonical realization

The definition of a well-separated pair decomposition places no particular restrictions on the well-separated realization nor, indeed, on the decomposition tree itself. Nevertheless, just as we will typically assume that the decomposition tree is a fair split tree, we will typically assume that the realization is the one produced by the algorithm **findpairs**. We will refer to this as the canonical realization with respect to the tree T.

For many applications, it will suffice to use any realization of linear size, but for others we may need stronger properties that hold only for the canonical realization. So far, we have defined this realization in terms of the algorithm to construct it. While this is useful, among other things, for showing it satisfies property r3 of the definition of a realization, there are advantages to having an explicit, non-algorithmic characterization. In this section, we present such a characterization, and prove its equivalence to the algorithmic definition of the preceding section.

Let T be a binary tree associated with a point set P. We define a *canonical pair* of nodes $A, B \in T$ to be any ordered pair (A, B) that satisfies the following conditions:

p1. A and B are well-separated.

p2. $p(A)$ and B are not well-separated.

p3. $B \prec p(A) \preceq p(B)$

An example is given in figure 2.3.

Note that the set of canonical pairs is uniquely defined for any T with a unique postorder traversal. We define the *canonical realization* of $P \otimes P$ with respect to T to be the set of all unordered pairs $\{A, B\}$ such that (A, B) is a canonical pair of nodes in T. In the next section we will prove that this set of pairs is a realization.

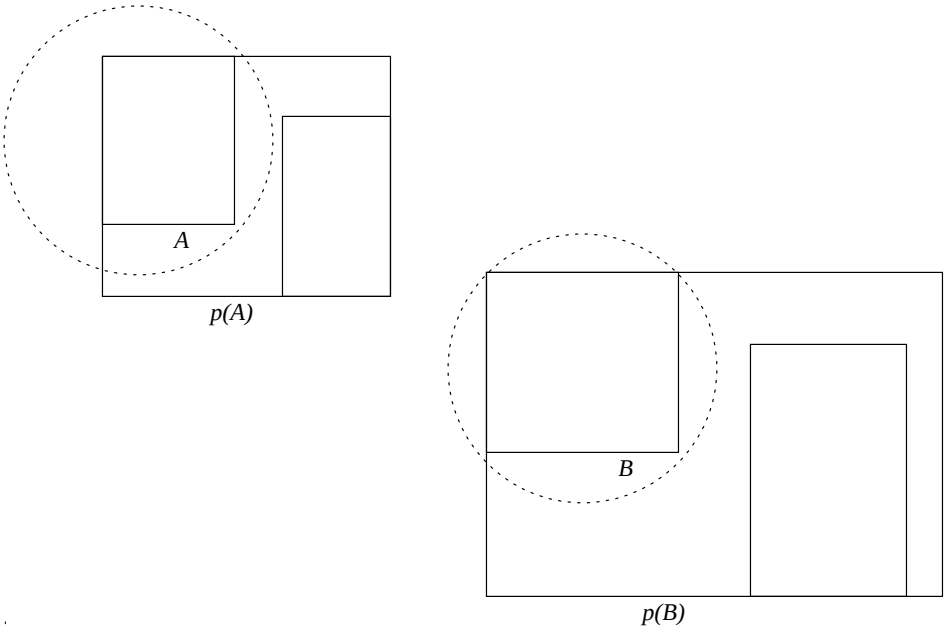


Figure 2.3: A canonical pair (A, B).

2.5 Procedure findpairs computes the canonical realization

Recall that **findpairs** can be unraveled into a collection of computation trees, as previously defined, each having some pair of siblings in T as its root. Let A and B be nodes in T whose least common ancestor has A_0 and B_0 as its children. Then the pair (A, B) (unless it is the root (A_0, B_0)) will denote a node in the computation tree of (A_0, B_0) corresponding to a recursive call at which we have obtained A by splitting $p(A)$ in the previous step.

Lemma 2.5.1: *The pair (A, B) is a node in the computation tree with root (A_0, B_0) iff A and B satisfy conditions p2 and p3.*

Proof. First, we verify that all nodes in the computation tree satisfy these conditions. It is obvious that the claim holds for the special case in which $A = A_0$ and $B = B_0$,

since $p(A_0) = p(B_0)$. With the exception of the root node, just considered, the node (A, B) was derived by splitting $p(A)$ in a pair containing $p(A)$ and B. Hence, it must hold that $p(A)$ and B are not well-separated, guaranteeing that condition p2 is always satisfied, and also that

$$B \prec p(A). \tag{2.1}$$

It remains to prove that p3 is satisfied for pairs other than the root. Assume w.l.o.g. that $B_0 \prec A_0$. Then A_0 appears only at the root of the computation tree, because it is split at the first step. B_0 can appear at a later node but we cannot have $A = B_0$, because A is the result of a split. If $B = B_0$, then clearly, $p(A) \prec p(B)$, because $p(A)$ must be a descendant of $p(B) = p(B_0)$. Otherwise, it must hold that $A, B \notin \{A_0, B_0\}$. Then consider the step in the procedure at which we split $p(B)$. It must have been true that $A' \prec p(B)$, where A' denotes the node paired with $p(B)$ at this step. But $p(A)$ is a descendant of A', so $p(A) \prec p(B)$. Combining this with (2.1), we see that p3 is satisfied.

For the other direction, assume that A and B satisfy conditions p2 and p3. To prove that (A, B) must be in the computation tree with root (A_0, B_0), we use induction on the sum of the depths in T of A and B. We use (A_0, B_0) as the basis, since it is the unique node that minimizes the depth sum. In this case, the claim holds trivially, because (A_0, B_0) is in the computation tree by definition.

Otherwise, one can easily verify that either $p(A) \prec p(B) \preceq p(p(A))$ or $B \prec p(p(A)) \preceq p(B)$ must hold in all cases, so either $(p(A), B)$ or $(B, p(A))$ must satisfy condition p3. Both pairs clearly satisfy condition p2, and have a depth sum that is one less than that of (A, B). By the inductive hypothesis, the pair satisfying both conditions is in the computation tree. We know that $B \prec p(A)$, so $p(A)$ is the node we will split at this step. It follows that (A, B) is also a node in the computation tree. □

Note that the realization produced by **findpairs** contains an unordered pair $\{A, B\}$ for each (A, B) in the computation tree that satisfies condition p1. Thus we have:

Corollary 2.5.2: *The algorithm* **findpairs** *computes the canonical realization of* $P \otimes P$ *with respect to* T.

2.6 Size of the canonical realization

In order to prove a bound on the size of the canonical realization we first derive the following packing result. We say that two rectangles of equal dimension *overlap* if the intersection of the corresponding open rectangles is non-empty. Let T be a fair split tree associated with P. Let C be a d-cube, and let $S = \{A_1, A_2, \ldots, A_l\}$ be a set of nodes in T such that $A_i \cap A_j = \emptyset$ for all $i \neq j$, $l_{\max}(p(A_i)) \geq l(C)/c$, and $R(A_i)$ overlaps C, for all $i = 1, \ldots, l$. Define $K(c,d)$ to be the maximum possible value for $|S|$, independent of P and T.

Lemma 2.6.1: $K(c,d) \leq (3c+2)^d$.

Proof. For any node A in T, the following inequality holds:

$$l_{\min}(\hat{R}(A)) \geq \frac{l_{\max}(p(A))}{3}. \tag{2.2}$$

By the fair split condition, this is true when $p(A) = P$. Now, for $p(A) \neq P$ consider the fair split of $p(A)$ that formed A. If $l_{\min}(\hat{R}(A)) = l_{\min}(\hat{R}(p(A)))$ then the inequality is preserved, because $l_{\max}(p(A)) \leq l_{\max}(p(p(A)))$. Otherwise, $l_{\min}(\hat{R}(A)) < l_{\min}(\hat{R}(p(A)))$. In this case, $\hat{R}(p(A))$ must have a unique minimum-length dimension, because only one dimension can be split at a time. In other words, there is a unique coordinate axis i such that $l_i(\hat{R}(p(A))) = l_{\min}(\hat{R}(p(A)))$. This axis must have been perpendicular to the split, so the fair split condition guarantees that $l_i(\hat{R}(A)) \geq l_{\max}(p(A))/3$.

Now, consider a d-cube C. Suppose we have a set of disjoint d-cubes of length l' that all overlap C. We can bound the size of this set by a packing argument. Similar arguments are used by Vaidya [67, 69] in his algorithm for all nearest neighbors. It

is not hard to see that $(l(C)/l' + 2)^d$ is an upper bound, since the cubes in this set must all be *contained* in a d-cube of length $l(C) + 2l'$, centered at the center of C.

Note that the outer rectangles of disjoint A are disjoint. By (2.2), each $\hat{R}(A)$ overlapping C contains a d-cube of length $l_{\max}(p(A))/3$ that also overlaps C. Hence, our packing result applies to the number of $\hat{R}(A)$ that overlap C. This is at least as large as the number of $R(A)$ that overlap C. We assumed that $l_{\max}(p(A)) \geq l(C)/c$, so our bound is $(3c + 2)^d$, as stated. □

Using the preceding packing result, we can prove the following lemma.

Lemma 2.6.2: *Let $P = \{p_1, \ldots, p_n\}$ be a point set with an associated fair split tree T. For any node A in T, there are $O(1)$ canonical pairs of the form (A, B).*

Proof. Let $d_R(X, Y)$ denote $d(R(X), R(Y))$ for any two nodes X and Y in T. By the definition of a canonical pair, $B \prec p(A)$, so $R(p(A))$ and $R(B)$ can each be contained in d-balls of radius $(\sqrt{d}/2)l_{\max}(p(A))$. The minimum distance between these d-balls is at least $d_R(p(A), B) - (\sqrt{d})l_{\max}(p(A))$, since $d_R(p(A), B)$ is at most the distance between their centers.

We know that $p(A)$ and B are not well-separated, so

$$d_R(p(A), B) - (\sqrt{d})l_{\max}(p(A)) < s(\sqrt{d}/2)l_{\max}(p(A))$$

where s is the separation. We may rewrite this as:

$$d_R(p(A), B) < (s(\sqrt{d}/2) + \sqrt{d})l_{\max}(p(A)). \tag{2.3}$$

Let S_A be the set of all B such that (A, B) is a canonical pair. By (2.3), all $B \in S_A$ must overlap a d-cube centered at the center of $R(p(A))$ and with length $(s\sqrt{d} + 2\sqrt{d} + 1)l_{\max}(p(A))$. From the definition of a canonical pair, we have $p(A) \prec p(B)$, implying that $l_{\max}(p(B)) \geq l_{\max}(p(A))$. Additionally, all $B \in S_A$ are disjoint, since otherwise the canonical realization would violate condition r3 in the definition of a realization. Therefore, $|S_A|$ is bounded by $K(s\sqrt{d}+2\sqrt{d}+1, d)$, which by Lemma 2.6.1 is no more than $(3(s\sqrt{d} + 2\sqrt{d} + 1) + 2)^d$. □

Corollary 2.6.3: *The size of the canonical realization of $P \otimes P$ with respect to T is* $\mathrm{O}(n)$.

Combining Observation 2.3.2 with Corollaries 2.5.2 and 2.6.3, we obtain the following theorem.

Theorem 2.6.4: *Given a set P of n points in $\Re^d$ and a fair split tree of P, a well-separated pair decomposition of P can be constructed in* $\mathrm{O}(n)$ *time.* □

Chapter 3

Applications

In this chapter, we present some applications of the well-separated pair decomposition. These specific examples will motivate a general tree-structured computation framework for applications. We will make this framework explicit when developing parallel and dynamic algorithms. Many of the results of this chapter were originally presented in [19, 20, 21].

We show that the k-nearest neighbors of each point can be computed in $O(kn)$ sequential time, once we have computed the well-separated pair decomposition. The total complexity of the sequential algorithm, including the time to compute the decomposition, is thus $O(n \log n + kn)$, which is optimal in terms of both n and k. Our algorithm has a strong similarity to that of Vaidya [67, 69], which achieves $O(n \log n + nk \log k)$ complexity. Vaidya's algorithm is optimal if k is assumed to be a constant, and this includes the important case of the all-nearest-neighbors problem, in which $k = 1$. By formulating our algorithm in terms of the well-separated pair decomposition, we were able to simplify the analysis considerably from that of Vaidya by focusing primarily on the geometry, removed from an algorithmic context. We believe it was this increased focus that made it possible to refine the technique and eliminate a factor of $\log k$. Our approach has other significant advantages that will become clear when we consider the parallel version of the problem.

We also apply the decomposition to the efficient computation of n-body

potential fields. Greengard and Rokhlin [37, 38, 56] have developed a very interesting $O(n)$ time sequential algorithm known as the *fast multipole method* that, given n charges and their locations, calculates, for each of the n locations, the potential field due to remaining $n-1$ charges, up to a fixed precision. However, the algorithm in [38] makes assumptions about the geometric distribution of charges, which might be justifiable on pragmatic grounds, but are not normally made when treating problems in computational geometry. Also, when the desired output precision is much less than the input precision, the justification becomes weaker. Without these assumptions, we cannot even see an obvious variant of the fast multipole method which runs in $O(n \operatorname{polylog}(n))$ steps. We show that once the well-separated pair decomposition is computed, we can adapt the fast multipole method so that the remaining computation takes $O(n)$ sequential time.

We present an algorithm to compute an approximate Euclidean minimum spanning tree. A Euclidean minimum spanning tree (EMST) is simply a minimum spanning tree of the complete Euclidean graph [52, 71]. An approximate EMST is a spanning tree whose weight is within $1+\epsilon$ of the exact minimum for some constant $\epsilon > 0$ [66, 68]. Vaidya [68] has shown how to compute such a graph in time $O(\epsilon^{-d} n \log n)$. An observation about the well-separated pair decomposition immediately results in a matching bound, and by applying two non-trivial refinements, we reduce the time to $O(n \log n + (\epsilon^{-d/2} \log \frac{1}{\epsilon})n)$.

We show how to compute a sparse graph that approximates the complete d-dimensional Euclidean graph in the sense that the shortest path between two points is within a factor of $1+\epsilon$ of Euclidean distance. The general term for a sparse subgraph that approximates distance in this sense is a spanner. The problem of constructing spanners for the complete Euclidean graph has received much attention [24, 31, 44, 57, 58, 59, 70]. In [57], Ruppert and Seidel present an $O(\epsilon^{-d+1} n \log^{d-1} n)$ algorithm for this problem. We first present a spanner, somewhat larger than that of Ruppert and Seidel, whose existence follows immediately from the well-separated pair decomposition. The size of our graph matches that computed by an algorithm of Vaidya

[70], but our algorithm is faster by a factor of $\log \frac{1}{\epsilon}$. We refine our methods to compute a graph whose size is the same as that of Ruppert and Seidel. The time for our refined algorithm is $O(n \log n + (\epsilon^{-d} \log \frac{1}{\epsilon})n)$, substantially improving upon the previous bound.

We apply our decomposition to the computation of the exact EMST. In [4], Agarwal et al. consider the relationship between computing the exact EMST and computing the bichromatic closest pair between m red points and n blue points. They show that if $T_d(m, n)$ denotes the time to solve this problem, one can find the exact EMST in time $O(T_d(n, n) \log^d n)$. We improve this bound to $O(T_d(n, n) \log n)$ by combining Sollin's minimum spanning tree algorithm [16] with a generalization of our k-nearest-neighbors algorithm (section 3.1) based on the well-separated pair decomposition. For both the old and new bounds, the complexity reduces to $O(T_d(n, n))$ when $T_d(n, n) = \Omega(n^{1+\alpha})$ for $\alpha > 0$.

Finally, we consider the problem of enumerating the k-smallest interdistances in a point set P. Our algorithm has optimal complexity $O(n \log n + k)$, like those of Salowe [60] and Lenhof and Smid [47]. However, we show how the well-separated pair decomposition leads to a particularly simple implementation.

3.1 k-nearest neighbors

One useful application of the well-separated pair decomposition is an optimal sequential algorithm for computing the k-nearest-neighbors of each point in a set. In this section, we present such an algorithm. The complete method, including the computation of the split tree and realization, is reminiscent of Vaidya's algorithm [67, 69]. However, the way in which we have split our algorithm into phases makes it easier to parallelize efficiently. In addition, our bound is an improvement over Vaidya's when k is not a constant. Our algorithm for the special case of all-nearest-neighbors (corresponding to $k = 1$, when interdistances are unique) is somewhat simpler, and can be found in [19].

As in the preceding section, we will assume that for every node A in T, $R(A)$ was stored at that node when the split tree was being computed. This will be necessary for constant-time tests involving the distance from a point to the smallest d-ball containing $R(A)$.

Given a well-separated pair decomposition of P, we can compute the k-nearest-neighbors of P with only $O(kn)$ additional steps. Note that this is optimal and improves on the $O(n \log n + kn \log k)$ complexity of Vaidya's algorithm. The output of the k-nearest-neighbors algorithm is a list of all pairs (a, b), such that b is one of the k-nearest-neighbors of a. We refer to (a, b) as a k-nearest-neighbor pair.

Lemma 3.1.1: *Let P be a point set, and let $\{A, B\}$ be a pair in a well-separated realization of $P \otimes P$, with a separation $s > 2$. Suppose that there is a k-nearest neighbor pair (a, b) such that $a \in A$ and $b \in B$. Then $|A| \leq k$.*

Proof. Suppose $|A| > k$. Let $A' = A - \{a\}$. Then because A and B are well-separated, $d(a, a') < d(a, b)$ for all $a' \in A'$, $b \in B$. Since $|A'| \geq k$, B cannot contain a k-nearest-neighbor of a. □

Note that as a consequence of this lemma (taking $k = 1$), if $\{a, b\}$ is the closest pair, then $\{\{a\}, \{b\}\}$ is a pair in the realization of $P \otimes P$. Hence, given the realization, we can find the closest pair in P in $O(n)$ time. This provides a very simple proof that our sequential algorithms are optimal, since $\Omega(n \log n)$ is a lower bound for the closest pair problem in the algebraic decision-tree model. [52]

For the following, we use $\overline{xy}$ to denote the line segment between points x and y.

Let B be a point set, and let O_B be the center of the smallest d-ball containing $R(B)$. Let $\{a\}$ and $\{a'\}$ be singleton point sets, each well-separated from B, such that $d(a, O_B) \geq d(a', O_B)$, and let α represent the angle between $\overline{aO_B}$ and $\overline{a'O_B}$, given in radians.

Lemma 3.1.2: *If $\alpha < \frac{s}{s+1}$ then $d(a, a') < d(a, b)$ for all $b \in B$.*

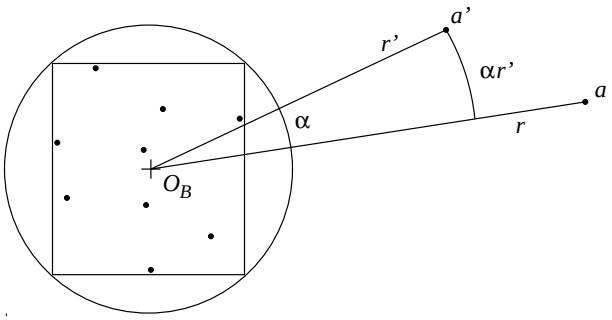


Figure 3.1: Points a and a'

Proof. Note that we can scale each dimension equally without affecting our claim. To simplify the analysis, we scale the points so that the smallest d-ball containing $R(B)$ has radius 1. Now we confine our attention to the plane containing $\overline{aO_B}$ and $\overline{a'O_B}$.

For convenience, let r and r' denote $d(a, O_B)$ and $d(a', O_B)$, respectively. We observe the following bound:

$$d(a, a') \leq r - (1 - \alpha)r'. \tag{3.1}$$

The right-hand side of this inequality is the length of the path obtained by going from a' along an arc segment of length $\alpha r'$ to the point on $\overline{aO_B}$ at distance r' from O_B, and going from this point to a along a line segment of length $r - r'$ (see figure 3.1). Clearly, this is an upper bound on $d(a, a')$.

Because $\{a\}$ and $\{a'\}$ are well-separated from B, both are at distance at least $s + 1$ from O_B. Hence, inequality (3.1) is preserved when we substitute $s + 1$ for r'. We can also substitute $\frac{s}{s+1}$ for α since we are assuming this is a strict upper bound. With these substitutions, (3.1) becomes a strict inequality, and simplifying, we obtain $d(a, a') < r - 1$. We observe that $d(a, b) \geq r - 1$ for all $b \in B$, so $d(a, a') < d(a, b)$ for all $b \in B$. □

Consider a convex cone C whose apex lies at O_B, and let θ denote the angle

of C, defined as the maximum possible angle between $\overline{O_B x}$ and $\overline{O_B y}$ for any x and y in C. Let A be a set of points that all lie in C.

Lemma 3.1.3: *Let P be a point set containing $A \cup B$ as a subset, and let a be a point in A. If $\theta < \frac{s}{s+1}$ and $|\{a' \in A - \{a\} : d(O_B, a') \leq d(O_B, a)\}| \geq k$ then no $b \in B$ can be a k-nearest-neighbor of a in P.*

Proof. By Lemma 3.1.2, every $a' \in A - \{a\}$ such that $d(O_B, a') \leq d(O_B, a)$ satisfies $d(a, a') < d(a, b)$ for all $b \in B$. Hence, if $|\{a' \in A - \{a\} : d(O_B, a') \leq d(O_B, a)\}| \geq k$, then a has k neighbors in A nearer to it than any $b \in B$. Therefore, no $b \in B$ can be a k-nearest neighbor of a in P. □

Yao [71] defines a *frame* to be a partition of d-dimensional space into cones whose apexes all lie at the origin. The angle of a frame is defined to be the maximum angle over all its cones. Yao has shown that for any d and any $\psi > 0$ one can construct a finite frame with angle less than ψ. Note that the size of the frame depends on d and ψ.

To apply the above results to the k-nearest-neighbor problem, we assume we have a well-separated pair decomposition of our point set P with separation $s > 2$ (say $s = 2.1$). Recall the definition of separation, given in section 2.1. For every node B in T, let $f(B)$ be the set of all a such that for some ancestor B' of B, $\{A, B'\}$ ($|A| \leq k$) is in the realization and $a \in A$. Lemma 3.1.1 guarantees that if (a, b) is a k-nearest-neighbor pair and $b \in B$, then $a \in f(B)$. For node B, we compute $N(B)$, some subset of $f(B)$ that includes all $a \in A$ such that $\{A, B\}$ is a pair in the realization, and (a, b) is a k-nearest-neighbor pair for some $b \in B$.

Using Lemma 3.1.3, we can find an $N(B)$ of size $O(k)$ for every B in T. Note that the constant in the O-notation depends on d. The computation is done top down, starting at the root and ending at the leaves. The set for the root P is the empty set, since every point is in P. The initial set for any node B is the union of $N(p(B))$ and the set of a such that $\{A, B\}$ ($|A| \leq k$) is a pair in the realization and $a \in A$.

In time proportional to the size of this set we can compute $N(B)$, as follows: Construct a frame of angle less than $\frac{s}{s+1}$, and translate it so that O_B is the origin. Now, partition the initial set into a constant number of subsets of points lying in each cone of the frame. For each of these subsets, select the k that are nearest to O_B. Note that we can perform this operation in linear time for each subset by selecting the k-th nearest point (or maximum if the set has less than k elements) and then retrieving all points nearer to O_B than this element (and possibly some with the same distance). We let $N(B)$ consist of the set of all the points thus selected.

By Lemma 3.1.3, the points that we have not selected can be eliminated from $N(B)$. The overall top down computation takes $O(kn)$ steps, since the size of the set passed from any node to its children is $O(k)$.

Suppose b is a k-nearest-neighbor of a. Using the preceding arguments, we can establish that a will be in $N(\{b\})$. Now, for each a, we compute the set of all b such that $a \in N(\{b\})$, and select the k nearest b in each set, giving us the k-nearest-neighbors of a. There are at most $O(kn)$ total elements over which we perform selection, so the complexity of this step is $O(kn)$.

Theorem 3.1.4: *Given a set P of n points in $\Re^d$ and a well-separated pair decomposition of P, the k-nearest neighbors of all points in P can be computed in $O(kn)$ steps.* □

3.2 n-body potential fields

A fundamental problem in the simulation of particle systems is that of computing the potential field of a set of particles with Coulombic or gravitational forces. For any particle at position a we define a potential function $\phi_a(x)$ that associates a real number with every point x in the space around it, and depends only on the distance from a.

For example, in three dimensions, $\phi_a(x) = q_a/d(a,x)$, where q_a is the strength of the charge at a. This value is related to electrostatic charge in the Coulombic case

and mass in the gravitational case. The potential field of a set of particles is the sum of the potential fields of each particle in the set.

We represent a collection of n distinct particles with potential fields as a set of points $\{z_1, \ldots, z_n\}$ and charge strengths associated with each point (we refer to a point as z_i in this instance to avoid confusion with the parameter p that we will later define). We would like to compute, for each point z_i, $i = 1, \ldots, n$, the potential due to the remaining points. The potential computation can clearly be performed in $O(n^2)$ time using the direct sum $\sum_{i \neq j} \phi_{z_j}(z_i)$ to find the potential at each z_i.

Greengard and Rokhlin [37, 38, 56] have developed a method of approximating this potential using truncated multipole expansions. This algorithm, known as the fast multipole method, has received a great deal of attention in the scientific computing community [39, 43, 62, 72]. While we consider this an important (indeed, motivating) application for our data structure, we do not intend this section as an introduction to the fast multipole method. Instead, we will only summarize the method here, and show how our decomposition fits into the general scheme. The interested reader is referred to Greengard's excellent exposition [38] for further details.

The multipole expansion of a collection of particles P is an infinite series that converges outside the sphere containing these particles and is equal to their potential field in this region. The truncated multipole expansion is an approximation of the multipole expansion containing only those terms of degree p or less. The size of this expansion depends only on p, and not on $|P|$.

For every point that is well-separated from P, the truncated multipole expansion approximates the potential field of P to a precision that depends only on p and s, the separation. In general, the precision increases monotonically with p and s, the relative error being $O(s^{-p})$. When high precision is required, there is clearly more to be gained in the long run by increasing p, but the issue is not as clear when lower precision is tolerable. This may be the case, for example, when the charges themselves are not known to a high degree of precision. Because of the variety of ways of implementing the fast multipole method, the most suitable tradeoff between

s and p will depend on many factors, and is best left as a final "tuning" step. We consider the flexible choice of s to be an advantage of our decomposition over Greengard's formulation, since his choice of s is fixed by the geometry of squares or cubes in a regular grid.

Using the truncated multipole expansion, we may approximate the potential due to arbitrarily large sets, provided we are sufficiently far away, by an arithmetic function with O(1) terms. Each expansion is centered at a point x, and expansions centered at the same point are added by taking the pairwise sum of their coefficients. In addition to the multipole expansion, which describes the potential field outside a given sphere, Greengard defines the local expansion, which describes the potential field within a given sphere. Given an arbitrary sphere that is well-separated from P, and a truncated multipole expansion of P, we may construct, in O(1) time, a local expansion of the field due to P within this sphere, whose error bound is similar to that of the truncated multipole expansion. Local expansions may also be added, provided they are centered at the same point. Greengard shows that the centers of both types of expansions may be shifted in O(1) time, allowing them to be added.

We now sketch how the above operations may be used for potential field computation, given a split tree T of the set P and a well-separated realization of $P \otimes P$ that uses T. It should be emphasized that the sequential algorithm is an adaptation of the fast multipole method when the split tree can have up to O(n) depth.

In the first phase, we compute the truncated multipole expansion for all nodes in T, proceeding bottom up from the leaves. We can compute the multipole expansion for all leaves in constant time, since each corresponds to a single particle. The expansion for an internal node is obtained by shifting the center of one of its children to that of the other, and adding the expansions. The process of shifting and adding expansions also takes constant time. The total time for this phase is linear.

In the second phase, we compute, for each node, the local expansion of the potential field due to the nodes with which it is paired in the realization of $P \otimes P$. To

do this, we convert the multipole expansions of the preceding phase to local expansions centered at the center of the bounding rectangle of this node, and take their sum. This requires time proportional to the total number of pairs in the realization, which is linear.

In the third phase, we proceed top down and compute a local expansion for each node in T, valid within the bounding rectangle of the set at this node, representing the field due to sets paired with this node or one of its ancestors in the realization. To compute this, we pass the local expansion of a node to its children, shift its center, and add it to the local expansion computed in the second phase.

Finally, to compute the potential at any point, we simply evaluate the local expansion of each leaf at the point in the set at this leaf. Our definition of a realization guarantees that this is the correct potential value. The total work required is clearly $O(n)$, giving us the following theorem.

Theorem 3.2.1: *Given a set P of n points with associated charge strengths, and a well-separated pair decomposition of P, we can compute the potential at all points in P in $O(n)$ time.* □

This corresponds to the main result of [37, 38], but we eliminate the need for assumptions about input distribution by insisting that a well-separated pair decomposition be provided in the input.

In our potential computation algorithm, we have been treating p, the degree of the multipole expansion, as a constant. However, the dependency on p can be important in practice. A result of Reif and Tate [51, 54] significantly reduces this dependency in both two and three dimensions from that given in [37, 38]. It is easy to adapt their method to our framework. In this way, we remove the dependency on input precision from the time complexity of constructing the tree T. In both cases, there is no dependency on bit precision after the tree has been constructed.

3.2.1 Computation of arbitrary force functions

While Greengard et al. present a derivation of potential computation using harmonic functions, it should be noted that the well-separated pair decomposition can be applied to more general force functions as well. To illustrate this fact, such an approach is provided below. Note that the derivation below requires some modification to insure numerical stability and to guarantee the same precision-dependent complexity. It is, however, a starting point for applying the well-separated pair decomposition to other potential functions, such as empirically determined energy functions that occur in molecular dynamics applications.

Let $P = \{x_1, x_2, \ldots, x_n\}$ be a set of points in $\Re^d$, and let $\{\{A_1, B_1\}, \ldots, \{A_k, B_k\}\}$ be a well-separated realization of $P \otimes P$ given by some well-separated pair decomposition of P. Let $\psi(x, x')$ be an arbitrary "force function" between x and x'. For example, it could be a vector denoting coulombic force, or a scalar denoting potential.

Suppose we have an infinite family of functions of the form

$$h_l(x, x') = f_l(x)g_l(x'), \text{ for } l = 0, 1, 2, \ldots \text{ where } x, x' \in \Re^d$$

such that for all $m = 1, \ldots, k$, there exists $\alpha_{m0}, \alpha_{m1}, \alpha_{m2}, \ldots$ for which the sum

$$\sum_{l=0}^{\infty} \alpha_{ml} h_l(x, x')$$

converges to $\psi(x, x')$ for all x and x' such that x is in the smallest d-sphere containing A_m, and x' is in the smallest d-sphere containing B_m.

For example, let $\{h_l | l = 0, 1, 2, \ldots\}$ be the set of all monomials in the scalar components of x and x', given in some lexical order. Then we can take our series to be a Taylor series with origin at (x_0, x_0') where x_0 is in the smallest d-sphere containing A_m and x_0' is in the smallest d-sphere containing B_m. It is easy to verify that such a series converges for all continuous distance-based force functions.

Now, consider the total force at x_i, denoted $\Phi(x_i)$. By definition,

$$\Phi(x_i) = \sum_{j \neq i} q_j \psi(x_i, x_j)$$

where q_j is the value of the charge at x_j. Assume, for the sake of convenience, that the sets A_m and B_m are named so that if $x_i \in A_m \cup B_m$, then $x_i \in A_m$. Then,

$$\Phi(x_i) = \sum_{j \neq i} q_j \sum_{l=0}^{\infty} \alpha_{m(i,j),l} h_l(x_i, x_j)$$

where $m(i,j)$ is the unique m such that $x_i \in A_m$ and $x_j \in B_m$.

Rearranging sums, we obtain

$$\begin{aligned} \Phi(x_i) &= \sum_{l=0}^{\infty} \sum_{j \neq i} q_j \alpha_{m(i,j),l} h_l(x_i, x_j) \\ &= \sum_{l=0}^{\infty} \sum_{m \in M(i)} \alpha_{ml} \sum_{j \in J(m)} q_j h_l(x_i, x_j) \end{aligned}$$

where $M(i) = \{m | x_i \in A_m\}$ and $J(m) = \{j | x_j \in B_m\}$.

We recall that $h_l(x_i, x_j) = f_l(x_i) g_l(x_j)$. Because $f_l(x_i)$ does not depend on j or m, we may move it outside of the two innermost summations. Hence, we obtain

$$\Phi(x_i) = \sum_{l=0}^{\infty} f_l(x_i) \sum_{m \in M(i)} \alpha_{ml} \sum_{j \in J(m)} q_j g_l(x_j).$$

We may interpret this sum as an expansion

$$\Phi(x) = \sum_{l=0}^{\infty} \beta_{il} f_l(x)$$

which converges in the neighborhood of x_i, where the values of β_{il} are obtained by matching coefficients with the preceding formula.

We cannot guarantee for all i that $x_i \in A_m \cup B_m$ implies $x_i \in A_m$. Hence, to generalize the above formulas, we need to redefine $M(i)$ to be $\{m | x_i \in A_m \cup B_m\}$ and replace $J(m)$ with

$$J(i,m) = \begin{cases} \{j | x_j \in B_m\} & \text{if } x_i \in A_m \\ \{j | x_j \in A_m\} & \text{if } x_i \in B_m \end{cases} .$$

Then,

$$\beta_{il} = \sum_{m \in M(i)} \alpha_{ml} \sum_{j \in J(i,m)} q_j g_l(x_j).$$

For some fixed choice of l, we consider the problem of computing all coefficients $\beta_{1l}, \beta_{2l}, \ldots, \beta_{nl}$. Recall that all sets A_m and B_m are represented by a union of the leaves of some subtree of T, the split tree in our well-separated pair decomposition. Hence, we can evaluate all sums of the form $\sum_{j \in J(i,m)} q_j g_l(x_j)$ in $O(n)$ time using a bottom-up procedure.

Given the values of the above sums, it is easy to see that we can evaluate the outer sum in the formula for β_{il} for all i, using a top-down procedure requiring $O(n+k)$ time. Our well-separated pair decomposition guarantees that $k = O(n)$, so this is $O(n)$ time.

From the above, it clearly follows that an approximation of $\Phi(x_i)$ based on the first p terms of our expansion can be evaluated for all $i = 1, 2, \ldots, n$ in time $O(pn)$, assuming we are given the values of α_{kl}, and $f_l(x_i)$, $g_l(x_i)$ for all $i = 1, 2, \ldots, n$ (in practice we will have to compute these).

Note that the value of p used above does not correspond to the value of p used by Greengard, and will in general need to be larger in order to guarantee the same error bound. It should also be noted that the preceding algorithm would be numerically unstable if used directly, though this problem could be reduced if, as in [38], we shifted the origin of our approximation to the center of each tree node as we carried out the computation at that node. The above is, however, intended not so much as a practical method as a demonstration of the general properties that make the well-separated pair decomposition useful for computing arbitrary pairwise force functions.

3.3 Approximate Euclidean minimum spanning trees

In this section, we consider the problem of constructing an approximate Euclidean minimum spanning tree (EMST) of a point set P. The EMST of a point set is

defined to be the minimum spanning tree of the complete Euclidean graph, and an approximate EMST is any spanning tree whose weight is no more than $1 + \epsilon$ times the weight of the exact EMST, where ϵ is some approximation factor.

Let $P = \{x_1, \ldots, x_n\}$ be a set of points in $\Re^d$ and suppose we have constructed a well-separated pair decomposition of P with separation s. The pairs in our decomposition are given as $\{A_1, B_1\}, \{A_2, B_2\}, \ldots, \{A_m, B_m\}$.

In the following, we wish to consider those graphs whose vertices are the points in P. We will consider edges between points a and b to be weighted by their Euclidean distance $d(a, b)$. We assume for simplicity that all distances are distinct. Our results can easily be extended to include degenerate cases.

Given any edge $\{a, b\}$, there is a unique well-separated pair $\{A_i, B_i\}$ in our decomposition such that $a \in A_i$ and $b \in B_i$. This follows from the definition of a well-separated realization. We define a *substitute* for $\{a, b\}$ to be any edge $\{a', b'\}$ such that $a' \in A_i$ and $b' \in B_i$.

Now let T be the Euclidean minimum spanning tree (EMST) of P, and let T' be a graph formed by replacing every edge in T by one of its substitutes.

Lemma 3.3.1: *If $s > 2$, then T' is a spanning tree of P.*

Proof. Let F_j denote the spanning forest of P induced by the j smallest edges of T, for $j = 0, \ldots, n-1$, and let F'_j denote the subgraph of T' induced by the substitutes of the edges of F_j. We claim that two points are connected in F'_j iff they are connected in F_j. Our lemma is easily seen to correspond to the case $j = n - 1$.

We proceed by induction on j. The basis is trivial, because F_0 and F'_0 are both empty graphs. Now, assume that our claim is true for $j - 1$, and let $\{a, b\}$ be the jth smallest edge of T, with $a \in A_i$ and $b \in B_i$.

Consider any pair of points whose distance is less than $d(a, b)$. Such a pair must be connected in F_{j-1}, because otherwise we could construct a spanning tree smaller than T by replacing $\{a, b\}$ with an edge between such a pair.

Because $s > 2$, A_i and B_i can each be enclosed in spheres of diameter less than $d(a,b)$, and this bounds the largest interpoint distance within these sets. Combining this with the previous observation, we see that all pairs of points in A_i are connected in F_{j-1}, as are all pairs of points in B_i. By the inductive hypothesis, this holds for F'_{j-1} as well.

The addition of $\{a,b\}$ to F_{j-1} connects the component containing the points in A_i with the component containing the points in B_i. By the preceding observation, the addition of a substitute for $\{a,b\}$ to F'_{j-1} does likewise. The set of vertices in the new component must be the same for both F_j and F'_j, so the connectivity claim is preserved. □

We now consider the problem of constructing an ϵ-approximate EMST of P; i.e., a spanning tree of P whose total weight is within a factor $1+\epsilon$ of the EMST. As in the $O(\epsilon^{-d} n \log n)$ algorithm of Vaidya [68], we will first compute a sparse graph G that contains an approximate EMST as a subgraph.

For a well-separated pair $\{A_i, B_i\}$, recall that $d(A_i, B_i)$ denotes the minimum distance between two points such that one lies in A_i and the other lies in B_i. Let $\mathbf{G}(s,\epsilon)$ be the set of graphs G defined by a well-separated pair decomposition of P with separation s such that for every pair $\{A_i, B_i\}$ in the decomposition, G contains exactly one edge $\{a,b\}$ such that $a \in A_i$, $b \in B_i$, and $d(a,b) \leq (1+\epsilon) d(A_i, B_i)$. We will refer to any such pair of points a and b as an *ϵ-approximate closest pair* between A_i and B_i. Recall that the vertices of any graph in $\mathbf{G}(s,\epsilon)$ correspond to points in P. We may thus refer to points and vertices interchangeably.

Lemma 3.3.2: *Let G be a graph in $\mathbf{G}(s,\epsilon)$. If $s > 2$, the minimum spanning tree of G is an ϵ-approximate EMST of P.*

Proof. For every edge in the exact EMST, G contains a substitute whose weight is within $1+\epsilon$ of that edge. By Lemma 3.3.1, T', the tree induced by the substitutes, is an ϵ-approximate EMST. The minimum spanning tree of G has weight at most that of T', so it is also an ϵ-approximate EMST. □

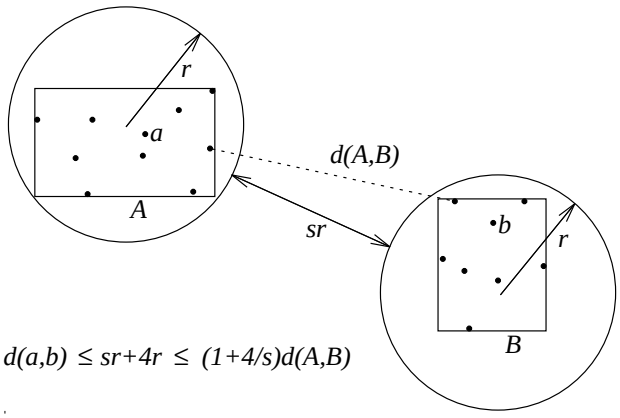


Figure 3.2: Approximate distance between points in well-separated sets

Note that if we let $\epsilon = 0$ in the above lemma, we obtain the corollary that the exact EMST is the minimum spanning tree of the graph formed by taking the closest pair of points between each well-separated pair. This graph has linear size and would allow us to find the EMST very rapidly. Unfortunately, we do not know how to compute this graph efficiently.

3.3.1 First approximation algorithm

We now consider the application of Lemma 3.3.2 to the efficient computation of an ϵ-approximate EMST for some $\epsilon > 0$. Our first algorithm is based on the following inequality, which holds for all $a \in A_i$, $b \in B_i$:

$$d(a, b) \leq (1 + 4/s)d(A_i, B_i). \tag{3.2}$$

This can be derived easily from the geometry of well-separated sets (see figure 3.2).

If we let $s = 4/\epsilon$, and compute the well-separated pair decomposition, we can construct the graph G by choosing an arbitrary edge between each well-separated pair. The size of G is the same as the number of pairs, as derived in Chapter 2. Since d is constant, one can easily see that the size of G is $O(\epsilon^{-d}n)$.

We can construct an ϵ-approximate EMST by first computing a well-separated pair decomposition in time $O(n \log n + \epsilon^{-d} n)$, and then computing the minimum spanning tree of G using a standard algorithm. This can easily be done in time $O(\epsilon^{-d} n \log n)$, matching Vaidya's result. We will improve on this by choosing the edge for each pair more carefully, and finding a graph G of size $O(n)$ independent of ϵ.

3.3.2 Refining the bound on ϵ

For each well-separated pair of sets, we are interested in finding a pair of points that approximates the closest pair between the sets to within a factor of $1+\epsilon$. In (3.2), we approximated the distance between points in well-separated sets by a constant, leaving a linear error term of $\epsilon = O(s^{-1})$. Suppose we instead use a linear approximation. Intuitively, we should then expect ϵ to be $O(s^{-2})$. This is the first refinement we will need for our improved algorithm.

For convenience in the following analysis, we will treat points as vectors, allowing operations such as addition and dot product. Recall that the square of the Euclidean distance between points a and b is $(a-b) \cdot (a-b)$.

Assume, as before, that $a \in A_i$ and $b \in B_i$. Since A_i and B_i are well-separated, we can enclose them in d-spheres of radius r whose distance of closest approach is at least sr. Let a_0 be some point lying in the d-sphere enclosing A_i, and let b_0 be some point lying in the d-sphere enclosing B_i. We represent a and b by $a_0 + \Delta a$ and $b_0 + \Delta b$, respectively. If we substitute these expressions into the above dot product and rearrange the terms, we obtain

$$((a_0 - b_0) + (\Delta a - \Delta b)) \cdot ((a_0 - b_0) + (\Delta a - \Delta b)).$$

We can expand this expression into

$$(a_0 - b_0)^2 + 2(a_0 - b_0) \cdot (\Delta a - \Delta b) + (\Delta a - \Delta b)^2.$$

Note that the Euclidean norms of Δa and Δb are bounded above by $2r$. If we consider them to be points in a d-sphere of radius $2r$, we can argue from geometrical considerations that

$$0 \leq (\Delta a - \Delta b)^2 \leq (4r)^2$$

and thus bound the last term of our expression.

Suppose we could determine a pair $a' \in A_i$, $b' \in B_i$ that minimized just the first two terms of our expression. By our bound on the third term, it would be within $(4r)^2$ of the minimum of the whole expression. This minimum must be at least $(sr)^2$ since sr is a lower bound on $d(A_i, B_i)$. Hence,

$$(d(a', b'))^2 \leq (1 + (4/s)^2)(d(A_i, B_i))^2.$$

Using standard error analysis techniques, we obtain

$$d(a', b') \leq (1 + \mathrm{O}(s^{-2}))d(A_i, B_i). \tag{3.3}$$

Now we consider the problem of determining the pair $\{a', b'\}$. The vectors a_0 and b_0 are constant, so the problem reduces to minimizing $u \cdot (\Delta a - \Delta b)$, where u is some vector in the direction of $a_0 - b_0$. By adding the constant term $u \cdot (a_0 - b_0)$ to this function, we obtain the equivalent problem of minimizing $u \cdot (a - b)$, or $u \cdot a - u \cdot b$. Since the choice of a is independent of the choice of b, we need only minimize $u \cdot a$ and maximize $u \cdot b$.

The preceding observation allows us to find an $O(s^{-2})$-approximate closest pair between A_i and B_i in time $\mathrm{O}(|A_i| + |B_i|)$, a clear improvement over the naive $\mathrm{O}(|A_i||B_i|)$ bound. However, it will not be efficient to carry out such a minimization on each pair separately. Instead, we make use of the fact that for a separation of s, we need only consider $\mathrm{O}(s^{d-1})$ distinct values of u. Because A_i and B_i come from a hierarchical decomposition, we can compute all necessary maxima and minima in $\mathrm{O}(s^{d-1}n)$ time, providing a substantial speed-up.

Intuitively, we choose the u vectors to be a uniform mesh of points on the surface of a d-cube of length $2l$ centered at the origin. More formally, we define the

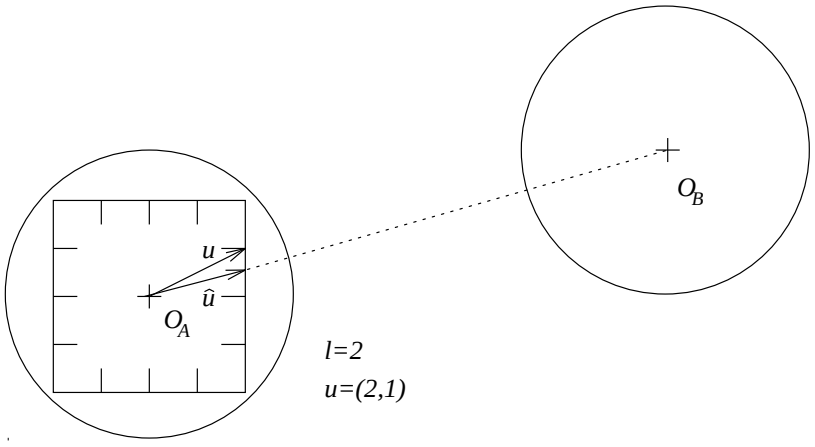


Figure 3.3: Approximate direction u of a well-separated pair

set to be those vectors of the form $(i_1, \ldots, i_d)$ such that for $j = 1, \ldots, d$, i_j is an integer in the range $[-l, l]$ and for at least one j, $|i_j| = l$. The size of this set is clearly $O(l^{d-1})$. We will later show how to choose l in terms of s.

We use the following approach to find a vector in the preceding set corresponding to each well-separated pair $\{A_i, B_i\}$. Construct d-spheres of radius r enclosing A_i and B_i such that the distance between these spheres is *exactly* sr. These may be found by expanding the spheres used to prove that the sets are well-separated.

Let O_A be the center of the sphere enclosing A_i, and let O_B be the center of the sphere enclosing B_i. Let $\hat{u}$ be the vector obtained by scaling $O_A - O_B$ uniformly so that the maximum absolute value over all its coordinates is l. Now, let u be the vector that results from replacing each coordinate of $\hat{u}$ with its nearest integer (see figure 3.3). Such a vector is clearly a member of the set we have defined. For this choice of u to be valid, we must be able to find some a_0 in the sphere enclosing A_i and some b_0 in the sphere enclosing B_i such that $a_0 - b_0$ has the same direction as u.

Note that $O_A - O_B = \gamma\hat{u}$ for some scalar γ. Let $b_0 = O_B$ and choose a_0 so that $a_0 - b_0 = \gamma u$. The distance from $\hat{u}$ to u is at most $\sqrt{d}/2$, because each coordinate was changed by at most $1/2$. Hence, the distance from $O_A - O_B$ to $a_0 - b_0$ is at most

$\gamma\sqrt{d}/2$. Because $b_0 = O_B$, this is the same as the distance from O_A to a_0. To show that a_0 lies in the sphere enclosing A_i, it then suffices to show that $\gamma\sqrt{d}/2 \leq r$, or $\gamma \leq 2r/\sqrt{d}$.

Choose l to be $\lceil (s+2)\sqrt{d}/2 \rceil$. The maximum absolute value of any coordinate of $O_A - O_B$ cannot exceed $(s+2)r$, since this is the distance between O_A and O_B. Thus $\gamma \leq (s+2)r/l$. Substituting for l, we obtain the inequality we wish to prove.

Therefore, given the preceding formula for l, we can always find some u in our set with the same direction as $a_0 - b_0$ for some allowable choice of a_0 and b_0. Since l is linear in s, the size of our set of vectors is $\mathrm{O}(s^{d-1})$, as claimed.

We are finally ready to compute an approximate closest pair of points between every well-separated pair. We first compute, for every subset S in our decomposition tree and every vector u in our set, the point $a(u,S) \in S$ that minimizes $u \cdot a(u,S)$, and the point $b(u,S) \in S$ that maximizes $u \cdot b(u,S)$. We compute these functions bottom-up on the decomposition tree, so the time required is $\mathrm{O}(s^{d-1}n)$.

Now, for every well-separated pair $\{A_i, B_i\}$, we take the approximate closest pair between A_i and B_i to be $\{a(u,A_i), b(u,B_i)\}$ for the u corresponding to A_i and B_i. We use the method outlined above. The time to calculate $\hat{u}$ is $\mathrm{O}(1)$. If we are allowed to use the floor function and array indexing, we can determine $a(u,A_i)$ and $b(u,B_i)$ in constant time from $\hat{u}$. Otherwise, the time required is at most $\mathrm{O}(\log(s^{d-1}))$, which is $\mathrm{O}(\log s)$. There are $\mathrm{O}(s^d n)$ well-separated pairs, so the total time is $\mathrm{O}((s^d \log s)n)$.

Now suppose $\epsilon = s^{-2}$. Using the above technique, we can construct a graph G of size $\mathrm{O}(s^d n)$, which by Lemma 3.3.2 contains an ϵ-approximate EMST as a subgraph. In other words, the size of G is $\mathrm{O}(\epsilon^{-d/2}n)$, and the time to compute G is $\mathrm{O}((\epsilon^{-d/2} \log \frac{1}{\epsilon})n)$. This gives a substantial improvement over Vaidya's algorithm.

3.3.3 Final approximation algorithm

It is worthwhile to reduce the size of G further before applying a standard minimum spanning tree algorithm, because such algorithms have superlinear time complexity. To do this, we compute two well-separated pair decompositions of P. We compute a *coarse* decomposition using a separation independent of ϵ, but greater than 2 (we can adapt the termination condition to insure that every terminal pair is well-separated with a separation strictly greater than 2). We also compute a *fine* decomposition, using a separation of $c\epsilon^{-1/2}$, where c is a constant chosen to insure that $d(a', b') \leq (1+\epsilon)d(A_i, B_i)$. The existence of c follows from inequality (3.3).

We need to insure that for every $\{A_i, B_i\}$ in the coarse decomposition, $A_i \otimes B_i$ has a realization composed of pairs in the fine decomposition. This is guaranteed if we use the same tree for both decompositions, and apply the algorithm we have previously developed for computing the pairs 2.3. To see this, note that in the sequential algorithm for computing the canonical realization, the separation s only changes the termination condition, not the way in which nodes are split. As a result, any well-separated pair in the coarse decomposition corresponds to a (possibly non-terminal) pair that arises during the computation of the fine decomposition.

We associate every $\{A_i, B_i\}$ in the coarse decomposition with a list containing the corresponding pairs in the fine decomposition. We first compute all ϵ-approximate closest pairs in the fine decomposition. The time for this step is $O((\epsilon^{-d/2} \log \frac{1}{\epsilon})n)$ using the previous method. Now, for every $\{A_i, B_i\}$ in the coarse decomposition, we find its ϵ-approximate closest pair by taking the minimum of the corresponding pairs in the fine decomposition. The total time over all pairs is linear in the size of the fine decomposition, which is $O(\epsilon^{-d/2}n)$.

Now, take G to be the graph induced by edges between the ϵ-approximate closest pairs of the coarse decomposition. The size of G is $O(n)$, and by Lemma 3.3.2, G contains an ϵ-approximate EMST. Using a straightforward method, this tree can be extracted in time $O(n \log n)$, independent of ϵ. Combining this with the preceding

results, we obtain the following theorem.

Theorem 3.3.3: *Given a set P of n points in $\Re^d$, we can construct an ϵ-approximate EMST of P in $O(n \log n + (\epsilon^{-d/2} \log \frac{1}{\epsilon})n)$ steps.* □

3.4 Approximate complete Euclidean graph

In the preceding section we were interested in the set of graphs $\mathbf{G}(s, \epsilon)$ because each graph contains an approximate EMST as a subgraph. However, these graphs are interesting in their own right, because they are approximations to the complete d-dimensional Euclidean graph in the sense of Chew [24].

In any such graph, the length of the shortest path between two points is within a constant factor of their Euclidean distance. Note that these graphs can be treated as a special case of *spanners* [23]. Sparse graphs with this property have received a great deal of attention [24, 31, 44, 57, 58, 59, 70]. We summarize this property of our graphs in the following lemma:

Lemma 3.4.1: *For any point set P, let G be a graph in the family $\mathbf{G}(s, \epsilon)$, defined for the point set P. If $s > 4$ then there is a path from a to b along the edges in G with length at most*

$$\frac{1+\epsilon}{1-4s^{-1}} d(a, b).$$

Proof. We give a recursive method for constructing a path from a to b, and then prove the length bound. Note that G must contain exactly one edge $\{a', b'\}$ that is a substitute for $\{a, b\}$. If $a \neq a'$, then recursively construct a path from a to a', and if $b \neq b'$, then recursively construct a path from b' to b. Concatenate these two paths to the edge $\{a', b'\}$, forming a path from a to b.

We prove the length bound by strong induction on the number of edges in the path. If the path contains just one edge, the claim clearly holds, since the length of the path is exactly $d(a, b)$. Otherwise, we can assume inductively that the claim

holds for the path from a to a' and the path from b' to b. Because $\{a', b'\}$ is an ϵ-closest pair between A_i and B_i, the length of this edge is at most $(1+\epsilon)d(a,b)$. Combining this with the inductive hypothesis, the length of the path from a to b is at most

$$(1+\epsilon)d(a,b) + \frac{1+\epsilon}{1-4s^{-1}}(d(a,a') + d(b,b')).$$

Because $a, a' \in A_i$ and $b, b' \in B_i$, the well-separated set condition guarantees that $d(a,a'), d(b,b') \leq 2s^{-1}d(a,b)$. Substituting this bound into the above formula gives us the claim of our lemma. □

It should be noted that a claim similar to the above lemma was previously shown by Salowe [58], whose approximate Euclidean graph is similar to the above and to that of Vaidya [70]. However, Salowe's size bound is $O(\epsilon^{-2d}n)$, which could be significantly larger than that of its competitors.

As long as $\epsilon = O(s^{-1})$, the formula of Lemma 3.4.1 is easily seen to be $1 + O(s^{-1})$. Suppose we construct a well-separated pair decomposition of P with separation s. Let G be the graph used in our first approximate EMST algorithm. That is, for every $\{A_i, B_i\}$, choose an arbitrary pair of points $a \in A_i$, $b \in B_i$. Now, by inequality (3.2), G is in $\mathbf{G}(4s^{-1}, s)$. To guarantee that the length of the shortest path between two points in G is within a factor of $1+\delta$ of Euclidean distance, we need only let $s = c\delta^{-1}$ for a sufficiently large constant c. G has $O(\delta^{-d}n)$ edges, and the algorithm to compute it requires $O(n \log n + \delta^{-d}n)$ time. Note that we can decrease ϵ to $O(s^{-2})$ using previous techniques, but this will only improve the constants, while making our algorithm harder to implement. Vaidya [70] has shown how to produce a graph of the same size, but the time complexity of his algorithm is greater by a factor of $\log \frac{1}{\delta}$.

We can reduce the size of our graph by a factor of $\frac{1}{\delta}$ using some observations due to Ruppert and Seidel [57]. These refinements will increase the running time of our algorithm by a factor of $\log \frac{1}{\delta}$. In contrast, the algorithm of [57] has time complexity $O(\delta^{-d+1} n \log^{d-1} n)$. Our graph can be at most a d-dependent constant

factor larger than that of [57].

Ruppert and Seidel define their graph in terms of a partition of the space around each point a into narrow polyhedral cones, each containing an axis, or ray internal to the cone, such that for any point b in the cone, the angle between the axis and the line segment $\overline{ab}$ is at most π/k for some $k > 6$.

In their graph, an edge is introduced between a and the point in P contained in each cone that minimizes distance from a projected along the axis of the cone. A graph that minimized Euclidean distance would work as well, but it appears more difficult to compute, since Yao [71] has shown that we can compute the exact EMST given such a graph. To find a path that approximates distance between a and b, we first traverse the edge $\{a, b'\}$, where b' is neighbor of a in the cone containing b. We recursively construct a path from b' to b.

To see intuitively why such a path approximates Euclidean distance, note that in any edge traversal, we are guaranteed to head in approximately the direction of our target, and we are not allowed to overshoot it by too much. Hence, the progress that we have made is proportional to the distance we have traveled. After traveling sufficient distance by such moves, we are guaranteed to reach the destination. Ruppert and Seidel prove an approximation factor of $1/(1-2\sin\pi/k)$, which is clearly $1 + \mathrm{O}(k^{-1})$. One can bound the number of cones to $\mathrm{O}(k^{d-1})$ in general and exactly k in the planar case. The size of the graph is thus $\mathrm{O}(k^{d-1}n)$.

We construct a similar graph by letting the u vectors defined in the preceding section serve as the axes of our conical space decomposition. It is easy to verify that for all $a \in A_i$, $b \in B_i$ in a well-separated pair decomposition with separation s, the vector $a - b$ and some vector u in our set form an angle bounded by $\mathrm{O}(s^{-1})$. We refer to such a u as the *approximate direction* of the pair $\{A_i, B_i\}$, since it approximates the direction from B_i to A_i. We choose s so that this angle is bounded by π/k. Note that $s = \mathrm{O}(k)$.

We define our graph in terms of the edges incident to any point $a \in P$. For convenience, assume that if $a \in A_i \cup B_i$ for some pair $\{A_i, B_i\}$ then $a \in A_i$. This is

just a matter of naming the sets. For every u vector, a has an edge to the point b, if one exists, that minimizes $-b \cdot u$ over all i for which u is the approximate direction of the pair $\{A_i, B_i\}$ and $b \in B_i$.

The size of such a graph is $O(s^{d-1}n)$ by the claim of the preceding section; this is $O(k^{d-1}n)$. To see that it is an approximation to the complete Euclidean graph, consider the following method of constructing a path from a to b. Let $\{A_i, B_i\}$ be the unique well-separated pair such that $a \in A_i$, $b \in B_i$. Then traverse the edge $\{a, b'\}$ corresponding to the approximate direction u of $\{A_i, B_i\}$, and recursively construct a path from b' to b. Because our bounds on the angles are the same as [57], we achieve the same approximation.

Using the well-separated pair decomposition, we can efficiently compute our graph. It is easy to see that the edge from a corresponding to u must be to some $b(u, B_i)$ (as defined in the preceding section) such that $a \in A_i$. We can compute all $b(u, B_i)$ bottom-up in $O(k^{d-1}n)$ steps.

For every S in the decomposition tree, let $\hat{b}(u, S)$ denote the point $b = b(u, S')$ that minimizes $-b \cdot u$ over all S' such that $\{S, S'\}$ is a well-separated pair with approximate direction u. If no such point exists, let $\hat{b}(u, S)$ take a special value such that $-\hat{b}(u, S) \cdot u = +\infty$. We can compute all of these values in time $O((k^d \log k)n)$, since there are $O(k^d n)$ pairs to consider and we can find u for each pair in $O(\log k)$ steps as in the preceding section.

Then, the edge from a corresponding to u is simply the point $b = \hat{b}(u, S)$ that minimizes $-b \cdot u$ over all nodes S in the path from the root of the decomposition tree to the node a. This can be computed using a top-down process in time $O(k^{d-1}n)$. Including the time to construct the well-separated pair decomposition, we can find the approximation to the complete Euclidean graph in time $O(n \log n + (k^d \log k)n)$. This is considerably faster than the time bound of Ruppert and Seidel for $d > 2$ and the resulting graph also contains $O(k^{d-1}n)$ edges. Our constant will in general be somewhat larger. Taking $\epsilon = O(k^{-1})$, we have the following theorem.

Theorem 3.4.2: *Given a set P of n points in $\Re^d$, we can construct an ϵ-approximate*

complete Euclidean graph of P having $O(\epsilon^{-d+1}n)$ edges in $O(n \log n + (\epsilon^{-d} \log \frac{1}{\epsilon})n)$ steps. □

Even for the planar case, our algorithm can be faster, because the dependence on k is added to the dependence on $\log n$, not multiplied. In this case, we can improve our constant by choosing $2k$ approximate-direction vectors placed uniformly at angles of π/k, each acting as the axis of a degree $2\pi/k$ planar cone. Every point in the plane is then covered by two overlapping cones. The resulting graph is only twice as large as that of Ruppert and Seidel [57].

Note that we cannot succeed in using k non-overlapping cones because, even for large s, we might have vectors from the same well-separated pair that lie in adjacent cones, making it impossible to choose an approximate direction for that pair.

3.5 Exact EMST

Agarwal et al. [4] have considered the relationship between the Euclidean minimum spanning tree and the bichromatic closest pair. In the latter problem, we are given a set of n red and m blue points, and are asked to find the closest pair of points such that one is red and the other is blue. They have shown that if $T_d(m,n)$ is the time to find the bichromatic closest pair in d dimensions, then an EMST for n points can be constructed in time $O(T_d(n,n) \log^d n)$. Moreover, if $T_d(n,n) = \Omega(n^{1+\alpha})$ for some fixed $\alpha > 0$, then the time becomes $O(T_d(n,n))$.

In this section, we show how to use the well-separated pair decomposition to compute the EMST in time $O(T_d(n,n) \log n)$. Our time complexity will also become $O(T_d(n,n))$ if $T_d(n,n) = \Omega(n^{1+\alpha})$. For $d > 2$, all known algorithms require $\Omega(n^{1+\alpha})$ to compute the bichromatic closest pair between n red and n blue points, so this is not an immediate improvement over the first algorithm. However, our result sheds additional light on the relationship between the EMST and bichromatic closest pair. Also, if

$T_d(n,n)$ could be reduced to $O(n \operatorname{polylog} n)$ time, then our result would represent a substantial improvement. While this may be too optimistic in the most general case, it seems attainable for a wide variety of input distributions that occur in practice.

The intuition behind our algorithm comes from the observation that if we compute the all-nearest-neighbors of a point set P, then these pairs already give us at least half the edges of our minimum spanning tree. Optimal algorithms exist to compute all-nearest-neighbors, so it is natural to ask whether similar algorithms can be run through $\log_2 n$ stages, reducing the number of unknown edges in half at each stage, and finally resulting in the spanning tree.

We show how the k-nearest-neighbors algorithm of section 3.1 can be adapted for this purpose. In order to accomplish this task, we will invoke bichromatic closest pair as a primitive operation at each stage. For simplicity, we assume that all interpoint distances in P are distinct, breaking ties in some systematic way. Our procedure is based on Sollin's Algorithm [16] for computing the minimum spanning tree of a general graph. A high-level outline is as follows.

We proceed in $\lceil \log_2 n \rceil$ stages. At stage $i = 0$, every point in P is a separate component in a trivial spanning forest. We find, for each component containing less than 2^{i+1} vertices, the nearest point not in that component, which we will call its *nearest outside point*. In the following, the term "nearest neighbor" of some component will be generalized to refer to the component containing its nearest outside point.

We add the minimum edge connecting this point to the component, thus merging the component with its nearest neighbor. The same edge may be introduced in two different ways, in which case we add just one copy. It is easy to verify that no cycles are possible as long as distances are distinct. At the ith intermediate stage, components have size at least 2^i, and the final stage is the EMST of P.

Note that the first stage of this algorithm is simply the all-nearest-neighbors problem. Later stages correspond to the same problem generalized to sets of points. To solve this problem, we first construct for each component of the current forest,

a set of candidate points that contains the nearest outside point of that component. For the sake of efficiency, we will need to guarantee that the sum of the number of candidates over all components is $O(n)$.

We will search for candidates top-down on the tree, maintaining, for each node in the tree, a list of all the components that can have candidates in the subtree of that node. Using the well-separated pair decomposition in a manner almost identical to our k-nearest-neighbors algorithm, we can insure that the size of each list is $O(1)$. Ultimately, we will "push" these lists down to the leaves, and the candidates corresponding to a given component will be just those leaves that contain that component in their lists. We apply a bichromatic closest pair algorithm to extract the nearest outside point of each component from the set of candidates.

The lists maintained around each node will consist of points, as in our k-nearest-neighbors algorithm, but each point will represent a whole component. In general, if a point a is paired with a node B, then the distance between a and any point in B will be a constant approximation of the distance between that point and the component containing a. Intuitively, if too many points are paired with B, then one of them will represent a component that is closer to another point in the component list than to any point in B. Such a component can be eliminated.

Before beginning the top-down pass, we eliminate, for each B, the well-separated pairs $\{A, B\}$ such that the points in A do not all lie in the same component. For separation $s > 2$, this is justified, because otherwise it is easy to verify that each component represented in A has a outside neighbor in A nearer to it than any point in B. The argument is similar to the proof of Lemma 3.1.1. For each A that remains, we choose some representative a, and associate each B with the set of a obtained in this manner.

At this point, the algorithm follows the top-down pass of the k-nearest-neighbors procedure of section 3.1, with k equal to 1. We find the candidates for each component by associating it with those points b such that a lies in the component and is paired with the leaf b after the completion of the top-down pass. Note that there

is at most one point a for each cone in the frame around b. From this it follows that the total number of candidates over all components is $O(n)$. Moreover, any point in any such cone that was rejected at some step in the procedure is closer to a than to b. Hence, if it lies in a component other than that represented by a, then this component cannot contain b as its nearest outside point. This guarantees that the candidate set of any component contains its nearest outside point.

Now it remains to extract the nearest outside point to each component. Let k be the number of components containing less than 2^{i+1} vertices, and let P_j be the set of candidates for the jth component.

For each component, find its nearest neighbor as follows. Split P_j into $\lceil |P_j|/2^{i+1} \rceil$ subsets of size at most 2^{i+1}, and solve the bichromatic closest pair problems obtained by coloring the points in each subset red and those in the component blue. The bichromatic closest pair that minimizes distance over all those computed is the pair connecting the component to its nearest neighbor.

We invoke bichromatic closest pair at most $\sum_{j=1}^{k} \lceil |P_j|/2^{i+1} \rceil$ times. We know that $k = O(n/2^i)$ at stage i, so this sum is $O(n/2^i)$. The sets in each invocation have size at most 2^{i+1}. Having found the closest pair between components, we merge them and continue with the next phase. The cost of merging can be amortized over the whole algorithm without affecting the complexity.

$O((n/2^i)T_d(2^{i+1}, 2^{i+1}))$ time is spent computing bichromatic closest pairs at each phase. Since T_d is at least linear, this dominates the time for each phase. For some constant c, the number of steps summed over all phases is at most

$$\sum_{i=0}^{\lceil log_2 n \rceil} \frac{cn}{2^i} T_d(2^{i+1}, 2^{i+1}).$$

From the above, we have the following theorem.

Theorem 3.5.1: *Given a set P of n points in $\Re^d$, we can construct the exact EMST of P in $O(T_d(n,n) \log n)$ steps. If $T_d(n,n) = \Omega(n^{1+\alpha})$, then the time complexity becomes $O(T_d(n,n))$.* □

3.6 Enumerating k smallest distances

As a final application, we will present an algorithm for enumerating the k smallest interdistances in P in optimal $\Theta(n \log n + k)$ steps. This result has been obtained previously by Salowe [60] and by Lenhof and Smid [47] without the use of the well-separated pair decomposition, but the algorithm we present here appears considerably simpler. The complexity will depend on combinatorial results similar to those used in previous algorithms for this problem, but which can be shown very simply using the well-separated pair decomposition.

Let $\{\{A_1, B_1\}, \ldots, \{A_m, B_m\}\}$ be the set of well-separated pairs in some realization listed in increasing order of $d(R(A_i), R(B_i))$. For any well-separated pair $\{A, B\}$ it is clear that the number of interdistances represented by the pair $\{A, B\}$ is $|A||B|$, and we will use this fact in our algorithm.

To enumerate the k smallest distances, we apply the following simple algorithm. First, we determine the smallest i such that $\sum_{j=1}^{i} |A_j||B_j| \geq k$ and retrieve all of those pairs $\{A, B\}$ such that $d(R(A), R(B)) \leq (1 + 4/s)r$, where $r = d(R(A_i), R(B_i))$. One can easily verify that any interdistance $d(a, b)$ not covered by one of the retrieved pairs is greater than any covered by a pair in $\{\{A_1, B_1\}, \ldots, \{A_i, B_i\}\}$ and is therefore not one of the k smallest distances. We then construct the set S_{pairs} of all pairs $\{a, b\}$ such that $a \in A$ and $b \in B$ for one of the pairs we retrieved, and apply k-selection to this set. From the above, it is clear that this algorithm will produce the k smallest distances, so it remains only to analyze its complexity.

The retrieval of well-separated pairs requires $O(n)$ time, since it amounts to a weighted generalization of the well-known linear-time selection algorithm. The remainder of the algorithm requires $O(|S_{\text{pairs}}|)$ steps, again by applying linear-time selection. To guarantee efficiency, we must show:

$$|S_{\text{pairs}}| = O(k + n). \tag{3.4}$$

To see this, first note that the contribution of pairs in $\{\{A_1, B_1\}, \ldots, \{A_{i-1}, B_{i-1}\}\}$ to $|S_{\text{pairs}}|$ is easily seen to be at most k, so we concentrate on retrieved pairs $\{A, B\}$

that do not lie in this set. Such pairs satisfy $d(R(A), R(B)) \in [1, 1 + 4/s]r$. Let S_r denote the set of all such pairs.

To show that (3.4) holds, we will require a combinatorial result (Lemma 4.6.1), which will be proven in the next chapter. This result implies that for all nodes A' such that $R(A')$ lies within a d-ball of radius r, there are $O(1)$ pairs $\{A, B\} \in S_r$ such that $A \subseteq A'$, because all pairs in S_r have approximately the same distance, and are packed into roughly the same location in space.

Assume w.l.o.g. that $|A_j| \geq |B_j|$ for all $\{A_j, B_j\} \in S_r$. Let $\{A'_1, \ldots, A'_l\}$ be the set of all maximal A_j in S_r (i.e., those that are not a subset of any other). Note that for sufficiently large s, the interdistance $d(a, a')$ for any $a, a' \in A'_j$ for $j = 1, \ldots, l$ must be one of the k smallest. Moreover, all A'_j are disjoint. Thus, it follows that

$$k \geq \sum_{j=1}^{l} \binom{|A'_j|}{2}. \tag{3.5}$$

We know from Lemma 4.6.1 that there exists some c independent of n such that for any choice of j, there are no more than c pairs $\{A, B\} \in S_r$ such that $A \subseteq A'_j$. It follows that the contribution to $|S_{\text{pairs}}|$ from pairs in S_r is at most $\sum_{j=1}^{l} c|A'_j|^2$, which is easily seen from (3.5) to be $O(k + l)$, which is $O(k + n)$.

Using (3.4) to analyze the complexity of the algorithm given above, we obtain the following theorem:

Theorem 3.6.1: *Given a set P of n points in $\Re^d$, and a canonical realization of $P \otimes P$, we can enumerate the k smallest interdistances of P in $O(k + n)$ steps.* □

Corollary 3.6.2: *The k smallest interdistances in P can be enumerated in $O(n \log n + k)$ steps.*

A related problem is to compute all interdistances below some fixed radius r. It is easy to see that the above algorithm can be modified to solve this problem. The only change we make is to omit the first step of computing i and setting r to $d(R(A_i), R(B_i))$. We instead use the value of r that is given. The time complexity is again $O(n \log n + k)$, where k is the number of interdistances less than r.

Chapter 4

Combinatorial properties

In this chapter we consider the well-separated pair decomposition in greater detail, and derive several combinatorial results. These properties are interesting in their own right, and several will be useful in applications. The lower bound on depth (section 4.1) was originally presented in [19, 20].

4.1 Minimum depth for linear size realizations

It is easy to see that the depth of fair split trees can be linear in the worst case. One is tempted to modify the definition of a fair split tree to insure that its depth is bounded by a polylog function, since this would greatly simplify parallel algorithms that used it. In this section, we will show that there exist point sets P, such that any tree permitting a linear size well-separated realization of $P \otimes P$ must have depth $\Omega(n^\epsilon)$, for some $\epsilon > 0$.

We show that the lower bound holds even for $d = 1$, by constructing a class of one-dimensional sets for which the bound is attained. Let $\alpha = 2/s + 1$, where s is the separation. For all $n \geq 1$, define $\tilde{P}_n$ to be the set of points $\{\alpha^0, \ldots, \alpha^{n-1}\}$ on the real line.

We show that if the depth of any binary tree associated with $\tilde{P}_n$ is $d(n)$, then the number of pairs in a well-separated realization is at least $cn \log n / \log d(n)$

for some constant $c > 0$. The proof involves a reduction to a simple coding problem. Consequently, if the total number of pairs is no more than en for some constant e, then $d(n)$ is no less than $n^{c/e}$.

Assume $\tilde{P}_n$ has an associated binary tree T, and consider a well-separated realization of $\tilde{P}_n \otimes \tilde{P}_n$, denoted $\{\{A_1, B_1\}, \ldots, \{A_k, B_k\}\}$, that uses T. For all $j = 1, \ldots, k$, A_j and B_j are well-separated, so one of them lies entirely to the right of the other. Assume w.l.o.g. that it is B_j. We note the following property.

Lemma 4.1.1: *For all $j = 1, \ldots, k$, $B_j = \{\alpha^i\}$ for some i such that $1 \leq i \leq n-1$.*

Proof. First note that B_j cannot contain α^0 since there is a non-empty set A_j to its left. Now, suppose there were a set B_j containing two points α^i, $\alpha^{i'}$ with $i < i'$. Let r denote the radius of the smallest d-ball containing $R(B_j)$. Clearly, $r \geq (\alpha^{i'} - \alpha^i)/2$. Because $\alpha^{i'} \geq \alpha^{i+1}$, we have $r \geq \alpha^i(\alpha - 1)/2 = \alpha^i/s$, or $\alpha^i \leq sr$. Since A_j is non-empty and to the left of B_j, it contains at least one point in the interval $(0, \alpha^i)$. But all points in this interval are at a distance less than α^i from B_j. Hence, A_j and B_j are not well-separated, giving us a contradiction. □

For all $i = 1, \ldots, n-1$, let S_i denote the set of all j, $1 \leq j \leq k$, such that $B_j = \{\alpha^i\}$. For all $i' = 1, \ldots, i-1$, there must exist a unique $j \in S_i$, such that $\alpha^{i'} \in A_j$. Hence, it is clear from Lemma 4.1.1 that

$$\bigcup_{j \in S_i} A_j = \{\alpha^0, \ldots, \alpha^{i-1}\}. \tag{4.1}$$

and $k = \sum_{i=1}^{n-1} |S_i|$.

Since we are proving a lower bound, we place no restrictions on the tree associated with a point set P, except to assume w.l.o.g. that it is binary. We will refer to this tree as the *subset tree* of P.

Lemma 4.1.2: *There exists a constant $c > 0$ such that for all n, there is a point set P, with $|P| = n$, such that for all subset trees of P with depth at most $d(n)$, the size of the smallest well-separated realization of $P \otimes P$ that uses the subset tree of P is at least $cn \log n / \log d(n)$.*

Proof. Consider a subset tree of $\tilde{P}_n$, for some $n \geq 1$, with depth at most $d(n)$. For every node in the subset tree, find the maximum i such that α^i is in the set at that node. We observe that the maximum of every internal node A is the same as that of one of its children, which we call A_1, and greater than that of its other child, which we call A_2. For every A, we assign 1 to the edge $\{A, A_1\}$ and 0 to the edge $\{A, A_2\}$, resulting in a $\{0,1\}$ assignment for all the edges of T.

Using (4.1), we can easily show that for every i, $|S_i|$ is at least as large as the number of 1's on the path from the root to the leaf α^i. Since $k = \sum_{i=1}^{n-1} |S_i|$, we can find a lower bound for k by seeking a tree with n leaves, having depth no more than $d(n)$, with an assignment of 0's and 1's to its edges such that the edges out of each node are labeled distinctly, and the sum of the number of 1's on each root-to-leaf path is minimized. This is equivalent to finding an assignment of $\{0,1\}$ strings to n numbers such that no two numbers are assigned the same strings, no string is longer than $d(n)$, and the total number of 1's is minimized.

Note that the number of strings we can represent using at most l 1's is bounded above by $\sum_{i=0}^{l} \binom{d(n)}{i}$. Using the crude upper bound of $(l+1)(d(n))^l$, we argue that there is a constant $c > 0$ such that if $l \leq 2c \log n / \log d(n)$, then we can represent at most $n/2$ strings. It follows that at least $n/2$ strings contain more than $2c \log n / \log d(n)$ 1's, and this establishes the lemma. □

As a consequence, we have the following theorem.

Theorem 4.1.3: *For any n and d, there exists a set P of n points in $\Re^d$ such that any subset tree of P that results in $O(n)$ well-separated pairs must be of depth $\Omega(n^\epsilon)$, for some $\epsilon > 0$.*

Proof. If en is a bound on the number of pairs, then choose $\epsilon = c/e$, and apply Lemma 4.1.2. □

Somewhat surprisingly, we can always construct an $O(\log n)$-depth tree that permits an $O(n \log^2 n)$ realization. We form such a tree from a fair split tree T as follows. Assume that the leaves of T are numbered in inorder. Then any node in T

contains all the leaves whose numbers fall within some interval. Let T' be a $\lceil \log_2 n \rceil$ depth tree whose leaves have the same inorder numbering. Clearly, any node in T can be represented by the union of $O(\log n)$ nodes in T'. It follows that any well-separated pair in a decomposition that uses T can be represented by $O(\log^2 n)$ well-separated pairs in a decomposition that uses T', resulting in a realization of size $O(n \log^2 n)$. Note that the nodes of T' are no longer defined by splitting hyperplanes.

Using a more careful balancing method, we can improve on the preceding observation to match the lower bound for $O(\log n)$ depth trees. The technique and proof of correctness is more involved and will be treated in section 4.4.

Rather than bounding the number of pairs of the form $\{A, B\}$, one may sometimes be interested in bounding the sum of $|A| + |B|$ taken over all pairs. Unfortunately, this can be $\Theta(n^2)$ in the worst case, as can be seen from the construction used in the preceding lower bound. This follows from the fact that every well-separated pair in the decomposition of $\tilde{P}_n$ must contain a singleton element. Because the realization is an exact covering of the set of unique pairs, the sum of $|A||B|$ taken over all pairs is exactly $\binom{n}{2}$. Hence, the sum of $|A| + |B|$ taken over all pairs is exactly $\binom{n}{2} + k$, where k is the number of pairs in the realization.

4.2 Properties of the contracted tree

Despite the fact that deep trees are necessary to maintain a linear bound on the size of the decomposition, there may still be applications in which we wish to use trees of low depth, and are willing to pay a penalty in terms of the number of pairs. It will turn out that a fair split tree can be contracted in a natural way to form an $O(\log n)$ depth tree with several useful properties.

The contracted tree may itself be used for the decomposition, in which case we will be able to compute a realization of size $O(n \log n)$. Also, we will use its properties to bound the number of pairs in a realization that uses the original fair split tree, but in which each well-separated pair must contain at least one node that

is a leaf, corresponding to a singleton point set.

Given a fair split tree T, we wish to consider a balanced binary tree partition T' with the property that the subtree of any node in T' is a tree partition of the nodes in a subtree of T from which at most one node and its subtree have been removed. T' will be a subset tree by the definition of the preceding section, since each non-leaf node A will have two children whose sets can be merged to form the set A. A tree of depth $O(\log n)$ with this property can be constructed using techniques such as those found in [29, 32, 45].

We will follow the approach of Frederickson [32], and use a structure called a *directed topology tree*. The tree partition T' can be extracted from such a structure in a natural way, but the complete structure will have several advantages in terms of the analysis to follow. In Chapter 6, we will also see its usefulness for maintaining the decomposition dynamically.

Frederickson's approach involves the contraction of adjacent pairs of nodes of a tree into single nodes (called clusters) over $O(\log n)$ levels, eventually contracting the entire tree into one node. This is similar to the technique of *rake-and-compress* [2, 34, 45], but allows more flexibility in the choice of pairs to be contracted. Because the tree is binary (possibly including nodes with single children) a node (or cluster) is either degree-1 (a leaf), degree-2 (an internal node with one child), or degree-3 (an internal node with two children). In the following, we develop the main ideas of Frederickson's approach, although our notation will differ to some extent.

At each level, there are three allowable contractions (see figure 4.1):

(a) A degree-1 cluster is merged with its degree-3 parent to form a degree-2 cluster.

(b) A degree-1 cluster is merged with its degree-2 parent to form a degree-1 cluster.

(c) A degree-2 cluster is merged with its degree-2 parent to form a degree-2 cluster.

Level 0 of a restricted multi-level partition is the tree itself, and level i is formed by applying contractions from above set to the tree of clusters at level $i-1$.

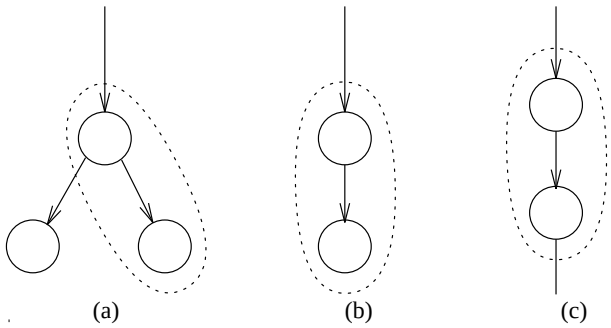


Figure 4.1: Node contractions in a restricted multi-level partition

The contractions applied at any level must be maximal in the sense that no pair of nodes that may be contracted is left uncontracted. However, the contractions need not be applied in any particular order, so there will in general be many ways of contracting a given level. Frederickson has shown that any valid way of contracting a level will reduce the number of clusters by a constant factor. A notable feature of this contraction scheme is that any degree-3 node at any level must correspond to a single node in the original tree.

While this structure is presented as a collection of levels, it can be interpreted as a tree T' in which there is a node v corresponding to each cluster at level $i+1$, whose children are the clusters at level i that were combined to form it. Note that at most two clusters can be combined at any level, so the tree is binary.

Because some clusters are restricted to have cardinality one, T' may contain internal nodes with a single child, violating the strict definition of a subset tree. However, this problem may be eliminated by contracting chains into single edges. The linear size of the directed topology tree follows from the fact that the size of any level is smaller than its predecessor by a fixed fraction less than one.

4.3 The level pairs in a restricted multi-level partition

One advantage to a directed topology tree is that we can consider each level to be a tree structure similar to a fair split tree, and construct a canonical pairing for it. The total number of pairs we obtain in this manner will be linear in the size of the original fair split tree, and these pairs can be used to define two variations of the well-separated pair decomposition.

Let T be a fair split tree, and suppose we construct a restricted multi-level partition of T. Let T_i denote the structure at each level of the partition, with $T_0 = T$. Each T_i will be a rooted tree in which each node has at most two children, but unlike the original fair split tree, some nodes (clusters of degree 2) may have only one child. The correspondence between nodes and point sets will not be as direct as in T, so we will assume that each node v in T_i is explicitly labeled by a set $\sigma(v)$.

These sets will be defined inductively as follows: The node v in T_0 corresponding to A in T will be labeled by the set A. For $i > 0$, any v in T_i inherited from level $i-1$ will be labeled by the same set as in level $i-1$. Otherwise, assume v is formed by combining two nodes w_1 and w_2 such that w_1 is the parent of w_2 in T_{i-1}. Then $\sigma(v) = \sigma(w_1)$. Note that each set of the form $\sigma(p(v)) - \sigma(v)$, is the set of some node in the balanced tree T'.

For each level T_i, we construct a set of *level pairs* as follows. Let $\{A, B\}$ be a pair in the canonical realization such that T_i contains nodes v and w satisfying $\sigma(v) \subseteq A \subset \sigma(p(v))$ and $\sigma(w) \subseteq B \subset \sigma(p(w))$. Note that if v or w exists, it is unique. Assume w.l.o.g. that $\sigma(p(v)) \preceq \sigma(p(w))$. Then the ordered pair (v, w) is a level pair in T_i.

Lemma 4.3.1: *For any node v in T_i, there are $O(1)$ level pairs of the form (v, w).*

Proof. We will charge each w paired with v to a node W in T such that W and $\sigma(p(v))$ are not well-separated, and $W \preceq \sigma(p(v))$.

There are two cases to consider. First, suppose (A, B) is the canonical pair resulting in the level pair (v, w). Then B and $p(A)$ are not well-separated, and it follows that B and $\sigma(p(v))$ are not well-separated. Moreover, $B \prec \sigma(p(v))$, so in this case, we let $W = B$. Otherwise, (B, A) must be a canonical pair. In this case, A and $p(B)$ are not well-separated, and $p(B) \preceq p(A) \preceq \sigma(p(v))$, so we let $W = p(B)$.

All sets B treated above must be disjoint. To see this, suppose otherwise, and let b be a point lying in two different sets. We know that $\sigma(v) \subseteq A$, so for any $a \in \sigma(v)$, there would have to be two pairs in the realization corresponding to the pair of points $\{a, b\}$, violating the definition of a realization. It follows that all W are disjoint, and at most two distinct pairs (v, w) are charged to the same W.

We know that $\sigma(p(v)) \preceq \sigma(p(w))$. By associating each $p(w)$ in T_i with an outer rectangle, we may apply the same packing argument used in the proof of Lemma 2.6.2 to bound the number of W. The number of W is therefore $O(1)$, sufficing to prove the bound. □

Corollary 4.3.2: *The total number of level pairs summed over all levels is* $O(n)$. □

4.4 Realizations for log-depth trees

In this section, we will show how to construct an $O(n \log n)$ well-separated realization that uses the balanced tree partition T'. We do this by first constructing a special type of $O(n)$ realization that we can charge to the level pairs. We require the following observation.

Observation 4.4.1: *For any node A in T, if there exists a node v in T_i such that $\sigma(v) \subseteq A \subset \sigma(p(v))$, then for all $i' < i$, there exists a node v' in $T_{i'}$ such that $\sigma(v') \subseteq A \subset \sigma(p(v'))$.*

Note that the subset condition will hold so long as A or one of its descendants in T is represented by some $\sigma(v)$ in T_i. Any $\sigma(v)$ represented at level i must clearly be represented at lower-numbered levels, so the claim follows.

Now we wish to construct a realization by representing every pair $\{A, B\}$ in the canonical realization by a collection of well-separated pairs that can each be charged to a level pair. We assume w.l.o.g. that for all i, if T_i contains a node w satisfying $\sigma(w) \subseteq B \subset \sigma(p(w))$ then it contains a v satisfying $\sigma(v) \subseteq A \subset \sigma(p(v))$. By the observation above, we can name the sets A and B so that this holds.

We construct a list of pairs $\{A, B_j\}$ $(j = 1, \ldots, k)$ such that each B_j is a node in the balanced tree partition T', and $\bigcup_{i=1}^{k} B_j = B$. Let w_i denote the node in T_i satisfying $\sigma(w_i) \subseteq B \subset \sigma(p(w_i))$, and consider any level i at which $\sigma(w_i) \neq \sigma(w_{i+1})$. Note that the only way for this to have occurred is for the node w_i to have been merged with $p(w_i)$ at level i. At this step, w_i must have had a single child labeled by $\sigma(w_{i+1})$. Thus, $\sigma(w_i) - \sigma(w_{i+1})$ is a node in T', and each such set can be used as a B_j value for $j = 1, \ldots, k-1$. Additionally, there is a level i such that w_i is defined and w_{i+1} is not defined. In this case, w_i must be a leaf in T_i that is merged with its parent. It follows that $\sigma(w_i)$ is a node in T', so this set can be used as B_k. One can easily verify that $\sigma(w_0) = B$, and it follows that $\bigcup_{i=1}^{k} B_j$ is a telescoping union equal to B.

By an assumption made previously, there must be a v satisfying $\sigma(v) \subseteq A \subset \sigma(p(v))$ at each level i that determines a value of B_j. Thus, at each such i there is a level pair corresponding to $\{A, B\}$, to which we may charge $\{A, B_j\}$. The set of all pairs of the form $\{A, B_j\}$ taken over all $\{A, B\}$ in the canonical realization is therefore a well-separated realization of $P \otimes P$ whose size is equal to the total number of level pairs. By Corollary 4.3.2, this is $O(n)$.

Finally, because T' is an $O(\log n)$-depth tree partition of T, each node A in T can be represented as the union of $O(\log n)$ nodes in T'. It follows that each pair $\{A, B_j\}$ in the realization defined above can be decomposed into $O(\log n)$ pairs of the form $\{A', B_j\}$, where each A' is a node in T'. This gives us an $O(n \log n)$ well-separated realization that uses T'. We summarize this in the following lemma.

Lemma 4.4.2: *For any point set P there is an $O(\log n)$-depth subset tree of P that permits a well-separated realization containing $O(n \log n)$ pairs.* □

4.5 One-to-many realizations

Another special type of realization that may lead to simplified algorithms is a one-to-many realization in which every pair has the form $\{\{a\}, B\}$ where a is a point and B is a point set. While a general realization is a decomposition of the Euclidean graph into complete bipartite graphs, a one-to-many realization is a decomposition into star graphs. Intuitively, it may be easier to develop algorithms for such a decomposition because we need only consider interactions between sets and points rather than between two sets that each contain more than one point. In this section we show that we can always construct such a realization with $O(n \log n)$ pairs.

The simplest way to construct such a realization is to begin with the canonical realization and decompose one of the sets in each pair into singletons. Assume that for each well-separated pair $\{A, B\}$ the sets are named so that $|A| \leq |B|$. Then, if $A = \{a_1, \ldots, a_m\}$, we replace $\{A, B\}$ with the list of pairs $\{\{a_1\}, B\}, \ldots, \{\{a_m\}, B\}$. Note that for every $a \in P$, there is a node $\{a\}$ in T, so the resulting realization uses T. Surprisingly, this simple procedure leads to a one-to-many realization of size $O(n \log n)$. To prove this, we develop a more involved decomposition of the canonical realization for which we can prove the bound, and observe that the simple procedure always produces the minimum one-to-many realization based on the canonical realization.

To prove the bound, we will present a new way of directing the pairs of the canonical realization. We will show that if we interpret each such pair as an arc between two nodes in T, then on any path from a leaf $\{a\}$ to the root of T, there are $O(\log n)$ out-going arcs. We can then construct an $O(n \log n)$ one-to-many realization by replacing each arc (A, B) on the path from a with the well-separated pair $\{a, B\}$.

It remains to show how to direct the arcs. For this purpose, we again make use of the level pairs of a restricted multi-level partition of T. However, we place an additional constraint on the clusters that can be paired at each level.

For any cluster v at some level, let $\pi(v)$ denote the total number of level

pairs of the form (v, w) or (w, v), and let $m = |T_i|$. It follows from Lemma 4.3.1 that $\sum_v \pi(v) = \mathrm{O}(T_i)$, so let κ be a constant (dependent on d and s) guaranteeing that $\sum_v \pi(v) \leq \kappa m$.

We add the constraint that no cluster v can be merged with another at level i if $\pi(v) > 5\kappa$. To see that this still leads to a $\mathrm{O}(\log n)$-depth $\mathrm{O}(n)$-size multilevel partition, we first note that without the new constraint, we can always form at least $n/4$ pairs of merged clusters. By an application of the pigeonhole principle, it is clear that at most $n/5$ nodes are prevented from joining such a pair. Hence, it is still possible to form $n/4 - n/5 = n/20$ pairs, guaranteeing that T_{i+1} is smaller than T_i by a constant factor. Note that this factor can be adjusted by relaxing the constraint, so that the number of pairs can be made to be $n/(4+\epsilon)$ for any $\epsilon > 0$.

Let $\{A, B\}$ be a well-separated pair in the canonical realization, and let $\{v, w\}$ be the level pair (either (v, w) or (w, v)) corresponding to $\{A, B\}$ at the highest level i for which such a pair exists. Assume w.l.o.g. that $\sigma(v) \subseteq A \subset \sigma(p(v))$ and v is a leaf in T_i. Note that if this did not hold for either A or B, then there would be a level pair corresponding to $\{A, B\}$ at level $i+1$. We direct $\{A, B\}$ using the arc (A, B).

Lemma 4.5.1: *Let $\{a\}$ be a leaf in T. The number of arcs (A, B) such that A is an ancestor of $\{a\}$ is $\mathrm{O}(\log n)$.*

Proof. For any (A, B), consider the lowest level i such that there is a leaf v' in T_i with $A \subseteq \sigma(v')$. We will charge (A, B) to the node v'. Because $a \in \sigma(v')$, and v' is a leaf, it follows that there is a unique v' at each level to which we may charge an arc. Moreover, at most 5κ pairs can be charged to any v', since our method of directing pairs guarantees that at level $i-1$, there was a level pair $\{v, w\}$ corresponding to (A, B) with $\sigma(p(v)) = \sigma(v')$. It must be the case that v was merged at this step, so $\pi(v) \leq 5\kappa$, bounding the number of these level pairs. Hence, for any given $\{a\}$, the number of arcs (A, B) is 5κ times the number of levels, which is $\mathrm{O}(\log n)$. □

By using the directed arcs to determine a one-to-many realization, we obtain the following corollary.

Corollary 4.5.2: *For any point set P for which we have constructed a fair split tree T, there is a one-to-many realization that uses T.*

Note that in any one-to-many realization constructed by directing pairs of some other realization, we obtain either $|A|$ or $|B|$ new pairs for each $\{A, B\}$ in the original realization. It follows that by directing the arcs from the set of smaller cardinality to the larger, we obtain the smallest one-to-many realization that can be constructed by directing the pairs of the canonical realization. Therefore, the algorithm given at the beginning of this section results in a one-to-many realization containing $O(n \log n)$ pairs.

In Chapter 6, we will present another way of directing pairs, based on a scheme involving weighted nodes, for which Lemma 4.5.1 also holds. This method will be preferable in some ways, particularly for the dynamic algorithms presented in that chapter. However it requires results that have not yet been presented.

4.6 Distance classes

For a variety of applications, we would like, given some $x \in P$, to bound the total number of pairs $\{A, B\}$ in the canonical realization such that $x \in A$. It is easy to see that this number can be as high as $n - 1$ in the worst case, which is not very illuminating. We can gain further insight into the properties of these pairs if we first group the pairs into classes, rather than merely counting the total number. In this section, we will define such classes in terms of approximate distance. This approach is natural, because a single well-separated pair represents a set of pairs of points all having roughly the same interpoint distance.

Given a well-separated pair $\{A, B\}$ we define its *approximate distance* $\tilde{d}(A, B)$ to be the distance from the center of $R(A)$ to the center of $R(B)$. Note that any dis-

tance between a point in $R(A)$ and another in $R(B)$ would be equally good for our purposes, but it is useful to settle on a unique value. Given a constant approximation factor ϵ, we define the *distance class* DC_r to be the set of all real distances r' lying in the interval $[(1-\epsilon)r, (1+\epsilon)r)$. For some purposes, it may be preferable to use the interval $[(1+\epsilon)^{-1}r, (1+\epsilon)r)$; essentially the same arguments apply to this case as well.

We are now ready to state the following lemma, which is the main result of this section. The result is somewhat more general than that suggested above, in that we are bounding all pairs with an endpoint lying near a point x. Clearly, this includes all those pairs for which one of the well-separated sets contains x. The more general result was used in section 3.6, and will also be used when we discuss dynamic algorithms.

Lemma 4.6.1: *For any x in $\Re^d$, and any distance r, the number of $\{A, B\}$ in the canonical realization such that $d(A, x) \leq r$ and $\tilde{d}(A, B) \in DC_r$ is a constant depending only on ϵ and d.*

Proof. We use a packing argument to bound the number of such pairs by bounding the number of nodes A and B that can appear in such a pair. Note that all A and B must overlap a cube C centered at x such that $l(C) = \alpha r$ for some ϵ-dependent constant α. To apply a packing argument, we need only show that $l_{\max}(p(A)), l_{\max}(p(B)) \geq \beta r$, for some constant β. Then, it can easily be seen that there are constantly many A and B for which such a pair can exist. We prove the preceding inequality as follows.

Choose β to be the largest number such that two rectangles each having maximum length βr are well-separated when the distance between their centers is r. The canonical pair that resulted in $\{A, B\}$ can be either (A, B) or (B, A). If it is (B, A), then it follows from our definitions that $A \prec p(B)$, and A and $p(B)$ are not well-separated. Hence, $l_{\max}(p(B)) \geq \beta r$, since otherwise, the first condition would imply that $l_{\max}(p(B)), l_{\max}(A) < \beta r$, and A and $p(B)$ would be well-separated.

Otherwise, the canonical pair must be (A, B). In this case, the above argument gives us $l_{\max}(p(A)) \geq \beta r$. It follows from our definitions that $p(A) \preceq p(B)$, completing the proof. $\Box$

Chapter 5

Parallel algorithms

In this chapter, we show that the well-separated pair decomposition can be used to design efficient parallel algorithms. This is the main advantage of our formulation over similar techniques that have made use of quadtree-like structures, such as Vaidya's all-nearest-neighbors algorithm [67, 69]. The difference between our approach and others is our relaxed splitting condition. This makes it possible to construct nodes deep in the tree without constructing their ancestors first. The same observation will also prove useful in developing dynamic algorithms.

We present an $O(\log^2 n)$ time n-processor parallel algorithm that uses the relaxed splitting condition to compute a fair split tree. We then show how the canonical realization can be constructed in $O(\log n)$ time, proving that an efficient parallel algorithm exists to compute the well-separated pair decomposition. Extending the techniques of the first section, we reduce the time complexity of the tree-construction algorithm to $O(\log n)$, proving that the well-separated pair decomposition can be constructed optimally in parallel. Finally, we present some parallel applications of the decomposition. The results of this chapter were originally presented in [18, 19, 20].

5.1 Parallel fair split tree construction

To parallelize the algorithm for constructing a split tree, it is sufficient to parallelize the splitting stage of the sequential algorithm (see section 2.2). Because each stage decomposes a set into sets that are no more than half of its size, the number of stages is $O(\log n)$, allowing the algorithm to be performed efficiently in parallel.

The split obtained by the parallel algorithm *might not be the same* as that of the sequential algorithm, since it is hard to guarantee that each rectangle will be split exactly in half perpendicular to its longest dimension. However, we can guarantee that it will be a fair split as defined earlier, so that the previous results will hold.

Our method will depend on the following lemma:

Lemma 5.1.1: *Let R be a d-dimensional rectangle containing a set of n points. Let S be the set of rectangles that are similar to R and share a corner with it and lie entirely within R. Assume that no two points ever touch the boundary of a rectangle in S. Then there exists a rectangle $R' \in S$, such that $l_{\max}(R') \leq \frac{1}{2} l_{\max}(R)$ and R' contains $\lfloor \frac{n}{2^d} \rfloor$ points in the set.*

Proof. Suppose we split R into 2^d similar rectangles by dividing it in half along each of its dimensions. Clearly, one of these rectangles contains at least $\lfloor \frac{n}{2^d} \rfloor$ points in the set. Choose such a rectangle and call it $\tilde{R}$. Now, consider all the similar rectangles contained in $\tilde{R}$ sharing the same corner of R as $\tilde{R}$, and which have a point touching their boundary. Since no two points touch the boundary of the same rectangle, one of these rectangles contains exactly $\lfloor \frac{n}{2^d} \rfloor$ points inside (including the point on the boundary). Call this rectangle R'. Note that $l_{\max}(R') \leq \frac{1}{2} l_{\max}(R)$ □

The assumption that no two points touch the boundary of a rectangle is included for simplicity. It is easy to generalize the algorithm when this assumption is omitted.

Our algorithm is as follows. We are given a set P containing n points, and its outer rectangle $\hat{R}(P)$, inherited from the preceding stage if P is not the

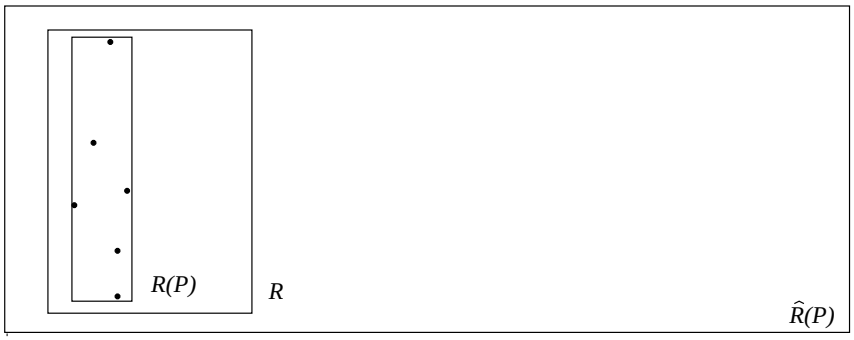


Figure 5.1: Choosing the rectangle R.

root. Let R be a rectangle containing P and contained within $\hat{R}(P)$, that satisfies $l_{\min}(R) \geq 2l_{\max}(R)/3$. Our splitting algorithm will guarantee that such an R always exists, though it may not be possible to satisfy this inequality using either $R(P)$ or $\hat{R}(P)$. In this case, R will lie somewhere between the outer rectangle and bounding rectangle, as shown in figure 5.1. Using R, we find a rectangle R' fulfilling the properties of Lemma 5.1.1.

We wish to decompose the set of m points not in R' into a collection of sets formed by a sequence of fair splits. Assume the points in R are sorted by increasing order of the size of the similar rectangle sharing the same corner as R', and with the point touching its boundary.

Our algorithm proceeds by divide-and-conquer. At each step, if we have not reached a terminal case, we can either split the problem with m points into two problems containing $\lfloor m/2 \rfloor$ and $\lceil m/2 \rceil$ points, or else we can split it into a terminal case and one containing no more than $\lceil m/2 \rceil$ points. The number of steps is thus $O(\log m)$.

At each recursive call, we are given the rectangles R and R', and must construct a split tree for $\langle R, R' \rangle$. This is a tree containing a chain of fair splits from R to R' that is consistent with some fair split tree of R. We assume the following

inequalities hold:

$$l_{\min}(R) \geq 2l_{\max}(R)/3 \tag{5.1}$$

and

$$l_{\max}(R') \leq 2l_{\max}(R)/3. \tag{5.2}$$

It is clear that these conditions are satisfied by our initial choice of R and R'. There are two terminal cases:

Terminal case 1: Suppose $m \leq 1$. Then there is at most one point outside R', and it can be separated from R' using a single fair split.

Terminal case 2: Suppose $l_{\max}(R') \geq l_{\max}(R)/3$. Then we decompose R by a succession of at most d fair splits. To do this, we consider each dimension of R in decreasing order of length. At each point, we can either perform a split with a hyperplane that contains the corresponding face of R', or else the region on the other side of this hyperplane is empty, and we do not need to perform a split.

Because R and R' are similar, it follows from (5.2) that the splitting hyperplane in dimension i lies at a distance of at least $l_i(R)/3$ from the nearest boundary of the current outer rectangle. Let $\tilde{R}$ denote the rectangle we are in the process of splitting, and consider any dimension i' for which we have already performed a split or concluded that a split is unnecessary. Then $l_{i'}(\tilde{R}) \leq 2l_{i'}(R)/3$, so by (5.1), $l_{i'}(\tilde{R})$ is no greater than $l_{\min}(R)$, which is no greater than $l_i(R) = l_i(\tilde{R})$ for any i. It follows that the maximum length dimension of $\tilde{R}$ remaining to be split is, in fact, the maximum length dimension. Combining this with the bound on the distance of the splitting hyperplane, we are guaranteed that the split is fair.

When we finish, the final set will contain exactly those points in R'. The result is a split tree that includes R' along with at most d other enclosing rectangles that partition the set of points outside R'.

Note that the terminal cases can result in point sets that violate $l_{\min}(\hat{R}(A)) \geq 2l_{\max}(A)/3$, creating difficulties in maintaining (5.1) for the next phase. However, they will always satisfy $l_{\min}(\hat{R}(A)) \geq l_{\max}(A)/3$. In $O(1)$ steps we can split any such point

set into one that satisfies the former inequality by a recursive process of splitting the bounding rectangle exactly in half along each dimension i in decreasing order of length until the inequality is satisfied. The split will never be repeated along the same dimension, so the result will be a tree of fair splits with depth at most d.

If neither of the terminal cases is true, we can find a rectangle R'' that satisfies one of the following cases:

(a) $3l_{\max}(R')/2 < l_{\max}(R'') < 2l_{\max}(R)/3$, and R'' contains $\lfloor m/2 \rfloor$ points not contained in R' (hence, R contains $\lceil m/2 \rceil$ points not contained in R'').

(b) $l_{\max}(R'') = 2l_{\max}(R)/3$ and R'' contains no more than $\lfloor m/2 \rfloor$ points not in R'.

(c) $3l_{\max}(R')/2 = l_{\max}(R'')$ and R contains no more than $\lceil m/2 \rceil$ points not in R''.

It is easy to verify that if cases (b) and (c) do not hold then (a) must hold. Hence, all possibilities are covered. We recurse on rectangle pairs $\langle R, R'' \rangle$ and $\langle R'', R' \rangle$. Note that these pairs satisfy (5.2). The split trees obtained for these can be combined to form a split tree of $\langle R, R' \rangle$. In all cases in which the number of points has not been cut in half, the corresponding rectangle pair is covered by the second terminal case. Hence, the depth of recursion is $O(\log n)$. We can obtain parallel time of $O(\log n)$ if each step can be done in constant time.

To perform each step in constant time, we need some pre-processing. For every corner of R, we construct an array of the points sorted in the order determined by the similar rectangles of R that share that corner. There are a constant number of corners, so we can construct the sorted arrays in $O(\log n)$ time with $O(n)$ processors. From these arrays, we can easily find R' and the associated corner.

Now, assume that we have a list of similar rectangles $R_1, \ldots, R_n$, sorted in order of increasing size, all sharing a corner, and each having a point in the set on their boundary. For each rectangle R_i, find the smallest rectangle $\text{next}(R_i)$ such that $l_{\max}(\text{next}(R_i)) \geq 3l_{\max}(R_i)/2$, and the largest rectangle $\text{prev}(R_i)$ such that $l_{\max}(\text{prev}(R_i)) \leq 2l_{\max}(R_i)/3$.

Given the above pre-processing, we can perform each non-terminal step as follows. We are given R and R', represented by the locations in the sorted array containing the smallest rectangles greater than or equal to each in size. Computing the midpoint of these locations can be done in constant time, and gives us an R'' fulfilling the properties of case (a) except, possibly, the condition $3l_{\max}(R')/2 < l_{\max}(R'') < 2l_{\max}(R)/3$. In this case, either a rectangle containing the points in $\text{prev}(R)$ satisfies case (b), or a rectangle containing the points in $\text{next}(R')$ satisfies case (c).

Thus, the splitting phase, including pre-processing, takes at most $O(\log n)$ time with n processors. The final result is a partial split tree in which all nodes A have size at most $\lceil \frac{2^d-1}{2^d} n \rceil$ and satisfy $l_{\min}(\hat{R}(A)) \geq 2l_{\max}(A)/3$.

Each point set A returned by this phase contains no more than $\lceil \frac{2^d-1}{2^d} n \rceil$ points, and its outer rectangle satisfies $l_{\min}(\hat{R}(A)) \geq 2l_{\max}(A)/3$

Now, we need only recurse on the point sets determined by the splitting phase. The condition on $\hat{R}(A)$ allows us to choose a suitable R for each point set in the next iteration. The depth of recursion will be at most $O(\log n)$, so the total parallel time will be $O(\log^2 n)$ using $O(n)$ processors. This gives us the following theorem.

Theorem 5.1.2: *Given a set P of n points in $\Re^d$, a fair split tree can be constructed in $O(\log^2 n)$ time using $O(n)$ processors.* □

5.2 Computing the pairs

Given the point set P, let T denote a fair split tree of P. We consider the problem of computing the well-separated pairs of the nodes of T. The pairs we compute will be the same as those produced by our sequential algorithm (section 2.3). In [19], we showed how to compute these pairs in $O(\log^2 n)$ time with $O(n)$ processors, and in this section, we improve the time bound to $O(\log n)$ using the same number of processors.

The idea behind our parallel algorithm is to retrieve the set of canonical pairs (A, B) (defined in section 2.3) simultaneously for all A. To do this, we develop a way to enumerate all B for any given A in $O(\log n)$ time with one processor.

Consider the problem of retrieving, for an arbitrary query cube C, all of those nodes B in T such that $l_{\max}(B) \leq \delta l(C) \leq l_{\max}(p(B))$ for some constant δ, and the outer rectangle $\hat{R}(B)$ overlaps C. Using a packing argument similar to that given in section 2.3, we can show that the number of such B is bounded by a function of δ and d, and is thus a constant.

We refer to the set of all such B as the δ-rendering of C, denoted $\rho_\delta(C)$. It is called a "rendering" by analogy to the graphics problem of rendering a continuous figure on a discrete bitmap. The set of all points such that each is contained in some $B \in \rho_\delta(C)$ is an approximate representation of the set of all points contained in C. That is, all points in C are represented in this set, but no points in this set lie outside a concentric cube C' such that $l(C') = (1 + 2\delta)l(C)$. An example of a δ-rendering is shown in figure 5.2.

To enumerate all B for a given A such that (A, B) is a canonical pair, we construct a suitably large cube C around $p(A)$. We then construct a δ-rendering of C, for δ, dependent upon s, and obtain a set of size $O(1)$ that contains all B paired with A. From this set, the pairs $\{A, B\}$ can be found in constant time by brute force. We now show how to solve this problem in $O(\log n)$ time using one processor.

We use a parallel tree contraction technique such as rake-and-compress [45] or the centroid decomposition [29] to construct a balanced tree partition T' such that the subtree of any node in T' is a tree partition of the nodes in some subtree of T from which at most one node and its subtree have been removed. Because T' is balanced, its depth is $O(\log n)$. T' shares essentially all useful properties of the balanced tree partition based on the directed topology tree of Frederickson [32] that was used in Chapter 4.

Note that the union of all outer rectangles of nodes in a subtree of T' consists of a rectangle containing at most one rectangular hole (i.e., the outer rectangle of the

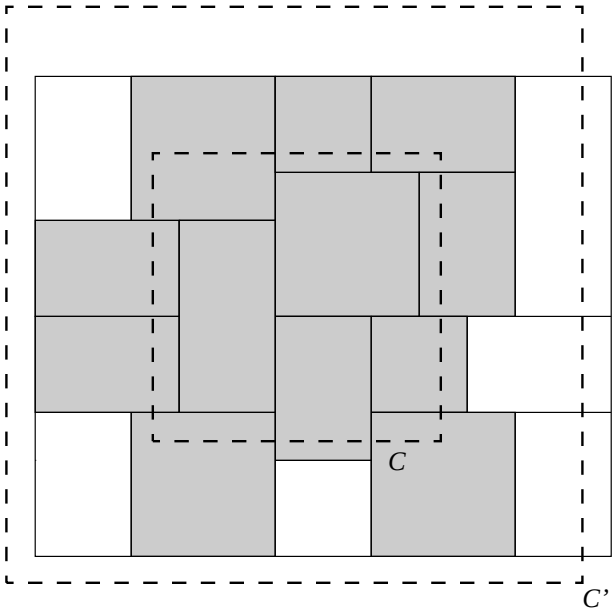


Figure 5.2: A δ-rendering of the cube C

missing subtree). We assume that each node in T' is labeled with such a geometric region. Given a node in T', we can therefore determine in constant time whether the outer rectangle of one of its descendants overlaps C.

To retrieve the set of B that overlap C and satisfy the size condition, we begin at the root of T' and only descend into subtrees corresponding to a sufficiently large region that overlaps C.

Observation 5.2.1: *In the above search process, we descend into a node iff it has a descendent in the δ-rendering of C.*

From the above observation, it is clear that every node at which we terminate this search process due to the size condition corresponds to some B in $\rho_\delta(C)$. Conversely, we determine all B in this manner, since we only skip nodes that are guaranteed not to contain such a B in their subtree. To analyze the complexity, we charge the work to the subtree consisting of all leaves B and their ancestors in T'.

This subtree has a constant number of leaves and a depth no greater than the depth of T', which is $O(\log n)$. Hence, the time to perform the search is $O(\log n)$.

We can find T' in $O(\log n)$ time with $O(n/\log n)$ processors on an EREW PRAM [29, 45]. After constructing T', we assign a processor to each node A in T, and perform the preceding search process for each A simultaneously. This may result in concurrent reads, so we must use the stronger CREW model. Therefore, the complete algorithm requires $O(\log n)$ time with $O(n)$ processors on a CREW PRAM.

5.3 Optimal parallel fair split tree construction

We now return to the problem of constructing the fair split tree, and show how to solve this problem in optimal work and running time. For this purpose, we use the technique of *multiway divide-and-conquer* [6, 11], in which a problem of size n is split into a set of subproblems, each of size $O(n^\alpha)$ for some $\alpha < 1$. This approach cannot be applied in an obvious fashion, since we must preserve the fair-split condition during the "divide" phase of our algorithm. For example, it will not suffice to divide the point set using hyperplanes in one dimension, since this may violate the fair-split condition. Instead, we must develop a non-trivial splitting phase that constructs a partial fair split tree whose leaves are rectangles, each containing $O(n^\alpha)$ points. This phase makes heavy use of the relaxed nature of the fair-split condition as contrasted with the less flexible characterization of quadtrees.[3]

The algorithm of this section is best viewed as a refinement of the original rather than a replacement. In fact, the divide-and-conquer within each phase of the original algorithm will be required as a subroutine for the new approach.

[3] A recent result of Bern et al. [17] on constructing quadtrees also leads to an optimal algorithm for this problem, but requires a more powerful machine model in which certain logical operations can be performed on integer words. Our algorithm requires only algebraic operations, as is standard in computational geometry

5.3.1 The constructible rectangles

We begin by splitting our point set into slabs in each dimension, using $\lceil n^{1-\alpha} \rceil + 1$ axis-parallel hyperplanes, spaced so that each slab contains at most n^α points. We will refer to these hyperplanes as *slab boundaries*. Recall that the definition of a fair split allows some freedom in the choice of the splitting hyperplane. In the present algorithm, we will use the slab boundaries to guide our choice of splits, thus limiting the kinds of rectangles that appear as nodes in the tree. In particular, we will be able to represent each side of a rectangle by a constant-size arithmetic expression in terms of slab boundaries and small integers. We will later show how to choose α to substantially limit the number of constructible rectangles. This will be essential in maintaining a reasonable processor bound.

Given a rectangle for which certain properties hold, we will develop a way of splitting it that results in new rectangles that preserve these properties and are constructible in a sense we will later make precise. In certain cases, we will not be able to split the rectangle, and must instead "shrink" it to a much smaller rectangle sharing a corner with the original and containing all but $O(n^\alpha)$ of the points in the original. This will lead to a special compressed edge in the resulting tree. After we have finished constructing the tree, we will expand each compressed edge into a chain of splits using a technique developed in our previous parallel algorithm (section 5.1). Finally, we will remove any rectangles that do not contain points, thus completing the splitting phase.

Let R be a rectangle that we wish to split. For the remainder of this discussion, let R' denote the largest rectangle that is contained in R and whose sides lie on slab boundaries. We will maintain the following invariants with respect to the rectangles we consider.

i. In each dimension, at least one side of R lies on a slab boundary.

ii. For all $i = 1, \ldots, d$, either $l_i(R') = l_i(R)$ or $l_i(R') \leq \frac{2}{3} l_i(R)$

iii. $l_{\min}(R) \geq \frac{1}{3} l_{\max}(R)$.

We now consider the three cases of our splitting rule. For the following, we use $i_{\max}$ to denote the longest dimension of R. The application of our splitting rule will be said to *produce* one or two new rectangles that must preserve the above invariants. In our illustrations, slab boundaries are shown with thin dotted lines, while each rectangle and its corresponding split are shown with thick solid lines. Although only planar rectangles are illustrated, the formal splitting rules extend to all $d \geq 1$.

Case 1: $l_{\max}(R) = l_{i_{\max}}(R')$ (see figures 5.3 and 5.4). In this case, both sides of R in the $i_{\max}$ dimension lie on slab boundaries. We find the slab boundary in this dimension that comes closest to splitting R into two equal halves. If (a) it lies at a distance of at least $\frac{1}{3} l_{\max}(R)$ from the closest side of R in the $i_{\max}$ dimension, we use it for our split. Otherwise (b), we split R into two equal halves in the $i_{\max}$ dimension. This case produces two rectangles, each lying on one side of the split.

Note that if case 1 does not hold, then by the second invariant,

$$l_{i_{\max}}(R') \leq \frac{2}{3} l_{\max}(R). \tag{5.3}$$

We will assume this inequality holds in the remaining cases. In the following, c is a constant whose value we will determine later.

Case 2: $l_{\max}(R') \geq c\, l_{\max}(R)$ (see figures 5.5 and 5.6). Exactly one of the sides of R in the $i_{\max}$ dimension lies on a slab boundary, which we denote by H. If (a) $l_{i_{\max}}(R') \geq \frac{1}{3} l_{\max}(R)$, we use the hyperplane containing the side of R' farthest from H for our split. Otherwise (b), we must choose another split that preserves our invariants. It is sufficient to use a hyperplane splitting R at a distance from H in the interval $(\frac{1}{2}, \frac{2}{3}] l_{\max}(R)$. We find the unique integer j such that $\frac{2}{3}\left(\frac{4}{3}\right)^j l_{\max}(R')$ lies in the interval, and use a hyperplane at this distance.

For technical reasons, we say that this case produces only one rectangle, namely the one containing R'. The other rectangle may be important in the tree we

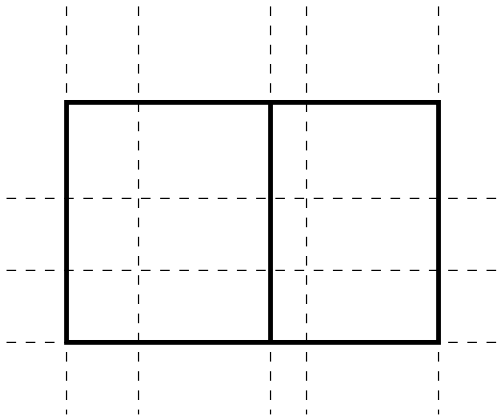


Figure 5.3: Case 1(a)

will ultimately construct, but we must defer its treatment to a later phase, because it will violate the first invariant after an application of case 2(b).

Case 3: $l_{\max}(R') < cl_{\max}(R)$ (see figure 5.7). For sufficiently small values of c, exactly one side of R lies on a slab boundary in each dimension. In other words, R' shares a unique corner with R. We construct a d-cube C containing R' such that $l(C) = \frac{3}{2}l_{\max}(R')$ and C shares the same corner with R as R'. We will later show how to find a sequence of fair splits decomposing the region between R and C into rectangles that each contain $O(n^{\alpha})$ points. This case produces the rectangle C.

We must prove that the above cases preserve the invariants. The first invariant follows immediately from the fact that we never split R' unless both sides parallel to the split lie on slab boundaries. The third invariant follows from the fair split condition. That is, after the split, the new minimum length cannot be less than $\frac{1}{3}l_{\max}(R)$, while the new maximum length cannot be more than $l_{\max}(R)$. The second invariant is somewhat more complicated to verify. If we split on a slab boundary, the invariant is clearly preserved. Otherwise, all slab boundaries passing through R in the $i_{\max}$ dimension must be at a distance of less than $\frac{1}{3}l_{\max}(R)$ from the sides parallel to them. Given this fact, it is not hard to see that our choice of a splitting hyperplane in cases 1 and 2 preserves the invariant. It is even easier to verify the invariant in

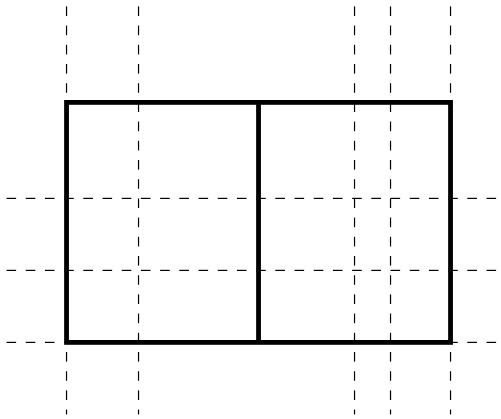


Figure 5.4: Case 1(b)

case 3.

In the above cases, the new rectangles produced by a split can always be encoded in terms of a constant-size list of slab boundaries and O(1) bits of extra information. We will consider case 2(b) in some detail, because it is the most complex and allows the greatest number of new rectangles to be defined.

Suppose some dimension of a rectangle has been generated by an invocation of case 2(b). To fully specify this dimension, we must supply a slab boundary for one side of the rectangle, and a distance of the form $\frac{2}{3}\left(\frac{4}{3}\right)^j l_{\max}(R')$, from which we may compute the position of the other side. We may represent this distance by the two slab boundaries that determine $l_{\max}(R')$, and an integer j.

We consider the possible values for j. Recall that the distance we represent must lie in the interval $(\frac{1}{2}, \frac{2}{3}]l_{\max}(R)$. We know that $cl_{\max}(R) \leq l_{\max}(R') < l_{\max}(R)$, and the right-hand side immediately gives us $j \geq 0$. From the left-hand side, one may easily verify that $j \leq -\lfloor \log c / \log \frac{4}{3} \rfloor$. Since c is a constant, there are O(1) choices of j.

Clearly, rectangle dimensions generated by other cases of the splitting rule can be treated in a similar manner. For example, the hyperplane splitting a rectangle into two equal pieces in case 1(b) can be represented as the arithmetic mean of two slab

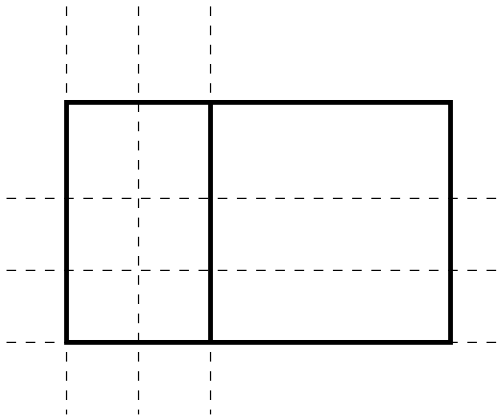


Figure 5.5: Case 2(a)

boundaries. An upper bound on the number of rectangles, valid to within a constant factor, may be obtained by considering only those rectangles whose dimensions come from case 2(b), since other cases result in fewer new rectangles. There are $O(n^{1-\alpha})$ slab boundaries, and at most $3d$ slab boundaries are required to represent a rectangle. This gives us the following result.

Lemma 5.3.1: *There are* $O(n^{3d(1-\alpha)})$ *constructible rectangles.* □

5.3.2 The production graph

Consider the directed graph in which each node corresponds to a constructible rectangle, and arcs are placed from it to any node it can produce by an invocation of the splitting rule. We call such a graph a *production graph* and denote it by G.

Let T_c be a subtree of G consisting of the nodes accessible from some initial rectangle R_0. We make the exception that a rectangle intersected by exactly two slab boundaries in the $i_{\max}$ dimension will be considered a leaf.

R_0 will normally be inherited from a previous phase in the algorithm, so the third invariant will be guaranteed automatically (initially, we use some d-cube that contains the entire point set). To guarantee the first and second invariants, we set

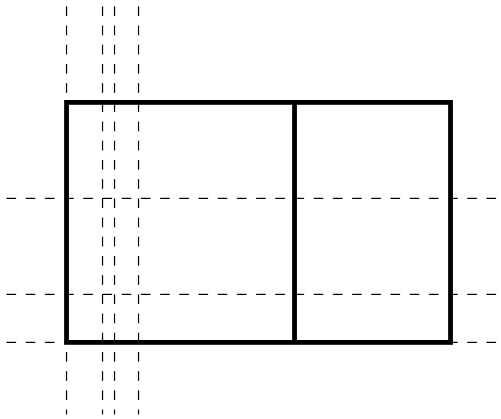


Figure 5.6: Case 2(b)

the extreme slab boundaries so that they contain the sides of the starting rectangle. For convenience, we may refer to nodes and rectangles interchangeably.

It is hard to see how to construct T_c without finding a transitive closure of G. We cannot rely on a general transitive closure algorithm, since this would be too time consuming. Fortunately, the structure of G is very restricted due to the following property:

Let u and v be two distinct nodes in G. If there exists a directed path from u to v, then this path is unique. In fact, given u and some v accessible from u, we can find the first arc along the path in constant time using the following observation:

Observation 5.3.2: *If there is a directed path from u to v in G, then the first arc in the path is the unique arc (u, w), such that (u, w) is an arc in G and the rectangle w contains the rectangle v.*

Consider the graph G', which contains a node $[u, v]$ for each pair of nodes in the production graph G. Let there be an arc from $[u, v]$ to $[w, v]$ iff (u, w) is an arc in G and v is contained in w. Nodes in G' have out-degree of at most 1 by the observation that (u, w) is unique. Hence, G' is a forest of rooted trees in which each node that is not a root has an arc leading to its parent. Each node of the form $[v, v]$

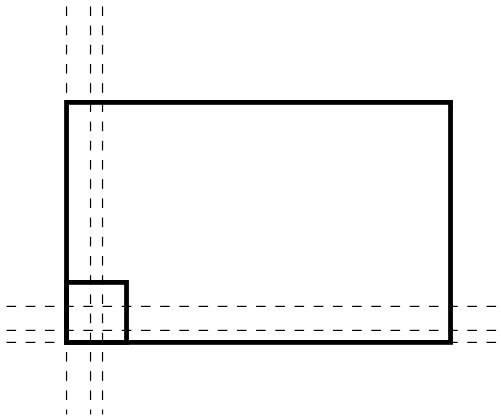


Figure 5.7: Case 3

is the root of some tree, and clearly v is accessible from u in G iff $[u, v]$ is a node in this tree.

5.3.3 Extracting T_c

Recall that for each v, we are ultimately interested in determining whether it is accessible from R_0 in G. In other words, we wish to know whether the tree rooted at $[v, v]$ contains the node $[R_0, v]$. With this observation, we are ready to present the complete algorithm for computing T_c:

i. Construct slabs in each dimension.

ii. Construct G from slab boundaries.

iii. Construct G' from G.

iv. Use G' to find v in G accessible from R_0.

Finally, T_c consists of those v accessible from R_0, together with their out-going arcs.

Correctness of the above algorithm follows from the preceding discussion. We now consider the parallel complexity of each step.

Step 1 can be solved in the time it takes to sort the points in each dimension. This can be done with n processors in $O(\log n)$ time on an EREW PRAM [26].

Step 2 can be solved by assigning a processor to each possible formula for constructing a rectangle, so there are $O(|G|)$ processors. Each processor can find an explicit value for its rectangle in constant time, and compute its out-going arcs in $O(\log n)$ time using a binary search on a sorted list of slab boundaries in the appropriate dimension.

Step 3 can be solved in constant time once we have assigned a processor to every ordered pair of nodes in G. Each processor finds the out-going arc, if it exists, of the corresponding node in G'.

Step 4 can be solved in several ways. Our approach is as follows. Assign a weight of 1 to all nodes in G' of the form $[R_0, v]$ and a weight of 0 to all other nodes. Now compute, for every node in G', the total weight of its subtree. The computed total for $[v, v]$ will be 1 iff $[R_0, v]$ is a node in its subtree, or equivalently, v is accessible from R_0 in G. This can be solved in $O(\log |G|)$ time with $O(|G|^2/\log |G|)$ processors on an EREW PRAM using the Euler tour technique [65].

According to Lemma 5.3.1, $|G| = O(n^{3d(1-\alpha)})$. Hence, if we choose some $\alpha \geq 1 - \frac{1}{6d}$, then $|G|^2 = O(n)$. Therefore, the above algorithm requires $O(\log n)$ time with $O(n)$ processors.

5.3.4 Constructing the partial fair split tree

We define a *partial fair split tree*, as in the sequential algorithm of Chapter 2, to be a tree that satisfies all the conditions of a fair split tree, except that its leaves need not be singleton point sets. In this section, we show how to construct a partial fair split tree from T_c.

First, recall that case 2 actually resulted in two rectangles, though we said it produced only one. This was necessary because the other violated the first invariant. To correct this problem, we treat the other as a leaf. We insert all such leaves in

constant time by assigning a processor to every rectangle R with a child produced by case 2.

At this point, the tree we have constructed defines a natural partition of the initial point set into points contained either in leaf rectangles, or else lying in the region between R and C in the nodes produced by case 3. We will need to compute this partition explicitly to continue our algorithm into the next phase. The fact that there are only $O(\sqrt{n})$ constructible rectangles allows us to solve this problem in $O(\log n)$ time on an EREW PRAM as follows.

Let P denote the set of points to be partitioned. We begin by partitioning space into the set of cells bounded by hyperplanes that occur as boundaries of constructible rectangles. The same argument used to bound the number of constructible rectangles will guarantee that there are $O(\sqrt{n})$ such cells. We use d sorting steps in each dimension to label each point in P with the pair of adjacent hyperplanes determining the slab that contains it. After sorting, which requires $O(\log n)$ time with n processors on an EREW PRAM, we have labeled each point with the cell that contains it. We also compute, for each cell, the number of points contained in it.

Now we go back to the regions defined by the tree partition. Each is either a rectangle or else a rectangle from which a cube has been removed, and it is easy to see that it is the disjoint union of a set of cells in the partition we have already computed. We label each cell by the region containing it. Because there are $O(\sqrt{n})$ cells and regions, this can be done efficiently by assigning a processor to every pair consisting of a cell and a region, and testing whether the cell is contained in that region. We accept only those pairs consisting of a cell and the region that contains it.

At this point, we have, for each cell, a pair that identifies the region to which it belongs. To assign points to regions, we first make, for each such pair, as many copies as there are points in the corresponding cell. We sort this list by cell and we sort the set of points by cell labels. Clearly, the ith point in the sorted list of points must lie in the region corresponding to the ith pair in the other list. Thus, we can

use these lists to label each point by the region that contains it. Having labeled the points according to region, we can compute the partition using a final sorting step.

We must further process T_c by expanding each edge produced by case 3 into a chain of splits. In presenting this part of our algorithm, we will derive the value of the constant c used in distinguishing cases 2 and 3.

Let R be a rectangle in T_c with child C produced by case 3. Construct the unique cube C' such that $l(C') = \frac{2}{3} l_{\min}(R)$, and C' shares a corner with both C and R. Find a sequence of splits from R to C' using a rule similar to case 1 in the preceding algorithm. Note that C' is sufficiently small to guarantee that such a sequence is possible, and sufficiently large to guarantee that only $O(1)$ splits are necessary. Hence, this can be done in constant time with one processor per rectangle R.

Now we need only find a sequence of fair splits between C' and C. Let m be the number of points inside C' and outside C. We have shown (section 5.1) how to find such a sequence of fair splits in $O(\log m)$ time with $O(m)$ processors. As a precondition, we require that $l(C') \geq \frac{3}{2} l(C)$. We must choose c consistent with this constraint. Recalling that $l(C) = \frac{3}{2} l_{\max}(R')$, we obtain the equivalent constraint $l_{\min}(R) \geq \frac{27}{8} l_{\max}(R')$. By the third invariant in our splitting rule, $l_{\min}(R) \geq \frac{1}{3} l_{\max}(R)$. Hence, if we let $c = \frac{8}{81}$, then the precondition on case 3 guarantees that the constraint is satisfied. If we expand all the compressed edges concurrently, then the total number of processors assigned cannot exceed $O(n)$, and the time will be $O(\log n)$.

To complete the splitting phase, we compress out any rectangles in the tree that do not contain points in the initial point set. This can be done as follows: Weight each leaf with the number of points contained in its rectangle. For each internal node, compute the total weight of the leaves in its subtree, and weight it with this value. Eliminate all nodes with weight zero, and compress non-branching paths to single edges.

Before compression, the tree cannot have size greater than $O(n)$. To see this, note that there were only $O(\sqrt{n})$ nodes before the expansion of case 3 edges, and any new rectangles generated by this expansion were guaranteed to contain points. Hence,

compression can be performed in $O(\log n)$ time with $O(n)$ processors on an EREW PRAM using standard techniques.

5.3.5 Correctness of the splitting phase

We use T_p to denote the tree ultimately constructed by the previous algorithm, and we wish to show that T_p is a partial fair split tree. Note that after the final compression, every leaf of T_p contains a non-empty point set, and every non-leaf node containing the point set P has two children formed by a split of P. It suffices to prove that such splits are fair.

We may focus our attention on those splits produced by cases 1 and 2 of the first part of the algorithm. Every node that is split in such a manner is labeled by a rectangle R, and it may easily be verified that the split is placed at a distance of at least $\frac{1}{3}l_{\max}(R)$ from the nearest boundary of R parallel to the split. Because R contains P, $l_{\max}(R) \geq l_{\max}(P)$. Moreover, the outer rectangle $\hat{R}(P)$ contains R, so the nearest boundary of $\hat{R}(P)$ is no closer to the split than the nearest boundary of R. Thus it follows that each such split is fair, and T_p is a partial fair split tree.

To prove the correctness of the splitting phase, we must finally argue that each leaf of T_p contains $O(n^\alpha)$ points. We should emphasize that the number of leaves in T_p will not necessarily be $O(n^{1-\alpha})$. In fact, we may create linearly many new leaves when we break up case 3 edges into chains of fair splits.

It suffices to show that the outer rectangle of every leaf lies entirely within a set of $O(1)$ contiguous slabs in some dimension, since each slab contains n^α points. This can be verified by considering all the termination conditions that result in leaves. Note that the precise sequence of splits used in expanding case 3 edges is not important, because any rectangle in the region inside R and outside C must lie in two adjacent slabs. Other cases are even easier to verify.

From the above discussion, we have the following lemma:

Lemma 5.3.3: *Let P be a set of n points in $\Re^d$, and let α be a real number in the interval $[1 - \frac{1}{6d}, 1)$. In $O(\log n)$ time with $O(n)$ processors on an EREW PRAM, we can construct a partial fair split tree T_p of P such that each leaf of T_p contains $O(n^\alpha)$ points.* □

5.3.6 The final algorithm

The above result leads directly to the following theorem.

Theorem 5.3.4: *Let P be a set of n points in $\Re^d$. We can find a fair split tree of P in $O(\log n)$ time with $O(n)$ processors on an EREW PRAM.*

Proof. Our algorithm is as follows: Let α be some constant greater than or equal to $1 - \frac{1}{6d}$ and less than 1 (the precise value may be chosen to optimize the constants in the complexity). For n less than some constant n_0, solve the problem by brute force. Otherwise, construct a partial fair split tree T_p, whose leaves are point sets of size $O(n^\alpha)$. Find fair split trees for these point sets by applying the algorithm recursively in parallel.

The point sets at the leaves form a partition of P, so the total number of processors remains $O(n)$ at all stages of the recursion. The total time $T(n)$ satisfies the recurrence $T(n) \leq \beta \log n + \gamma T(n^\alpha)$ for suitable choices of β and γ. From this, it is easily seen that $T(n) = O(\log n)$. □

Combining this with the result of the previous section, we obtain the following theorem.

Theorem 5.3.5: *Let P be a set of n points in $\Re^d$. We can find a well-separated pair decomposition of P in $O(\log n)$ time with $O(n)$ processors on a CREW PRAM.* □

5.4 Applications

The optimal parallel algorithm for computing the well-separated pair decomposition leads to efficient parallel algorithms for many applications of the decomposition. This

is one of the main advantages of our technique over other algorithms (e.g. Vaidya [67, 69]) that involve the computation of similar trees.

We will consider the parallel applications to n-body potential fields and k-nearest neighbors in detail, as these illustrate the main techniques involved in developing parallel applications of the well-separated pair decomposition. The algorithm for k-nearest neighbors requires some additional geometric analysis which will be presented as needed. We conclude the section by sketching the techniques involved in using the decomposition to parallelize other applications.

5.4.1 Parallel computation of n-body potential fields

It is well known that given an arithmetic expression tree whose leaves are constants and whose internal nodes are labeled by $+$ and $\times$ operators obeying certain algebraic properties, one can evaluate all nodes of this tree in $O(\log n)$ time with $O(n/\log n)$ processors on an EREW PRAM, where n is the size of the tree. A standard approach to this problem is to apply the technique of *rake-and-compress* [2, 34, 45], which is suitable for a wide variety of tree-structured computation problems. In this subsection, we sketch how the rake-and-compress technique can be applied to the parallel implementation of the fast multipole method.

First, note that the operation of adding two multipole or local expansions is both commutative and associative (though we will not make use of commutativity). This follows from the fact that it is merely the exact addition of two functions. The translation operation necessary for addition only changes the representation of the function, not the function itself. Hence, there is sufficient algebraic structure to apply rake-and-compress.

Let the multipole expansion at node A be denoted $M(A)$. For the first phase, we use the fair split tree to construct an expression tree in which each leaf $\{z_i\}$ is labeled by the function $M(\{z_i\})$ (computed for all leaves simultaneously in $O(1)$ steps), and each internal node is labeled by a $+$ operator denoting the addition of two

multipole expansions. The computation of $M(A)$ for all internal nodes A reduces to the problem of evaluating all nodes in this expression tree.

Now let $L(A)$ denote the local expansion at node A representing the contribution to potential due to all nodes with which A is paired in the realization. For the second phase, we first convert all multipole expansions to local expansions about the nodes with which they are paired. This can be done in $O(1)$ steps by assigning a processor to every pair in the realization. To evaluate $L(A)$ for any A, we must compute a sum of m_A terms, where m_A denotes the number of pairs of the form $\{A, B\}$. The sum of m_A over all A in T is exactly twice the number of pairs, which is linear, so this phase can be performed in $O(\log n)$ time with $O(n/\log n)$ processors on an EREW PRAM.

Finally, let $\Phi(A)$ denote the sum of all $L(A)$ on a path from the root of T to A. For the third phase, we evaluate this using rake-and-compress as follows:

At any step in forward raking, let A be a node with parent B in the compressed tree. Node A will contain the function $\Phi_B(A)$, denoting the sum of all local expansions on the path from B to A, not including $L(B)$. We can maintain these values using a constant amount of work each time we compress a leaf. During backward raking, we use these functions to find $\Phi(A)$ for each node A as it emerges in the process of decompression. Finally, we obtain $\Phi(\{z_i\})$ for all leaves, which is the local expansion about any point in P.

All of the preceding phases can be performed in $O(\log n)$ time with $O(n/\log n)$ processors, giving us the following theorem.

Theorem 5.4.1: *Given a set P of n points with associated charge strengths, and a well-separated pair decomposition of P, we can compute the potential at all points in P in $O(\log n)$ time with $O(n/\log n)$ processors.* □

5.4.2 Computing k-nearest-neighbors in parallel

The k-nearest-neighbors problem for points in fixed dimensions has received a great deal of attention [15, 25, 27, 28, 33, 48, 67, 69]. Prior to the present work, the fastest known parallel algorithm for arbitrary fixed dimension d was due to Frieze, Miller, and Teng [33]. Their algorithm, which is randomized, requires $O(\log n)$ time with $O(kn)$ processors in an EREW model that assumes that prefix sum can be performed in $O(1)$ steps. Cole and Goodrich [27, 28] have presented a deterministic $O(\log n)$ time $O(n)$ processor EREW PRAM algorithm for the all-nearest neighbors problem in the plane ($d = 2$). The algorithm we present here is, to our knowledge, the first optimal deterministic parallel algorithm for solving all-nearest-neighbors for arbitrary d, and requires $O(\log n)$ time with $O(n)$ processors on a standard CREW PRAM.

In [19] we sketched a parallel algorithm for the all-nearest-neighbors problem using rake-and-compress [2, 34, 45]. We can adapt this algorithm to the k-nearest-neighbors problem as follows.

At any stage of forward raking, let B' be the parent of B. At this stage, node B will store the set $N(B', B)$ defined below.

Let $f(B', B)$ be the set of all $a \in A$ such that $\{A, B''\}$ ($|A| \leq k$) is a well-separated pair for some node B'' on the path from B to B', B inclusive, in the original tree. Then $N(B', B)$ is the subset of $f(B', B)$ that includes the k nearest points to $O_{B'}$ in each cone in the frame around B'.

This algorithm can be made to run in $O(\log n \log k)$ steps with $O(nk/\log n)$ processors on an EREW PRAM. If we wish to reduce the time to $O(\log n)$ steps independent of k, we run into some difficulty, since it appears that we need to apply k selection at each of $O(\log n)$ phases of the algorithm. We can eliminate this using several further geometric observations.

Roughly speaking, the well-separated pairs corresponding to a given node tend to represent decreasing distances as the depth of the node increases. Hence, rather than applying k selection in a top down fashion, we can use the structure of

the fair split tree directly to obtain a set of $O(kn)$ pairs that contains all k-nearest-neighbor pairs.

In our sequential algorithm, we partitioned the space around each node in T into narrow cones, and selected the k nearest points within each cone. There was nothing to prevent points from changing cones during this process. If we are careful, however, we can choose our cones so that a point remains in the same cone as it is pushed down to the leaves of T.

For our purposes it will be convenient to define each cone by an axis vector u. Let α denote the angle and b denote the apex of such a cone. Then, the cone of u consists of all points x lying above the hyperplane passing through b with normal u such that $\overline{xb}$ makes an angle of at most $\alpha/2$ with u. We can choose a set U of vectors with $|U|$ dependent only on s and d, such that for any well-separated pair $\{\{a\}, B\}$, there is a $u \in U$ such that for all $b \in R(B)$, a lies in the cone of u with apex b and angle $\alpha = O(s^{-1})$. This follows easily from the geometry of well-separated sets.

For our parallel algorithm, we first need to construct a well-separated pair decomposition with $s > 2$ sufficiently large to insure that some $\alpha < \frac{s}{s+1}$ satisfies the above claim. The set of cones defined by the vectors in U covers the space around the origin in a manner similar to a frame, but the cones may overlap.

For every node B, we retrieve the set of a such that $\{A, B\}$ ($|A| \leq k$) is a pair in the realization and $a \in A$. Now, we associate every pair $\{B, \{a\}\}$ with some u such that a lies in the cone of u with apex at O_B. This can be done in constant time with $O(kn)$ processors by assigning a processor to each pair.

We perform a single pass of k selection to obtain for each B and u, the k nearest a within the cone of u. This can be done in $O(\log n)$ time with $O(kn)$ processors. We use $F(B)$ to denote the set of all a thus paired with B. By Lemmas 3.1.1 and 3.1.3, we know that for any k-nearest-neighbor pair (a, b) there is an ancestor B of b such that $a \in F(B)$.

Having guaranteed that every point is assigned to the same cone throughout the top down procedure, we can simplify our description of the set of points to

compute at the leaves of T. Specifically, for every point b in P, we wish to compute, for each $u \in U$, the k nearest points a such that $\{B, \{a\}\}$ is a pair associated with u for some ancestor B of b.

To solve this new version of the problem, we exploit the fact that smaller distances tend to occur deeper in the tree. The ordering is not exact, so we will not try to obtain the k nearest points immediately. Instead, we will obtain, for each leaf in T and vector u, a set of size $O(k)$ that contains as a subset the k nearest points within the cone of u that have not already been eliminated by Lemma 3.1.1. We require several geometric results.

Suppose $\{A_i, B_i\}$ is a pair in a realization constructed by the algorithm given in section 2.3, and let $d(A_i, B_i)$ denote the minimum distance between $R(A_i)$ and $R(B_i)$. Then, for some d-dependent constant c,

$$sl_{\max}(A_i) \leq d(A_i, B_i) \leq csl_{\max}(p(A_i)). \tag{5.4}$$

Clearly $d(A_i, B_i) \geq sl_{\max}(A_i)$, since otherwise the pair would not be well-separated. To prove the other half of (5.4), we need to consider several cases. A trivial case occurs when $p(A_i) = p(B_i)$. Otherwise, there are two ways in which our algorithm could have formed the pair $\{A_i, B_i\}$. If $\{A_i, B_i\}$ was formed by splitting $p(A_i)$, then $p(A_i)$ and B_i are not well-separated, so the inequality holds. Otherwise, $\{A_i, B_i\}$ was formed by splitting $p(B_i)$, so $d(A_i, B_i) \leq csl_{\max}(p(B_i))$. In this case, $p(A_i)$ was split at some earlier point. Hence, $l_{\max}(p(A_i)) \geq l_{\max}(p(B_i))$, and the inequality holds here as well.

We now consider an additional property of fair split trees. Let A be a node in some fair split tree T, and let $p^j(A)$ denote the ancestor of A that is j levels above it in the tree, if such an ancestor exists. Then we have the following inequality.

$$l_{\max}(A) \leq \frac{2}{3} l_{\max}(p^{6d}(A)) \tag{5.5}$$

To see this, note that after a fair split is performed 3 times perpendicular to dimension i, then l_i can be no more than $2/3$ its original value. That is, if the first two

splits did not reduce l_i sufficiently, then they must have each been close to opposite boundaries of $R(A)$ in the ith dimension. At this point, the third split must divide this length into two pieces whose size is no more than $2/3$ the original in order to insure the fair split condition. Moreover, once l_i has been reduced to $4/9$ its original size, a fair split cannot occur in the ith direction until $l_{\max}$ has been reduced to $2/3$ its original size. Hence, after $6d$ splits, $l_{\max}$ is no more than $2/3$ its original value. In practice, this is almost certainly an overestimate.

We say that a is *paired with* B when $a \in F(B)$. Let $S(B, u, b)$ denote the set of all a in the cone of u paired with b or one of its ancestors up to and including B. Also, let $B(u, b)$ denote the nearest ancestor of b such that $|S(B(u, b), u, b)| \geq k$.

Lemma 5.4.2: *There exists a constant c' dependent only on d such that for any leaf b in T and $u \in U$, $S(p^{c'}(B(u, b)), u, b)$ contains the k nearest a to b in the cone of u paired with some ancestor of b.*

Proof. let a' denote the kth nearest neighbor of b in the cone of u paired with some ancestor of b. For at least one $a'' \in S(B(u, b), u, b)$, $d(a'', b) \geq d(a', b)$, because $S(B(u, b), u, b)$ has at least k elements. It follows from the well-separated set condition that $d(a'', B) \geq \frac{s}{s+2} d(a', b)$, where B is the node paired with a''. Combining (5.4) and (5.5), we can determine some constant c' such that for all $a \in S(T, u, b) - S(p^{c'}(B), u, b)$, $d(a, b) \geq d(a', b)$. Hence, $S(p^{c'}(B), u, b)$ contains the k nearest a to b in the cone of u paired with some ancestor of b. B is either $B(u, b)$ or one of its descendants, so the original claim follows. □

We now need to retrieve $N(u, b) = S(p^{c'}(B(u, b)), u, b)$ for each u and b. This is similar to the problem of retrieving, for every leaf in a tree, its k nearest ancestors. In fact, it can be reduced to a weighted version of the latter problem by constructing a separate tree for each vector u, and compressing out all nodes B not paired with any a in the cone of u. This is no longer a geometric problem, and can be done $O(\log n)$ time with $O(kn)$ processors using standard parallel techniques.

Finally, for each leaf a in T, we know by previous arguments that its k

nearest neighbors are among those b such that $a \in N(u, b)$ for some $u \in U$. We can easily retrieve the k-nearest-neighbors among these points in $O(\log n)$ time with $O(kn)$ processors, thus completing our parallel algorithm.

Theorem 5.4.3: *Given a set P of n points in $\Re^d$ and a well-separated pair decomposition of P, the k-nearest neighbors of all points in P can be computed in $O(\log n)$ time with $O(kn)$ processors.* □

Combining this with Theorem 5.3.5, we obtain an $O(\log n)$ time $O(kn)$ processor deterministic parallel algorithm for the k-nearest neighbors problem.

5.4.3 Other applications

The application of the decomposition to the fast multipole method illustrates the most general sort of algorithm that can be parallelized directly using rake-and-compress. This algorithm consists of three phases: a bottom-up phase, a phase involving the pairs, and a top-down phase. The second phase tends to have an obvious parallel implementation, while the remaining phases can be parallelized using rake-and-compress. Our algorithms for computing an approximate EMST and an approximate complete Euclidean graph are composed of phases similar to those in potential computation, and therefore permit parallel implementations.

In the computation of the approximate EMST, the most involved phase performed on the well-separated pair decomposition is that used to refine the error bound (section 3.3.2). This involves the computation of points $a(u, S)$ and $b(u, S)$ for every node S in T, that minimize or maximize dot products with vectors from a set of $O(1)$ elements. In the sequential algorithm, these are computed bottom-up on the fair split tree T.

Because minimum and maximum are associative operations, this computation can be parallelized using rake-and-compress in a manner similar to that used in the parallel algorithm for potential computation. Taking into account this phase and

those used to construct the well-separated pair decomposition, the total time required is $O(\log n)$ using $O(n)$ processors on a CREW PRAM. The result is a sparse graph having $O(n)$ edges for which we must compute a minimum spanning tree. Using the algorithm of Kwan and Ruzzo [46], we can compute this in $O(\log^2 n)$ time using $O(n/\log n)$ processors on a CREW PRAM.

The computation of an approximate Euclidean graph (section 3.4) uses the same bottom-up phase as above. In addition, it involves a top-down phase to compute the $\hat{b}(u, S)$ labels for each node. Like the $a(u, S)$ labels, these functions are defined in terms of minimizing a vector dot product, in this case for all nodes in a path from the root to the node S. This phase can be parallelized in a manner similar to the parallelization of the top-down phase of the potential computation algorithm. The result is an algorithm that runs in $O(\log n)$ steps with $O(n)$ processors on a CREW PRAM.

While it is impossible to predict that all natural applications of the well-separated pair decomposition will lead to efficient parallel algorithms, these examples illustrate a powerful approach. Once a well-separated pair decomposition has been computed for some point set, many problems can be posed as structured computations on the fair split tree. Although this tree may be very deep, and the obvious form of level-by-level parallelization therefore too time consuming, the additional structure imposed on the problem makes it possible to apply the powerful techniques already developed for parallel tree-structured computation.

Chapter 6

Dynamic algorithms

In this chapter, we show how to make well-separated pair algorithms fully dynamic under insertions and deletions into P. To do this, we will present a method of maintaining the fair split tree of P along with an explicit representation of the set of well-separated pairs. Additionally, we will develop an abstract framework that captures the essential characteristics of algorithms that use the decomposition, and show how to maintain computations in such a framework. The results of this chapter were originally presented in [22].

A significant consequence of this technique will be its application to the closest-pair problem, in which we must maintain, under insertions and deletions into P, the pair of points that minimizes interpoint distance. We present the first fully dynamic deterministic algorithm for this problem for which the worst case update time is $O(\log^c n)$ for c independent of d. Our algorithm will require $O(\log^2 n)$ time for updates, and $O(n)$ space. This matches the best randomized algorithm for dynamic closest pair [35] and significantly improves upon the best deterministic algorithm, due to Kapoor and Smid [41], which gives an $O(\log^{d-1} n \log\log n)$ amortized bound using $O(n)$ space. It should be noted that the problem is considerably simpler when only insertions are allowed. An optimal data structure has been developed for the latter case in [63].

Our dynamic closest-pair algorithm will be based on a dynamization of the k-

nearest-neighbors algorithm of section 3.1. *Note that this will not result in a dynamic algorithm to perform nearest-neighbor queries, a problem which is conjectured to be impossible for* polylog *time and* $O(n \text{ polylog } n)$ *space.* This is because the all-nearest-neighbors algorithm is made dynamic only up to the step at which the attractors of each point x (those that have x as a nearest neighbor) are computed. We show how to maintain P to allow efficient queries that retrieve a constant-size superset of the attractors for any $x \in P$ and show how we can use such queries, along with an auxiliary heap, to maintain the closest pair.

We will also present a dynamization of the fast multipole method [37, 38], discussed in section 3.2. This will allow us to maintain a set of charges under insertions and deletions so that we can efficiently answer queries of the potential at a point x. The time for queries and updates will be $O(\log^2 n)$, with constants dependent on d and the numerical precision.

6.1 Maintaining the fair split tree

Suppose we have a fair split tree of a point set, and we wish to insert a new point. Because the tree can be quite deep, it is not even obvious how we can efficiently find the place to insert it. A more serious problem is how to insert the point while maintaining the fair split property. While a variety of general techniques have been developed that solve the former problem, the latter problem can only be treated by specifically considering the geometry of fair split trees. We will solve this problem by developing a more careful characterization of the tree we wish to maintain.

For the dynamic problem, it is useful to distinguish nodes in the tree from the subsets of points they represent, because insertions and deletions may cause the value of these subsets of points to change. We will use $\sigma(v)$ to denote the subset at v at some instance in the sequence of updates. As usual, if v has two children w_1 and w_2, then $\sigma(v) = \sigma(w_1) \cup \sigma(w_2)$. If v is a leaf, then $|\sigma(v)| = 1$.

Recall that the fair split property depends on the value of $l_{\max}(\sigma(v))$ (i.e.,

the longest dimension of the bounding rectangle of the set at node v). Each time we insert a new point, this may increase the value of $l_{\max}(\sigma(v))$ for all its ancestors in T. The fair split property may be violated at all of these nodes as a result. It is not clear how the tree can be efficiently restructured to restore this property. Instead, we will show how to guarantee that when we insert a new point, all previously existing splits remain fair.[4]

Our difficulties stem from our reliance on $l_{\max}(\sigma(v))$, a quantity that may vary in the course of updates. Suppose instead we explicitly associate the node v with a rectangle $\tilde{R}(v)$ that is not modified by updates, and maintain the invariant that all points in $\sigma(v)$ must lie within $\tilde{R}(v)$. Then we may safely replace $l_{\max}(\sigma(v))$ with $l_{\max}(\tilde{R}(v))$ in the fair split condition because $l_{\max}(\sigma(v)) \leq l_{\max}(\tilde{R}(v))$ at all times. This observation is the key to our approach.

We must be careful in our choice of $\tilde{R}(v)$, because certain rectangles consistent with the point set at one instance may interfere with the insertion of a point later on. Intuitively, we need to use rectangles that are consistent with a sequence of fair splits. We introduce the slightly modified notion of a *fair cut*, which is defined for any rectangle R without reference to a point set. A fair cut is simply a partition of R into two rectangles by means of an axis-parallel hyperplane intersecting R at a distance of least $\frac{1}{3}l_{\max}(R)$ from the nearest parallel side. To make this notion precise, we should regard the rectangles involved in a fair cut as the product of half-open intervals.

We introduce the notation $R \rightsquigarrow R'$ to indicate that R' can be constructed from R by applying a (possibly empty) sequence of fair cuts. We also define this to hold when $R' \subseteq R$ is a closed interval consisting of a single point, since it clearly holds for any sufficiently small half-open d-cube containing R'.

We wish to maintain a labeled binary tree T in which each node satisfies the following invariants:

[4] It is interesting to note that the deletion of a point will not increase the value of $l_{\max}(\sigma(v))$ for any of its ancestors, and hence can be performed on any fair split tree without restructuring.

i. For all internal nodes v with children w_1 and w_2, there is a fair cut that partitions $\tilde{R}(v)$ into rectangles R_1 and R_2, such that $\sigma(w_1) = \sigma(v) \cap R_1$, $\sigma(w_2) = \sigma(v) \cap R_2$, $R_1 \rightsquigarrow \tilde{R}(w_1)$, and $R_2 \rightsquigarrow \tilde{R}(w_2)$.

ii. For all leaves v, $\sigma(v) = \{p\}$, and $\tilde{R}(v) = p$.

Let $P = \sigma(v)$, where v is the root node. Then it is easy to verify that such a tree is a fair split tree of P.

To insert a point p into this structure, we first retrieve the deepest v in T such that $p \in \tilde{R}(v)$, temporarily ignoring the special case in which p lies outside the rectangle at the root node. There are no duplicate points, so v is an internal node. Let R_1 and w_1 have the same meaning as in the first invariant. Assume w.l.o.g. that $p \in R_1$. The way we chose v guarantees that $p \notin \tilde{R}(w_1)$. Now we introduce a new internal node u, which replaces w_1 as a child of v. We insert w_1 along with its subtree as a child of u, and insert a new leaf u' as the other child of u, where $\sigma(u') = \{p\}$. Finally, we construct a rectangle $\tilde{R}(u)$ satisfying the first invariant.

While the above description is straightforward at a conceptual level, it leaves several issues open. The most significant is whether a rectangle $\tilde{R}(u)$ always exists, and if so, how to find it. Existence follows from the fact that $R_1 \rightsquigarrow \tilde{R}(w_1)$. Consider a sequence of fair cuts from R_1 to $\tilde{R}(w_1)$. It is clearly sufficient to let $\tilde{R}(u)$ be the last rectangle in such a sequence that satisfies $p \in \tilde{R}(u)$. Unfortunately, this does not lead immediately to an efficient way of finding $\tilde{R}(u)$, because such a sequence may be very long. We can, however, bound its length in terms of the sizes of the rectangles involved.

Observation 6.1.1: *If $R \rightsquigarrow R'$, then the number of fair cuts in any sequence from R to R' is* $O(\log(l_{\max}(R)/l_{\max}(R')))$.

This follows from the fact that a fair cut reduces the sum of the dimensions of a rectangle by a factor of at least $d/(d - \frac{1}{3})$.

To find $\tilde{R}(u)$ efficiently, we will divide the problem into two cases, depending on $l_{\max}(\tilde{R}(u))$, which we will be able to estimate in constant time. By Observation 6.1.1, if $l_{\max}(\tilde{R}(u))$ is within a constant of $l_{\max}(\tilde{R}(v))$, then $O(d)$ splits will be sufficient to construct $\tilde{R}(u)$, and we can afford to solve the problem by brute force. Otherwise, $\tilde{R}(u)$ will be quite small in comparison to $\tilde{R}(v)$ and we will have a great deal of flexibility in choosing it. This will allow us to construct such a rectangle in constant time. To apply this technique, we derive the following result, establishing sufficient conditions for R and R' to satisfy $R \rightsquigarrow R'$. Note that these conditions are similar to the requirement in the box decomposition of Arya et al. [9, 10] that boxes be "sticky."

Lemma 6.1.2: *Let R and R' be two rectangles such that $R' \subseteq R$, and $l_{\min}(R') \geq \frac{1}{3} l_{\max}(R')$. Suppose that for all $i = 1, \ldots, d$, the distance from each of the sides of R' normal to the ith dimension to the nearest parallel side of R is either exactly 0 or else greater than or equal to $l_i(R')$. Then, $R \rightsquigarrow R'$.*

Proof. Let i be chosen to maximize $l_i(R)$ over all dimensions such that $l_i(R) > l_i(R')$. If no such i exists, then it must be the case that $R = R'$, and we may conclude trivially that $R \rightsquigarrow R'$. Otherwise, we can perform a fair cut in the ith dimension, as we will show, producing a rectangle that contains R' and preserves the other assumptions of the lemma. First, if it is possible to perform a fair cut in the ith dimension with a hyperplane intersecting one of the boundaries of R', then we do so. Note that if $l_i(R) < l_{\max}(R)$, then $l_{\max}(R) = l_{\max}(R')$, so the assumption that $l_{\min}(R') \geq \frac{1}{3} l_{\max}(R')$ implies that such a cut is possible, because, by the assumptions of the lemma, one of the boundaries of R' lies at distance at least $l_i(R') \geq l_{\min}(R')$ from the nearest side of R. Otherwise, $l_i(R) = l_{\max}(R)$, and if no fair cut could be found intersecting the boundary of R', then $l_i(R') < \frac{1}{3} l_{\max}(R)$. In this case, we perform a fair cut with a hyperplane at distance $\frac{1}{3} l_{\max}(R)$ from the boundary of R farthest from R'. The distance of this hyperplane must be at least $\frac{1}{3} l_{\max}(R)$ from the nearest side of R', or else the preceding case would have applied. This distance is an upper bound on $l_i(R')$, so the assumptions of the lemma are preserved.

Clearly, any sequence of fair cuts applied to R and preserving $R' \subseteq R$ must eventually terminate. If we apply the above rules, the sequence of cuts can only terminate by satisfying $R = R'$. Hence, it follows that $R \rightsquigarrow R'$. $\square$

We will refer to the process of the above proof as *cutting towards* R'. This will be an important technique in our dynamic algorithm, since it allows us to construct a sequence of rectangles $R^1, \ldots, R^k$ all satisfying $R^i \rightsquigarrow R'$. In fact, as long as $R \rightsquigarrow R'$, we will be able to cut towards R', even if R' does not satisfy the stronger assumptions of the lemma. We state this as follows:

Lemma 6.1.3: *If $R \rightsquigarrow R'$, then the process of cutting towards R', starting with R, will eventually produce R'.*

Proof. Let $R^1, \ldots, R^k$ denote the sequence of intermediate rectangles. We begin by observing that if a boundary of R' in dimension i lies at distance less than $\max\{l_{\max}(R')/3, l_i(R^j)/3\}$ from the nearest side of R^j, then it must also lie at this distance from the corresponding side of R. That is, no cut we made could have led to this situation, unless it existed initially. One can easily verify that if the process of cutting towards R' does not succeed (i.e., does not produce R') then there must be some R^j for which the above condition holds for one of the boundaries of R'. If this is so, then it would have to hold for all sequences of fair cuts that result in this difficulty. Hence, no sequence of fair cuts can succeed, and it follows that $R \not\rightsquigarrow R'$. $\square$

Let u, p, R_1, and w_1 have the same meaning as in the high-level description of insertion. To determine $\tilde{R}(u)$, we begin by constructing the bounding rectangle $R(\tilde{R}(w_1) \cup \{p\})$. For the following discussion, let $l = l_{\max}(R(\tilde{R}(w_1) \cup \{p\}))$.

First, if $l \geq \frac{1}{3} l_{\max}(R_1)$, then we begin cutting towards $\tilde{R}(w_1)$, starting with R_1. We finish cutting when we obtain a rectangle that does not contain p. We use its predecessor in the sequence as $\tilde{R}(u)$. By Observation 6.1.1, this procedure terminates in $O(\log(l_{\max}(R_1)/l))$ steps, which is constant.

Otherwise, we construct a d-cube C contained in R_1 and containing $\tilde{R}(w_1)$, such that $l(C) = 3l$, and any side of C not touching $\tilde{R}(w_1)$ is at a distance of at least

l from it. We form the rectangle R'' by considering every side of C at a distance less than $3l$ from the nearest side of R_1, and extending it so that it intersects this side. Clearly, $l_{\min}(R'') \geq \frac{1}{3} l_{\max}(R'')$, and it follows from Lemma 6.1.2 that $R_1 \rightsquigarrow R''$ and $R'' \rightsquigarrow \tilde{R}(w_1)$.

Starting with R'', we begin cutting towards $\tilde{R}(w_1)$. As before, we finish cutting when we obtain a rectangle that does not contain p, and use its predecessor in the sequence as $\tilde{R}(u)$. Because $R_1 \rightsquigarrow R''$, it follows by transitivity that $R_1 \rightsquigarrow \tilde{R}(u)$, thus proving correctness. By Observation 6.1.1, this procedure terminates in $O(\log(l_{\max}(R'')/l))$ steps, which is constant.

For completeness, we need to consider the case in which p lies outside the rectangle labeling the root. This is treated in the same manner as other insertions, except that the new internal node u becomes the root, and w_1 in the preceding discussion corresponds to the old root. The "rectangle" R_1 can be thought of as all of Euclidean d-space, so the second case is used in determining $R(u)$.

The above analysis shows that once we have determined where to insert a point p, we may perform such an insertion in constant time, while preserving the invariants of the tree. We still need to consider the problem of finding the node v at which we insert u and u'. We will show in the next section how such a search can be performed in $O(\log |P|)$ time, where P is the current point set.

To make the algorithm fully dynamic, we need to be able to handle deletions. To delete the point p, we simply find the leaf v such that $\sigma(v) = \{p\}$, delete v, and compress out the internal node $p(v)$. Clearly, this will preserve our invariants.

6.2 Point location

In the preceding section, we showed how to perform an insertion in constant time once we have solved the following point location problem: Given a point p, what is the lowest node v in T such that $p \in \tilde{R}(v)$? Using the directed topology tree of Frederickson [32], we can maintain a dynamic structure allowing us to solve this

problem in $O(\log m)$ steps, where m is the current size of the point set.

Recall Frederickson's directed topology tree, as described in section 4.2 of this dissertation. Using this technique, Frederickson [32] has shown how to maintain a collection of rooted trees under various restructuring operations to support certain queries such as arithmetic expression evaluation in efficient worst-case time complexity. The space requirement is linear in the total size of the forest being maintained.

The restructuring operations we will require are simple leaf insertions and deletions. In an insertion, an edge is replaced by a chain of two edges, and the new leaf is added as the second child of the middle node of the chain. In a deletion, these steps are performed in reverse. It is easy to see how to reduce these steps to a sequence of three *swap* operations as defined in [32], though in practice it is more efficient to implement the operations separately. In either case, the update complexity is $O(\log m)$, where m is the size of the tree.

To complete the description of insertion we need only show how to answer the point location query in $O(\log m)$ steps. To solve this problem, we perform a search that considers each of the $O(\log m)$ levels of the restricted multi-level partition in sequence, beginning at the root. A similar search procedure, using a contracted fair split tree, has been given in [10]. For completeness, we describe the search within our framework.

Recalling the notation of section 4.2, let c denote a cluster at some level in the partition. It is clear that if c is degree-1 then it corresponds to a subtree of some node $v(c)$ in T, and if it is degree-2, it corresponds to a subtree of some node $v(c)$ from which the subtree of its descendant $w(c)$ has been removed. A degree-3 cluster corresponds to a single internal node $v(c)$. For the following, we assume that all clusters c are labeled with $v(c)$, and degree-2 clusters are also labeled with $w(c)$.

Let the region corresponding to c be $\tilde{R}(v(c))$ if it is degree-1, and $\tilde{R}(v(c)) - \tilde{R}(w(c))$ if it is degree-2. If it is degree-3, let the region be $\tilde{R}(v(c)) - (\tilde{R}(w_1) \cup \tilde{R}(w_2))$, where w_1 and w_2 are the children of $v(c)$ in T. At all levels of the search, we maintain the invariant that p is in the region corresponding to c. Clearly, the invariant holds

at the root cluster as long as p lies inside the rectangle at the root of T. It is also easy to verify that as we move to a lower level, c may be split into at most two smaller clusters, and exactly one of them satisfies the invariant. Therefore, the procedure requires $O(\log m)$ steps.

At the end of this procedure, assuming p is not identical to one of the points in the current point set, we are left with a degree-3 cluster c such that $p \in \tilde{R}(v(c))$. In addition, the search invariant will insure that $p \notin \tilde{R}(w_1)$ and $p \notin \tilde{R}(w_2)$, where w_1 and w_2 are the children of $v(c)$. As a result, $v(c)$ may be returned as the result of our point location query.

Summarizing the results of this section and the preceding, we have the following theorem.

Theorem 6.2.1: *The fair split tree of a point set P can be maintained in linear space at a worst case cost of $O(\log |P|)$ time per insertion or deletion.* □

6.3 Maintaining the pairs

It would be useful if the number of pairs involving any node v were bounded by a constant, but this is not the case in general. In fact, there may exist a node v such that there are linearly many pairs of the form $\{v, w\}$. This poses a special problem for the dynamic algorithm, because the introduction of a single new point can require the creation of many new pairs. We can get around this problem if we are willing to introduce some extra nodes in the tree, as we will show.

The idea is to predict all but a constant number of the new pairs ahead of time. To see that this is possible, we use the notion of distance classes, introduced in section 4.6. Recall that $DC_r = [1 - \epsilon, 1 + \epsilon)r$ for some constant $\epsilon > 0$. We make the following observation.

Observation 6.3.1: *Let P be a point set, let x be any point in $\Re^d$, and let y be the nearest neighbor of x in P. Then, for any $z \in P$, if $d(x, z) > \epsilon^{-1} d(x, y)$, then*

$d(x,z) \in \mathrm{DC}_{\frac{1}{1-\epsilon^2}d(y,z)}$.

This can be derived from the triangle inequality by noting that $d(y,z) \in [1-\epsilon, 1+\epsilon)d(x,z)$. Intuitively, this observation implies that the only new distance classes created by the introduction of a new point x are within some constant factor of the distance between x and its nearest neighbor. According to Lemma 4.6.1, there are $O(1)$ well-separated pairs in these classes, and we can enumerate them all, once we have computed an approximation of $d(x,y)$.

The pairs we maintain will be the pairs of the canonical realization, with the definition of $\prec$ and "well-separated" modified to use the rectangle $\tilde{R}$ rather than the bounding rectangle of a node in T. To guarantee that all but a constant number of new pairs exist before the insertion of a new point x, we maintain the following invariant, which will require the introduction of some extra nodes in the tree.

Invariant 6.3.2: *For all $y, z \in P$, and all $x \in \Re^d$ such that $d(y,x) \in \mathrm{DC}_{\frac{1}{1-\epsilon^2}d(y,z)}$, there is a well-separated pair $\{v,w\}$ such that $y \in \sigma(v)$ and $x \in \tilde{R}(w)$.*

6.3.1 Dummy points

Intuitively, one can see that Invariant 6.3.2 would hold automatically if space were densely packed, but it will not hold for all distributions of points. A way to obtain the effect of densely packed space, is to introduce *dummy points* where appropriate. The advantage to this approach is that we can analyze such operations in terms of the insertion and deletion operations already defined.

Let $\hat{P}$ denote the set of dummy points. Such points will not be counted in $\sigma(v)$ for any v in T, but the tree T will have the same structure and rectangle labels as a fair split tree of $P \cup \hat{P}$. For efficiency, we must introduce only a constant number of dummy points for each well-separated pair $\{v,w\}$ such that $\sigma(v)$ and $\sigma(w)$ are non-empty. We know that the number of such pairs is linear.

We use Invariant 6.3.2 to determine the location of the dummy points. Suppose the invariant does not hold for some $y \in \sigma(v)$ and $z \in \sigma(w)$. Then there must

be $x \in \Re^d$ such that $d(y,x) \in DC_{\frac{1}{1-\epsilon^2}d(y,z)}$, and for which there does not exist a pair $\{\hat{v},\hat{w}\}$ such that $y \in \sigma(\hat{v})$ and $x \in \tilde{R}(\hat{w})$. In this case, we add x to $\hat{P}$, and let $\{\hat{v},\hat{w}\}$ be the pair in the canonical realization such that $y \in \sigma(\hat{v})$, and x is a leaf in the subtree of $\hat{w}$. We continue to add dummy points in this matter until the invariant holds for all $y \in \sigma(v)$ and all $z \in \sigma(w)$.

Lemma 6.3.3: *The number of dummy points added for any pair is* $O(1)$.

Proof. Using Lemma 4.6.1, it is easy to see that only constantly many new pairs are created. Each dummy point added creates a new pair, so, there are constantly many dummy points. □

There are several ways to compute the dummy points explicitly. The procedure given above could be implemented by computing the set of rectangles covering the vicinity of $\tilde{R}(v)$ using an $O(\log|T|)$ search similar to that used in section 5.2. The point x could then be selected arbitrarily from any space left uncovered. For a more explicit construction, let $\alpha,\beta > 0$ be constants, and construct a mesh of points with spacing $\alpha d(y,z)$, covering a sphere of radius $\beta d(y,z)$ centered at y. It is easy to verify that for sufficiently small α and sufficiently large β, this construction will preserve the invariant.

6.3.2 Point insertions and deletions

Finally, we consider the actions to perform when a point is inserted or deleted. Our goal will be to maintain an explicit representation of a set of well-separated pairs that includes a pair covering each $\{x,y\}$ such that $x,y \in P$, but which may include additional pairs required to preserve Invariant 6.3.2. The reason for computing this set explicitly will become clear when we consider the problem of maintaining the result of computations on the well-separated pair decomposition.

When a point x is inserted, we first need to approximate the distance to its nearest neighbor in P. This can be done in $O(\log|T|)$ steps using a search procedure

such as that given in [10], modified to use the directed topology tree data structure in place of the compressed tree. By Invariant 6.3.2, all pairs created by the insertion will be of approximately this distance and will include x. Hence, we can compute them all in constant time.

In addition, we need to add dummy points. In computing the dummy points, we consider only the new pairs, because by Observation 6.3.1 any previously existing pairs must lie at approximate distances around x that were considered at an earlier step. There are $O(1)$ dummy points to be inserted, so the total time is $O(\log |T|)$.

For the deletion of x, as with insertion, we need only consider those pairs whose distance is roughly the distance between x and its nearest neighbor, because it follows from Observation 6.3.1 that in order to maintain Invariant 6.3.2, all other pairs must continue to exist after the deletion of x. Note that for efficiency, it is important to delete all dummy points dependent on the pairs that are destroyed by the deletion of x, so that $|T| = O(|P|)$ at all times.

From the results of this section, we obtain the following theorem.

Theorem 6.3.4: *The set of canonical pairs of a point set P, along with $O(|P|)$ additional pairs, can be maintained in $O(|P|)$ space at a worst case cost of $O(\log |P|)$ time per insertion or deletion.* □

6.4 Abstract tree-pair computations

Many of the algorithms that use the well-separated pair decomposition can be formalized in terms of an abstract computation on a tree in which the nodes are paired in a particular way. In this section, we formalize this notion of tree-pair computations, and show how to maintain such a computation under arbitrary changes to leaf values.

We formalize a tree-pair computation as follows: We assume we have two binary operators, one ($+$) for bottom-up computation and another ($\oplus$) for top-down computation. Both operators are associative and have an identity element, and we assume we can evaluate each in constant time.

In addition to the tree, there are ordered *cross-pairs* (v, w), and one of our primitive operations will consist of *redirecting* a pair (v, w), in which we replace it with (w, v), and update the structure in a consistent fashion.

For the bottom-up computation, we label each node v in T with a value $BU(v)$, which is an element from the domain of the associative operator $+$. For every leaf v, $BU(v)$ is given as a base value. For every internal node v with children w_1 and w_2, $BU(v) = BU(w_1)+BU(w_2)$.

We assume every node v has a corresponding function F_v that maps sets of elements from the domain of $+$ to elements from the domain of $\oplus$, and we let W_v denote the set of nodes w such that (w, v) is a cross-pair. We assume that we can maintain F_v efficiently under insertions and deletions to W_v.

We wish to label each v with a top-down value $TD(v)$ in the domain of $\oplus$. For the root, we let $TD(v)$ be the identity. For all other nodes, we let $TD(v)$ be $TD(p(v))\oplus F_v(\{BU(w)|w \in W_v\})$.

It is easy to see how to compute the preceding labelings for all nodes v in two sequential passes. For a dynamic algorithm, however, it is clear that the modification of a single leaf x may require an additional update for each pair of the form (v, w) such that v is an ancestor of x. The definitions above give us no useful bound on the number of such pairs, but we will present such a bound for fair split trees in the next section. In this section, we consider only the abstract problems of redirecting a pair, and performing leaf queries and updates.

Recall the directed topology tree of Frederickson [32], as used in section 6.2. Suppose we maintain a directed topology tree of T, At any level, we must associate partial computations of BU and TD labels with the clusters of T. The BU labels can be maintained precisely according to the method of Frederickson because the bottom-up computation is simply an expression tree evaluation. For the TD labels, we must associate an additional value with each degree-2 cluster.

Note that each degree-2 cluster corresponds to a path in T determined as follows. In the case that this cluster has been formed by merging a degree-3 cluster

with a degree-1 cluster, the path consists of the vertex v corresponding to the degree-3 cluster. Otherwise, the cluster was formed by merging two degree-2 clusters, and its path consists of the concatenation of their paths. In this case, cluster c corresponds to the path from $v(c)$ to $p(w(c))$. For any degree-2 cluster the associated value $TD'(c)$ is the $\oplus$-sum of $F_{v'}(\{BU(w)|w \in W_{v'}\})$ for all v' in this path.

When maintaining such values, we will assume that $F_{v(c)}(\{BU(w)|w \in W_{v(c)}\})$ has been evaluated for any degree-2 cluster $v(c)$ whose path is the single node $v(c)$. When merging two clusters c_1 and c_2 to form c, we let $TD'(c) = TD'(c_1) \oplus TD'(c_2)$.

We now consider the problem of redirecting a pair. We assume that the value of $F_v(\{BU(w)|w \in W_v\})$ can be maintained in $O(\log |W_v|)$ time per insertion or deletion of a member of W_v. This assumption will hold in all our applications. The first step of redirecting the pair (v, w) is to delete v from W_w, and update the computation of F_w accordingly. In the worst case, the time for this operation is $O(\log |T|)$. Next, we must update the dynamic computation of TD, which also requires $O(\log |T|)$ time using the directed topology tree. We then insert w into W_v, making appropriate updates to the computation of F_v, and again update the dynamic computation of TD. Now the tree-pair computation is consistent with the redirected pair (w, v). This gives us the following lemma.

Lemma 6.4.1: *A cross-pair (v, w) can be redirected in $O(\log |T|)$ time.* □

Using the operation of redirecting cross-pairs, we can easily describe the update operation in which we modify a leaf value. First, note that the BU labels can be maintained in $O(\log |T|)$ time per update using the method of Frederickson, so the only problem is how to maintain the TD labels. Modifying a leaf x can affect TD labels only through pairs (v, w) such that v is an ancestor of x. Let $out(x)$ denote the number of such pairs. We can reduce the problem of leaf updates to the problem of redirecting pairs. To do this, we redirect all (v, w) such that v is an ancestor of x. Then the modification of x will not have any effect on TD labels, and we can make the

structure consistent in $O(\log |T|)$ time by updating only BU labels. By redirecting the pairs a second time, we restore the tree-pair computation to its original structure, but with values that are now consistent with the new value of x. We assume that all such (v, w) can be retrieved in $O(out(x) \log |T|)$ steps; the method will be given in the next section. Summing the total time to redirect all pairs, we obtain the following lemma.

Lemma 6.4.2: *A leaf x can be updated in $O(out(x) \log |T|)$ time.* □

Using the method of Frederickson, we can query $TD(x)$ for any leaf in $O(\log |T|)$ time. However, for reasons that will be made clear in the next section, we will typically wish to redirect all pairs directed out of an ancestor of x before performing such a query. In order to develop an efficient algorithm using the well-separated pair decomposition, we need to bound $out(x)$. We will show in the next section how to maintain the pairs of a well-separated pair decomposition so that $out(x) = O(\log |T|)$ in the worst case.

6.5 Directing the pairs

In this section, we present a way of directing the pairs in a well-separated pair decomposition to guarantee that $out(x) = O(\log |T|)$.

For technical reasons, it will be useful to associate weights with the leaves of T that affect the direction of the pairs. These weights will be fixed at unity for most points, but will sometimes be modified in order to redirect the pairs along a given leaf-to-root path. This reweighting operation will be useful for queries and updates. For each leaf v, the weight $w(v)$ is an integer in the range 0 to $|T|$, and for each internal node v with children u_1 and u_2, $w(v) = w(u_1) + w(u_2)$.

Recall the definition of a δ-rendering of a d-cube C, denoted $\rho_\delta(C)$, as given in section 5.2. Roughly speaking, this is a covering of C by nodes in T that corresponds approximately to the set of all points contained in C. We redefine $\rho_\delta(C)$ in

terms of the $\tilde{R}$ labels to consist of those nodes v such that $l_{\max}(\tilde{R}(v)) \leq \delta l(C) < l_{\max}(\tilde{R}(p(v)))$, and $\tilde{R}(v)$ overlaps C. It is straightforward to show by a packing argument that the size of a δ-rendering is O(1) (with constants dependent on δ and d). The weight $w(\rho_\delta(C))$ of a δ-rendering is defined to be $\sum_{v \in \rho_\delta(C)} w(v)$. The value of δ will follow from the analysis.

Let $\{v, w\}$ be the well-separated pair we wish to direct, and let $r = d(\tilde{R}(v), \tilde{R}(w))$. Let $C(v, w)$ denote the d-cube centered at the midpoint between the centers of $\tilde{R}(v)$ and $\tilde{R}(w)$ that satisfies $l(C(v, w)) = 2r$, and let $R_{\text{dir}}(v, w)$ be $\tilde{R}(u)$, where u is the node in $\rho_\delta(C(v, w))$ that maximizes $w(u)$. The pair $\{v, w\}$ can directed from v to w (with the directed pair denoted (v, w)) so long as

$$d(\tilde{R}(v), R_{\text{dir}}(v, w)) > \frac{1}{3} r. \tag{6.1}$$

Note that there may be some pairs that can be directed either way, but we must introduce exactly one directed pair for each undirected pair. It is important to guarantee that every pair can be directed in at least one way, and we prove this below.

Lemma 6.5.1: *All undirected pairs $\{v, w\}$ must satisfy either $d(\tilde{R}(v), R_{\text{dir}}(v, w)) \geq (\frac{1}{2} - \sqrt{d}\delta)r$ or $d(\tilde{R}(w), R_{\text{dir}}(v, w)) \geq (\frac{1}{2} - \sqrt{d}\delta)r$.*

Proof. Suppose otherwise. One can easily see that $R_{\text{dir}}(v, w)$ lies entirely within a sphere of diameter $2\sqrt{d}\delta r$. Thus, the path from $\tilde{R}(v)$ to $\tilde{R}(w)$ through $R_{\text{dir}}(v, w)$ would have length less than r, which is a contradiction. □

Note that for sufficiently small δ, this satisfies a somewhat stronger condition than that needed to direct the pair $\{v, w\}$. We will say that a directed pair (v, w) is *strongly directed* if it satisfies the stronger condition. We immediately have the following corollary.

Corollary 6.5.2: *All undirected pairs can be strongly directed.* □

In our algorithm, we will strongly direct any pair considered in an update. Pairs can then be allowed to degrade over time, as long as they do not violate the weaker condition (6.1).

The purpose of directing the pairs in this manner is to guarantee that there are not too many outgoing pairs along any path in the tree. To prove this, we will exploit the following property, expressed in terms of a constant $\alpha = 3\sqrt{d} + 4$.

Lemma 6.5.3: *For some node u, let (v, w) be a directed pair, such that v is an ancestor of u, and let (v', w') be a directed pair such that v' is also an ancestor of u (v' may be equal to v), and $d(\tilde{R}(v'), \tilde{R}(w')) > \alpha d(\tilde{R}(v), \tilde{R}(w))$. Then, for sufficiently large separation s, there is a constant κ dependent only on δ and d such that $w(\rho_\delta(C(v', w'))) \geq \frac{\kappa}{\kappa-1} w(\rho_\delta(C(v, w)))$.*

Proof. Consider the rectangle $R_{\text{dir}}(v', w')$. Because it is the rectangle of the node with maximum weight, the points lying inside $R_{\text{dir}}(v', w')$ must contribute at least $\frac{1}{\kappa}$ of the total value of $w(\rho_\delta(C(v', w')))$, where κ is an upper bound on the size of a δ-rendering. But, as we will show, the distance bound enforced for $R_{\text{dir}}(v', w')$ guarantees that these points do not contribute to $w(\rho_\delta(C(v, w)))$.

To see this, let x denote the center of $C(v, w)$, and let $r = d(\tilde{R}(v), \tilde{R}(w))$. Then any point $y \in C(v, w)$ must satisfy $d(x, y) \leq \sqrt{d}r$. We derive a lower bound on $d(x, R_{\text{dir}}(v', w'))$ by considering a path that goes from x to the nearest point in $\tilde{R}(v) \cap \tilde{R}(v')$, and from there to $R_{\text{dir}}(v', w')$. The length of the first segment of this path is $(\frac{1}{2} + \mathrm{O}(s^{-1}))r$. Moreover, the length of the second segment is at least $d(\tilde{R}(v), R_{\text{dir}}(v', w'))$, which by (6.1) is at least $\frac{1}{3}\alpha r$. So, the triangle inequality insures that for sufficiently large s,

$$d(x, R_{\text{dir}}(v', w')) \geq (\frac{1}{3}\alpha - 1)r.$$

Substituting $3\sqrt{d}+4$ for α, we see that $R_{\text{dir}}(v', w')$ is too far from x to overlap $C(v, w)$, and thus the points in $R_{\text{dir}}(v', w')$ do not contribute to $w(\rho_\delta(C(v, w)))$.

One can easily verify that the distance from the center of $C(v', w')$ to x is bounded above by $(\frac{\alpha+1}{2} + \mathrm{O}(s^{-1}))r$. So, for sufficiently large s, x lies inside $C(v', w')$ at a distance of at least r from the nearest boundary. It follows that $C(v, w) \subseteq C(v', w')$, insuring that any point that contributes to $w(\rho_\delta(C(v, w)))$ must also contribute to

$w(\rho_\delta(C(v',w')))$. Therefore, $w(\rho_\delta(C(v,w)))$ must be smaller than $w(\rho_\delta(C(v',w')))$ by a factor of at least $1-\frac{1}{\kappa}$, which suffices to prove the lemma. □

We use this result to prove the following bound:

Lemma 6.5.4: *For any u in T, there are, for sufficiently large separation s, $O(\log|T|)$ directed pairs of the form (v,w), where v is an ancestor of u.*

Proof. The idea is to show that the pairs (v,w) belong to at most $O(\log|T|)$ distinct distance classes. Then Lemma 4.6.1 implies that there are $O(\log|T|)$ such pairs total.

Let $\epsilon=\frac{\alpha-1}{\alpha+1}$, and construct k disjoint distance classes $\mathrm{DC}_{r_1},\ldots,\mathrm{DC}_{r_k}$ ($r_1<r_2<\ldots<r_k$) in terms of ϵ such that for every $i=1,\ldots,k$, there is some (v,w) such that v is an ancestor of u and $\tilde{d}(v,w)\in\mathrm{DC}_{r_i}$. Note that we adapt the definition of $\tilde{d}$ in section 4.6 to our present purposes, and define $\tilde{d}(v,w)$ to be the distance between the centers of $\tilde{R}(v)$ and $\tilde{R}(w)$. Clearly, $\tilde{d}(v,w)=(1+O(s^{-1}))d(\tilde{R}(v),\tilde{R}(w))$. Because the distance classes are disjoint and increasing, it follows that any $r\in\mathrm{DC}_{r_i}$ is greater than any $r'\in\mathrm{DC}_{r_{i-2}}$ by a factor of at least $\frac{1+\epsilon}{1-\epsilon}\tilde{d}(v,w)=\alpha\tilde{d}(v,w)$. Thus, by applying induction on the bound from Lemma 6.5.3, we can show that $w(\rho_\delta(C(v,w)))$ for any (v,w) such that $\tilde{d}(v,w)\in\mathrm{DC}_{r_k}$ is at least $(\frac{\kappa}{\kappa-1})^{\lfloor\frac{k}{2}\rfloor}$. Because the weight of a leaf is at most $|T|$, it follows that $(\frac{\kappa}{\kappa-1})^{\lfloor\frac{k}{2}\rfloor}\leq|T|^2$. Hence, k, the number of distance classes, is $O(\log|T|)$, and the claim follows from Lemma 4.6.1. □

Recall that we can compute the δ-rendering of any C in $O(\log|T|)$ time. This fact was already used in the procedure of section 5.2 to compute well-separated pairs in parallel. Assuming we maintain the weights of internal nodes using the method of Frederickson, we obtain the following result.

Lemma 6.5.5: *A pair $\{v,w\}$ can be strongly directed in $O(\log|T|)$ time.* □

We now consider the problem of determining which pairs change direction when the weight of a leaf is modified. In particular, we will show that the number of such pairs is $O(\log|T|)$.

Suppose we wish to update the weight $w(x)$ of a leaf x. Our method will be to enumerate a relatively small set of pairs that includes all of those whose direction must change when the value of $w(x)$ is modified. Once these have been found, we need only update $w(x)$ and recompute the direction of each of these pairs, which by Lemma 6.5.5 requires $O(\log |T|)$ time per pair.

We consider the way in which the pair directions vary under updates. Clearly, a change in the weight of a leaf, whether an increase or a decrease, can only affect the direction of a pair (v, w) by changing the value of the rectangle $R_{\text{dir}}(v, w)$ in $\rho_\delta(C(v, w))$. We cannot afford to track all such changes at every update, as there may be linearly many. Fortunately, this is not necessary because a newly directed pair satisfies a somewhat stronger condition than (6.1). Thus, it is only necessary to guarantee that any sequence of weight updates that causes the position of $R_{\text{dir}}(v, w)$ to move by as much as $(\frac{1}{6} - \delta\sqrt{d})d(\tilde{R}(v), \tilde{R}(w))$ cannot occur without (v, w) being redirected.

As we will later see, it will be relatively easy to detect the case in which a single update causes the position of $R_{\text{dir}}(v, w)$ to jump by a distance within a constant of $(\frac{1}{6} - \delta\sqrt{d})d(\tilde{R}(v), \tilde{R}(w))$. A more troublesome case occurs when the position shifts gradually over the course of many updates. To monitor such shifts, we will develop an auxiliary structure that alerts our main algorithm to the cumulative effect of gradual changes in position.

6.5.1 A data structure for detecting movement

We consider the problem of detecting cumulative movement. We will solve this independently of the well-separated pair decomposition, though we will need some previous results on maintaining fair split trees. We first treat the problem for one-dimensional point sets and then show how, using the L_∞ approximation to Euclidean distance, we can apply our one-dimensional data structures to the more general problem.

The problem is to maintain a point set P under insertions and deletions to allow us to process a sequence of queries, each of which is called a *movement event*. For the time being, we restrict our attention to points in one-dimension. We treat these interchangeably with real numbers, and denote the distance between points x and y by the equivalent expression $|x - y|$, or $y - x$ if it is known that $x \leq y$.

A movement event is given as an interval $[x, y]$, and for each event, we must output a real number Δ. The sequence of output is considered correct if it satisfies the following: Given a sequence of movement events, consider any subsequence $[x_1, y_1], \ldots, [x_m, y_m]$, which need not be contiguous. We denote the output at each such event by Δ_i. Suppose the following holds, for some $\eta > 0$:

i. $\bigcup_{i=1}^{m} [x_i, y_i] = [a, b]$

ii. For all $i = 1, \ldots, m$, $y_i - x_i < \eta(b - a)$.

iii. For all $i = 1, \ldots, m$, there exists some $z \in [x_i, y_i]$ such that z lies in the point set being maintained at all times between events $[x_1, y_1]$ and $[x_m, y_m]$.

Then there must exist some j between 1 and m such that $\frac{1}{7}(b - a) \leq \Delta_j \leq b - a$.

Intuitively, the above states that if a sequence of many small movement events spans a much larger interval, then one of these events must return a length approximately that of the larger interval. If each event is thought of as a shift of some position from one endpoint of the event interval to the other, then any such sequence with a large cumulative effect will be detected at some point in the course of updates. It now remains to show how to compute the values to be output.

Our algorithm will be relatively simple, although its correctness will require some analysis. To compute Δ for each movement event, we maintain a one-dimensional fair split tree T on the point set P. Then, each time we are given a movement event $[x, y]$, we find the deepest node v in T such that $[x, y] \subseteq \tilde{R}(v)$ and return $l(\tilde{R}(v))$ (we omit the subscript "max" because $\tilde{R}(v)$ has only one dimension). We know that insertions and deletions can be performed in $O(\log |P|)$ steps. It is also

easy to see that movement events can be handled in the same time complexity using a generalization of point location.

Correctness will follow from the lemmas below. The forest F^* denotes the subforest that is common to all fair split trees during the time between movement events $[x_1, y_1]$ and $[x_m, y_m]$.

Lemma 6.5.6: *Consider any subsequence $[x_1, y_1], \ldots, [x_m, y_m]$ of events satisfying the conditions stated above. For all v in F^* such that $\tilde{R}(v) \subseteq [a, b]$ and $\tilde{R}(v) > 3\eta(b - a)$, there exists some j such that v is the deepest node satisfying $[x_j, y_j] \subseteq l(\tilde{R}(v))$.*

Proof. Consider any v satisfying the assumptions of the lemma. There is some point z in $\tilde{R}(v)$ at which a fair cut is applied to $\tilde{R}(v)$ to produce the children of v. Let j be chosen so that x_j and y_j lie on opposite sides of z. Such a j must exist, because otherwise, $z \notin \bigcup_{i=1}^{m} [x_i, y_i]$ and the first condition on movement events would be violated. It follows from the fair cut condition and the size bound on $\tilde{R}(v)$ that z lies at a distance of at least $\eta(b - a)$ from the nearest boundary of $\tilde{R}(v)$. Thus, by the second condition on movement events, $[x_j, y_j]$ lies inside $\tilde{R}(v)$. However, does not lie inside the rectangle of any of the descendants of v; this is clear from the fact that the end points of $[x_j, y_j]$ lie on opposite sides of z. Therefore, v is the deepest node satisfying $[x_j, y_j] \subseteq \tilde{R}(v)$. □

Lemma 6.5.7: *For sufficiently small values of $\eta > 0$, there is a node v in F^* such that $\tilde{R}(v) \subseteq [a, b]$ and $\frac{1}{7}(b - a) \leq l(\tilde{R}(v)) \leq b - a$.*

Proof. To find such a node, we begin with the deepest w such that $[a, b] \subseteq \tilde{R}(w)$. Then the fair cut used to produce the children of w must lie in the interval $[a, b]$. Let w' be the child of w such that $\tilde{R}(w')$ lies on the side of the cut for which the resulting interval spans at least half the length of $[a, b]$. Note that the distance between $\tilde{R}(w')$ and the cut can be no more than $\eta(b - a)$, since the interval in between must be empty, and by the third condition on movement events, this can only hold if it is smaller than the maximum size of the interval of any movement event. Now, consider

the deepest descendant w'' of w' (possibly w' itself) that does not lie entirely in $[a, b]$. By the preceding observations, $l(\tilde{R}(w''))$ must be at least $(\frac{1}{2} - \eta)(b - a)$. Moreover, by the fair cut condition and the third condition on movement events, we see that it must have a child v such that $l(\tilde{R}(v)) \geq ((\frac{1}{2} - \eta)/3 - \eta)(b - a)$, which for sufficiently small η is at least $\frac{1}{7}(b - a)$. □

By Lemma 6.5.6 it follows that for sufficiently small η, the length of every $\tilde{R}(v)$ lying in $[a, b]$ of size greater than $\frac{1}{7}(b - a)$ must be output for some event in any subsequence fulfilling the stated conditions. Lemma 6.5.7 insures that one of these lengths is in the desired range, guaranteeing the correctness of the algorithm to compute each Δ_i.

6.5.2 Detecting movement for $d > 1$.

We generalize the preceding problem so that a movement event consists of a d-cube. Then, we wish to detect the case in which a subsequence of movement events forms a connected path of sufficiently great length as compared to the size of the largest cube in the sequence. For the d-dimensional problem, the output for each event will not be a single number, but a set containing d real numbers.

To handle such movement events on a d-dimensional point set P, we maintain d separate one-dimensional projections, each of which consists of the set of real numbers appearing as the ith coordinate of some point in P, for $i = 1, \ldots, d$. Then, the output for a d-dimensional movement event is obtained by projecting the given d-cube on each of the coordinate axes in turn, and treating the corresponding interval as a movement event in the one-dimensional point set maintained for that axis.

We will analyze the output of the above algorithm when we consider its application to the main problem. However, its usefulness can be seen intuitively from the following observation.

Observation 6.5.8: *Let $(a_1, \ldots, a_d)$ and $(b_1, \ldots, b_d)$ be points in $\Re^d$ whose Euclidean distance is r. Then*

i. There exists some i such that $|a_i - b_i| \geq r/\sqrt{d}$.

ii. For all i, $|a_i - b_i| \leq r$.

From this, one can see that a long, connected path of many small events in d dimensions will, when projected on each of the coordinate axes, become, for at least one such projection, a correspondingly long interval (smaller by at most a factor of $\sqrt{d}$) covered by relatively small event intervals.

6.5.3 Maintaining pair directions

Let $w_{\min}$ denote the function w under the assumption that $w(x)$ is equal to the smaller of the two values it may hold during the update of x. We will use $w_{\min}$ only when referring to nodes v that may be ancestors of x, since otherwise $w(v)$ is independent of $w(x)$. When a point x is updated, we begin by enumerating a set of distances $\mathcal{R}_x$, satisfying the following for all real $r > 0$: Let C denote the d-cube of length $2r$ whose center is x, and let C' denote a concentric d-cube of length $2\beta r$, where $\beta > 0$ is a constant to be determined in the analysis. Then if $w_{\min}(\rho_\delta(C')) \geq \frac{\kappa+1}{\kappa} w_{\min}(\rho_\delta(C))$, there must exist some $\hat{r} \in \mathcal{R}_x$ such that $r \in [1, \beta)\hat{r}$.

The following lemma can be shown using arguments that are similar to those in the proof of Lemma 6.5.4, but simpler.

Lemma 6.5.9: *For all x, there is a set $\mathcal{R}_x$ of size $O(\log |T|)$ satisfying the required conditions.* □

For efficiency we require the following result as well.

Lemma 6.5.10: *A set $\mathcal{R}_x$ satisfying the required conditions can be retrieved in $O(\log^2 |T|)$ time.*

Proof. We reduce this to the problem of approximate k-nearest-neighbor queries. For some integer m, suppose we consider a node v in T to be a leaf (and thus ignore all of its descendants) if the number of points in its subtree, including dummy points,

is less than or equal to m (i.e., $|(P \cup \hat{P}) \cap \tilde{R}(v)| \leq m$). Suppose we then place a single point at the center of $\tilde{R}(v)$ and consider approximate k-nearest-neighbor queries with respect to the modified tree. It is not hard to see how to adapt the algorithm of Arya et al. [10] to solve this problem using only the directed topology tree we already maintain. The key is to modify the point location query so that we do not continue to descend into regions containing less than m points.

Now we let $k = \kappa^2$ and solve the modified approximate k-nearest-neighbor problem for all values of m of the form $\lfloor (\frac{\kappa+1}{\kappa})^i \rfloor$, such that $m \leq |T|$, and let $\mathcal{R}_x$ consist of the set of distances to all such neighbors. Clearly, we must perform $O(\log |T|)$ queries, each of which requires $O(\log |T|)$ time [10]. Thus, the total time for constructing $\mathcal{R}_x$ is $O(\log^2 |T|)$. It remains to show that the definition of $\mathcal{R}_x$ is satisfied by these choices.

To see this, consider any r for which we can construct the d-cubes C and C', as used in the definition of $\mathcal{R}_x$, such that $w_{\min}(\rho_\delta(C')) \geq \frac{\kappa+1}{\kappa} w_{\min}(\rho_\delta(C))$. Then at least one of the modified approximate k-nearest-neighbor queries must be performed for $m \in (\frac{1}{\kappa^2} w_{\min}(\rho_\delta(C)), \frac{1}{\kappa^2} w_{\min}(\rho_\delta(C'))]$. Consider all rectangles in $\rho_\delta(C')$ whose centers lie outside of C. Note that the total weight of these rectangles is at least $\frac{1}{\kappa} w_{\min}(\rho_\delta(C))$, because any excluded rectangles must contribute to $\rho_\delta(C)$.

Because there are at most κ such rectangles, it follows that at least one of these rectangles has a descendant that is a leaf in the tree as modified for the value of m. In addition, the bound on $w_{\min}(\rho_\delta(C'))$ guarantees that it is one of the (κ^2)-nearest neighbors of x. One of the distances enumerated by a query is, thus, an approximation to the distance from x to some point outside of C and inside of C'. For a sufficiently small approximation constant ϵ, it follows that this distance is also an approximation to r as required by the definition of $\mathcal{R}_x$. □

The distances in $\mathcal{R}_x$ allow us to construct a small set of movement events that will capture all necessary information about the change in position of $R_{\text{dir}}(v, w)$. In particular, for each $\hat{r} \in \mathcal{R}_x$, we consider the cube of length $2\sqrt{d}\beta\hat{r}$, centered at x, to be a d-dimensional movement event, and treat it according to the data structure

we have developed. For the data structure to be applicable, we need to show that any such sequence of movement events results in a set connecting one position of $R_{\text{dir}}(v, w)$ to another. In order to prove this, we will need the following result.

Let $x_1, x_2, \ldots, x_m$ denote a subsequence of the points updated over the course of maintaining the decomposition, and let C_i denote, for each x_i, the movement event associated with the largest $\hat{r} \in \mathcal{R}_{x_i}$ such that $\hat{r} \leq \max\{d(x_i, x_{i+1}), d(x_{i-1}, x_i)\}$, ignoring the undefined distance for $i = 1$ and $i = m$.

Lemma 6.5.11: *Suppose that for all $i = 1, \ldots, m-1$, no point y is updated at any time between the updates of x_i and x_{i+1} such that y lies within a d-cube of length $2\beta\sqrt{d}\max\{d(x_i, x_{i+1}), d(x_{i-1}, x_i)\}$ centered at either x_i or x_{i+1}. Then $\bigcup_{i=1}^{m} C_i$ is a connected set in $\Re^d$.*

Proof. It will suffice to show that for all $i = 1, \ldots, m-1$, either $x_i \in C_{i+1}$ or $x_{i+1} \in C_i$, because this allows us to construct a path that lies in $\bigcup_{i=1}^{m} C_i$ and traverses $x_1, x_2, \ldots, x_m$ in order. To prove this, we construct d-cubes of length $\frac{1}{2\sqrt{d}} d(x_i, x_{i+1})$ centered at x_i and x_{i+1}. It is easy to verify that for sufficiently small δ, the δ-renderings of these d-cubes are disjoint. Let W_i and W_{i+1} denote, respectively, the total weight of the δ-renderings of the d-cubes constructed around x_i and x_{i+1}, under the assumption that $w(x_i)$ and $w(x_{i+1})$ each holds the minimum of its two values during the update of the corresponding point.

Now, assume $W_i \leq W_{i+1}$. In this case, we will show that $x_{i+1} \in C_i$. Let C denote the d-cube constructed around x_i, and let C' denote a concentric d-cube of length $\frac{\beta}{2\sqrt{d}} d(x_i, x_{i+1})$. For sufficiently large β, $\rho_\delta(C')$, which contains $\rho_\delta(C)$, must also contain the δ-rendering of the d-cube constructed around x_{i+1} Thus, it follows that $w(\rho_\delta(C')) \geq W_i + W_{i+1} \geq 2W_i$. Note that this inequality holds for both values of $w(x_{i+1})$.

The assumptions of the lemma guarantee that no other update can occur within C_i between x_i and x_{i+1}; hence none occur in C', and there must be some

$\hat{r} \in \mathcal{R}_{x_i}$ such that $\hat{r} \in [1, \beta)\frac{1}{2\sqrt{d}}d(x_i, x_{i+1})$. From this, one can easily verify that C_i contains C' and must therefore also contain x_{i+1}.

Using the same argument, one can show that if $W_{i+1} \leq W_i$, then $x_i \in C_{i+1}$. Clearly, either $W_{i+1} \leq W_i$ or $W_i \leq W_{i+1}$ must hold, so this suffices to prove the lemma. □

We now define a set $\mathcal{S}_x$, which consists of approximations to those distances around x at which a well-separated pair may need to change direction when the weight of x is updated. The distances in this set are computed according to two different cases. First, it may be that a single update causes the position of $R_{\text{dir}}(v, w)$ for some pair $\{v, w\}$ to shift by a distance that is significant compared to $l_{\max}(R_{\text{dir}})(v, w)$. To handle this case, we include all distances in $\mathcal{R}_x$ in $\mathcal{S}_x$. If this case does not hold, and the position of $R_{\text{dir}}(v, w)$ changes gradually over many updates, then, as we will show, there must be a subsequence of updates $x_1, x_2, \ldots, x_m$ satisfying the conditions of the preceding lemma, such that $d(x_1, x_m)$ is approximately equal to the distance moved. To handle the second case, we will include in $\mathcal{S}_x$ all of those distances returned for each of the corresponding movement events. Then, because the union of movement events is a connected set, one of the distances returned over the subsequence of updates must be approximately equal to $d(x_1, x_m)$, and thus approximately equal to the total shift.

Observation 6.5.12: $|\mathcal{S}_x| = \mathrm{O}(\log |T|)$.

Finally, we are ready to describe the algorithm for enumerating all pairs that may need to be redirected. For every $v \in \mathcal{S}_x$, we consider those pairs (u, w) such that $d(\tilde{R}(u), x) \leq \gamma r(v)$ and $\tilde{d}(u, w) \in [\gamma^{-1}, \gamma] r(v)$ for some constant γ that will be made sufficiently large to insure the correctness of our algorithm.

Lemma 6.5.13: *For each $u \in \mathcal{S}_x$, the number of (v, w) we consider depends only on γ and the dimension d.*

Proof. This follows directly from Lemma 4.6.1 □

Having enumerated the above set of pairs, we consider each as a candidate to be redirected. Lemmas 6.4.1 and 6.5.5 imply that we can find the correct direction and, if necessary, redirect each pair so that it is strongly directed in $O(\log n)$ time. The correctness of our algorithm follows from the theorem below.

Theorem 6.5.14: *The direction of every pair (v, w) satisfies (6.1) at all times over any sequence of weight updates.*

Proof. For a pair (v, w), consider any time interval over the update sequence that begins with an update at which (v, w) is redirected, and is thus strongly directed, and ends with the first update at which (v, w) no longer satisfies (6.1). Let $x_1, \ldots, x_m$ denote the subsequence of all points updated over that interval such that for all $i = 1, \ldots, m$ there is some $R \in \rho_\delta(C(v, w))$ such that $x_i \in R$. To prove the theorem, it will suffice to show that (v, w) is redirected during the update of some x_i, because the only way (6.1) could ever be violated is for such a sequence of updates to occur without (v, w) being redirected.

We consider two cases. For the first, and simpler case, assume that for some x_i, there is some $\hat{r} \in \mathcal{R}_{x_i}$ such that $\hat{r} \in [\gamma^{-1}, \gamma]\tilde{d}(v, w)$. Then, because $\hat{r}$ is also in $\mathcal{S}_{x_i}$, one can easily verify that the well-separated pair $\{v, w\}$ will be among those in the set of pairs to be redirected at this update.

Otherwise, it must be the case that for all $i = 1, \ldots, m$ there is no $\hat{r} \in \mathcal{R}_{x_i} \cap [\gamma^{-1}, \gamma]\tilde{d}(v, w)$. This fact gives us some information about the distribution of points around each x_i. That is, suppose we assume $\beta \geq \sqrt{d}\gamma$. Let C be a cube of length $2\gamma^{-1}\tilde{d}(v, w)$ centered at x_i, and let C' be a concentric cube of length $2\beta\gamma^{-1}\tilde{d}(v, w)$. Then by the definition of $\mathcal{R}_x$, $w_{\min}(\rho_\delta(C')) < \frac{\kappa+1}{\kappa} w_{\min}(\rho_\delta(C))$.

From this, it is easy to verify that any rectangle in $\rho_\delta(C(v, w))$ that lies outside of C cannot possibly be $R_{\text{dir}}(v, w)$, since its weight is too low. It follows that each x_i in an update sequence corresponding to the present case must lie at a distance no greater than $\sqrt{d}\gamma^{-1}\tilde{d}(v, w)$ from $R_{\text{dir}}(v, w)$ when $w(x_i)$ has the minimum of its two values during the update. If the update changes the position of $R_{\text{dir}}(v, w)$,

then clearly x_i is contained in $R_{\text{dir}}(v,w)$ when $w(x_i)$ has the maximum of its two values.

The above guarantees that x_i is a good approximation to the position of $R_{\text{dir}}(v,w)$ both before and after the update of $w(x_i)$. Hence, our assumptions about the update sequence guarantee that $d(x_1, R(v)) \geq (\frac{1}{2} - \sqrt{d}\delta)r$ and $d(x_m, R(v)) \leq \frac{1}{3}r$. Let j denote the maximum j between 1 and $m-1$ such that $d(x_j, x_m) \geq (\frac{1}{6} - \sqrt{d}\delta)r$. Note that such a j must exist according to the preceding inequalities.

The sequence $x_j, \ldots, x_m$ satisfies the assumptions of Lemma 6.5.11. This follows from the fact that the points in the sequence $x_j, \ldots, x_m$ lie within the interior of $C(v,w)$ and sufficiently far from the boundary that any y violating the assumptions of Lemma 6.5.11 must also lie in $C(v,w)$. But any such y that was updated between x_j and x_m would have to be included in the sequence $x_j, \ldots, x_m$. Hence, by Lemma 6.5.11, there is a connected sequence of movement events spanning the distance between x_j and x_m.

By Observation 6.5.8, the projection of these movement events on one of the d dimensions must cover an interval of length at least $\frac{1}{\sqrt{d}}(\frac{1}{6} - \sqrt{d}\delta)r$. The projection of each movement event must itself be small, compared to the length the larger interval, thus satisfying the second condition on movement events, for sufficiently large values of γ. To satisfy the third condition on movement events, we assume that the dummy points introduced in section 6.3 are included in the trees maintained to for each dimension. It is easy to verify that for sufficiently small ϵ, the dummy points introduced for pairs in the same distance class around $R(v)$ as $\{v,w\}$ must cover the interval in the required fashion.

It then follows from Lemmas 6.5.6 and 6.5.7 that some distance approximately equal to $d(x_j, x_m)$ must be output during the update of one of the points in the sequence $x_j, \ldots, x_m$. Therefore $\{v,w\}$ will be redirected at that step, proving the theorem. □

At any update, there are $O(\log |T|)$ pairs to be redirected. These can be determined in $O(\log |T|)$ time for each pair, using a search on the compressed tree.

The redirections can each be performed in $O(\log |T|)$ time, including the time to recompute values in the abstract tree-pair computation. We summarize this in the following theorem.

Theorem 6.5.15: *In $O(|T|)$ space and worst-case $O(\log^2 |T|)$ time per update, an abstract tree-pair computation can be maintained on a fair split tree under arbitrary changes to leaf weights (between 0 and $|T|$) such that the directions of pairs are consistent with the weight assignment at all times.* □

6.6 Applications

Using the preceding results, any algorithm that can be expressed as an abstract tree-pair computation on the well-separated pair decomposition of the point set P can be made fully dynamic with a worst-case time complexity of $O(\log^2 |P|)$ for point insertions, deletions, updates to leaf values, and queries at leaves. Note that insertions and deletions can be performed without respect to the pairs, so long as $w(x) = 0$ and $BU(x)$ is the identity, and these values can later be modified. Updates and queries can be performed by first setting $w(x)$ to $|T|$. This guarantees that $out(x) = 0$. Hence an update can be performed simply by changing $BU(x)$. Additionally, a query is valid for the top-down algorithm applied to pairs directed both ways (as ours will be) because all pairs on the path from the root to x are incoming. The "usual" value of $w(x)$ will be 1, and it is during the change between this weight and $|T|$ that the actual computation is performed, by redirecting pairs and making the information consistent with this redirection. We discuss some specific applications below.

6.6.1 Maintenance of attractor sets

In Vaidya's all-nearest-neighbors algorithm [67, 69] as well as the algorithm of section 3.1, nearest neighbor pairs are extracted from a set of pairs of points and their *attractors*. We define the attractors of a point x to be those points y that have x

as a nearest neighbor, though this deviates somewhat from Vaidya's definition. It is well-known that any point has O(1) attractors. Both of the cited algorithms retrieve, for each point, a set of O(1) points that includes all of its attractors. For the purposes of this and the following dynamic application, we will refer to this set as the *attractor set* of x, even though it is technically a superset. To maintain attractor sets dynamically, we will pose the bottom-up and top-down phases of the k-nearest-neighbors algorithm of section 3.1 (for $k = 1$) as an abstract tree-pair computation.

For the bottom-up computation, we define the + operator as follows. An element in the domain of + is either a point in $\Re^d$ or else one of the two special elements 0 and ∞. A leaf corresponding to a dummy point has the value 0, while a leaf corresponding to a point in P has that point as its value. The element 0 acts as the identity under +, while any sum that does not involve 0 has ∞ as its result. It is easy to verify that + is associative under this rule, and that its effect is to label each v in T with 0 if $\sigma(v) \cap P$ is empty, with the unique $x \in \sigma(v) \cap P$, if $\sigma(v) \cap P$ is a singleton, and with ∞ if $|\sigma(v) \cap P| > 1$.

Recall the k-nearest-neighbors algorithm of section 3.1. For this algorithm, we used the result of Lemma 3.1.1, which in this context implies that we can ignore the pair $\{v, w\}$ on any root-to-leaf path containing v if $BU(w) \in \{0, \infty\}$. That is, we may restrict our attention to edges from v to nodes w such that $\sigma(w)$ contains exactly one point from P. Also recall Lemma 3.1.2, which implies that each attractor of any point x is the nearest point to x within some narrow cone in a partition of space around x. Note that it will suffice to use projected distance along the axis of the cone, since all attractors will minimize this value as well as Euclidean distance. Thus, we may proceed as in the parallel algorithm of section 5.4.2 and begin by constructing a covering of space by a set of O(1) overlapping cones, each defined by some axis vector u. For each well-separated pair, let u be its approximate direction vector, as defined in section 3.4.

For the top-down computation, the elements of $\oplus$ will consist of vectors of elements from $P \cup \{0\}$, with one element for each axis vector u. Suppose (v, w) is an

outgoing pair. Then to convert $BU(v)$ into a vector in the domain of $\oplus$ we set all elements to 0, except for that corresponding to u, the approximate direction of the pair $\{v, w\}$. We set this element to $BU(v)$ if $BU(v)$ is a point in P; otherwise, we set it to 0. F_w is then computed as a $\oplus$-sum of all such vectors computed for w. To evaluate $\oplus$, we compute the vector sum in a pairwise fashion, again considering 0 to be the identity. When both elements to be added are points in P, we let the result be that point that minimizes a dot product with u (thus corresponding to the nearest point within a cone). It is easy to verify that $\oplus$ is associative. Using results given for earlier nearest-neighbor algorithms, it is easy to verify that the set of points in the vector computed for each leaf x is a superset of the attractors of x. Thus, we have an algorithm for maintaining attractor sets under insertions and deletions of points.

Note that this does not appear to result in an efficient dynamization of the all-nearest-neighbors problem. The reason is the final step in which all attractor pairs must be considered as a batch. In the dynamic algorithm, a query operation on a leaf x will retrieve its attractor set, including all those points that may have x as a nearest neighbor. It does not provide us with any way to select between the possible candidates for the nearest neighbors of x, particularly when there are many at the same approximate distance, nor does it allow us to determine which points in the attractor set of x have x as a nearest neighbor. It is nonetheless interesting to have a fully dynamic version of the most involved portion of the all-nearest-neighbors algorithm. Additionally, dynamic attractor set maintenance provides an elegant solution to the dynamic closest pair problem.

6.6.2 Dynamic closest pair

In the dynamic closest pair problem, we must maintain a point set P under insertions and deletions of points in a way that allows us to maintain a pointer to the *closest pair*, or the pair $\{x, y\}$ ($x, y \in P$ and $x \neq y$) that minimizes $d(x, y)$. We solve this problem by keeping track of a set S of all *mutual attraction pairs* $\{x, y\}$. These are

the pairs such that x is in the attractor set of y and y is in the attractor set of x. Clearly, the closest pair is a mutual attraction pair. If S can be maintained explicitly, then a heap can be used to keep track of the closest pair. This will allow its retrieval in constant time. Heap maintenance requires $O(\log |P|)$ time per update, so it will not affect the total asymptotic complexity.

Thus, the problem reduces to determining which mutual attractors are created and destroyed during an insertion or deletion of the point x. We are aided in this by the following observation.

Observation 6.6.1: *Let x, y, and z be points in P such that $\{y, z\}$ is a mutual attraction pair in one of P and $P - \{x\}$, but not both. Then either y or z must be in the attractor set of x for the point set P.*

This is easily verified for the sets of true attractors and can be extended to hold for the supersets computed by the dynamic algorithm. Note that x, y and z need not all be distinct, and it will be useful in particular to consider $z = x$, since this corresponds to the case in which a mutual attraction pair is destroyed by the deletion of x. More interesting is the case in which the deletion of x creates a new mutual attraction pair $\{y, z\}$ where $y, z \neq x$. However, the observation covers this case as well. Its significance is that we can determine all changes to the set of mutual attractors for an update involving x by considering only those points in the attractor set of x. There are $O(1)$ such points, so the total time to maintain the closest pair is worst-case $O(\log^2 |P|)$ time per insertion and deletion.

6.6.3 Fast multipole method

Recall that in section 3.2, we presented an adaptation of the fast multipole method [37, 38] for approximating the electrostatic potential of sets of charges using the well-separated pair decomposition. It is straightforward to represent this algorithm as an abstract tree-pair computation in which + is the operation for adding multipole

expansions and $\oplus$ is the operation for adding local expansions. The operations are, in fact, quite similar to those used in the rake-and-compress parallel algorithm of section 5.4.1. Dummy points may be handled simply by setting their charge strengths to 0. As a result, we may maintain arbitrary sets P of particles with charges in $O(\log^2 |P|)$ worst case time for queries and updates. A query computes the potential at position x due to all charges in $P - \{x\}$.

Chapter 7

Empirical tests

In this chapter, we consider an implementation of our sequential algorithm for computing the well-separated pair decomposition. This algorithm is the simplest among those presented in this dissertation, and seems the most likely to be useful for practical applications. By performing tests on our implementation, we obtain an estimate of the packing constants for which we have derived theoretical upper bounds. This gives us some insight into the input sizes for which the results presented here may have practical value. In our theoretical analysis, our goal was to simplify the proof techniques, and this resulted in substantial overestimates of the constants. For this reason, we consider the following empirical study an essential supplement to theoretical analysis.

7.1 Implementation considerations

The theoretically optimal $O(n \log n)$ sequential algorithm for constructing a fair split tree seemed a good candidate for implementation, since the auxiliary data structures were straightforward to program. This algorithm was implemented in C and worked well enough in practice to run on a desktop workstation (a SparcStation ELC) for point sets numbering in the tens of thousands. Due to the number of pointer cross-references, however, each point required $8 + 24d$ bytes of auxiliary storage while the

tree was being constructed. These space requirements proved prohibitive for larger point sets.

To make larger point sets feasible, we implemented a program to construct fair split trees by performing a split and recursing directly on each half, even if this resulted in an unbalanced divide-and-conquer. The complexity of this approach is $O(n \min\{\log(d_{\max}/d_{\min}), n\})$ where $d_{\max}$ and $d_{\min}$ denote the largest and smallest interpoint distances, respectively. After implementing the simpler algorithm, it was found to require considerably less space, allowing much larger point sets to be tested. Additionally, it actually ran faster than the $O(n \log n)$ algorithm for all point sets tested so far due to decreased overhead requirements. The final set of tests were performed using a Sparc 10, and involved sets of up to two million points in three dimensions.

The point sets tested were generated from a uniform distribution in a cube. In this case, $d_{\max}/d_{\min}$ tends to be a polynomial in n, so the complexity is no worse than that of the optimal algorithm. In fact, it is difficult to represent point sets for which $\log(d_{\max}/d_{\min})$ is significantly greater than $\log n$, and such sets do not arise in any known practical applications.

7.1.1 Specifics of the decomposition

In our experiments, we were primarily interested in computing efficiency factors that result from geometry and are therefore implementation-independent. The advantage of this approach is that our results should be reproducible by any program on any computational platform. We consider these numerical results to be a useful supplement to theoretical analysis, allowing us, among other things, to determine the point at which it is worthwhile to use the well-separated pair decomposition in place of a quadratic time algorithm for the same problem.

In order to make these results completely reproducible, it is essential to outline some specifics of the well-separated pair decomposition that we compute for

our point sets, since the decomposition is not uniquely defined for a point set.

The fair split tree

The fair split tree we compute is that which is obtained by the sequential algorithm of Chapter 2. In particular, it is the tree computed recursively by finding the bounding rectangle of a point set, and splitting its longest dimension exactly in half geometrically.

The well-separated realization

For reasons of efficiency, we deviate slightly from the sequential algorithm to compute a realization. The main modification is that we do not store a bounding rectangle at every internal node of the tree, but instead a bounding sphere, centered at the center of the bounding rectangle. This saves space, because we need only store a center point and a radius. It also saves time in our method of determining the separation of two nodes. For practical reasons, we define the radius of the bounding sphere to be the smallest numerically representable radius larger than the radius of the circumscribed sphere of the bounding rectangle. Minor deviations from this definition should not change the experimental results significantly.

To compute our realization, we make the following modifications to the sequential algorithm that computes the canonical realization. First, instead of splitting the node with the largest value of $l_{\max}$ during the recursion, we split the node whose bounding sphere has the largest radius. Second, our function to determine if two spheres are well-separated is somewhat more conservative than necessary. Given two bounding spheres of radii r_1 and r_2 whose center-to-center distance is ℓ, we define them to be well-separated if $\ell - (r_1 + r_2) > s \max\{r_1, r_2\}$. Note that any two nodes said to be well-separated by this definition are well-separated by the original definition, though the converse may not hold. One can show, using the same analytical techniques as before, that the realization computed in the above manner satisfies all

the desirable properties of the canonical realization.

7.2 The number of well-separated pairs

While it has been shown that the number of well-separated pairs is always linear in the number of points, the theoretical upper bound on the constant is much too large to be practical. This is because the primary goal was to simplify proof techniques and guarantee that the results were fully general. To determine the values in practice, empirical tests were performed on points drawn from a uniform distribution in a cube.

We conjecture that the uniform distribution actually gives a good idea of the worst case for the number of pairs, as these distributions tend to pack the space in the neighborhood of a node in the fair split tree, making it more likely that any possible pair involving this node will actually occur. For additional intuition, note that the point set that seems qualitatively least like the uniform distribution is one in which points are placed at exponentially increasing distances from the origin (used in section 4.1 to prove the lower bound on depth). This distribution achieves the absolute minimum number of pairs, as each node in the tree is paired with its sibling, resulting in $n - 1$ well-separated pairs.

It is interesting that the factor for which we have a constant upper bound actually varies considerably with n for points in a cube, and only begins to level off for very large values. This seems to be due to the fact that a large percentage of points are close to the boundary of the cube when n is small, and these belong to fewer pairs. To test this claim, we introduced periodic boundary conditions so that, for example, a point close to the top of the cube would have as its near neighbors points at the bottom of the cube, thus eliminating the problem of empty space. With this modification, the number of pairs approached its limit much more rapidly. For this reason, we have chosen to use periodic boundary conditions in most of our experiments.

The numerical parameters to be varied during testing are n, the number of

points, d, the number of dimensions, and s, the separation. While it is not feasible to test all possible assignments to these values in any but the most limited range, some idea of the dependencies can be determined by varying one parameter while keeping the others fixed. In this section, the value we will be computing is the parameter c, which is the ratio between the number of pairs and n. Based on our theoretical bounds, we expect c to approach a fixed limit as n increases.

7.2.1 Varying n

Our first test involved keeping s and d fixed while increasing n. The purpose of such a test was to get an idea of the constant in the bound to the number of pairs. We let $d = 3$ and $s = 2.0$ for this test. While this choice is somewhat arbitrary, it has practical significance, because three dimensional point sets are very common in applications, and the separation s would be set to roughly 2.0 for applications such as all-nearest-neighbors. For larger values of d, the computation time would have been prohibitive in the range we chose for n.

When performing this test initially, we were surprised to discover that c continues to increase significantly even for n greater than 100000. The graph of c is shown in figure 7.1. Because we can test sets of at most several million points in three dimensions, this creates difficulty in concluding with confidence that c is approaching a limit.

Fortunately, it appears that the greatest part of this effect is due to the fact that points close to the boundary of the cube are involved in fewer pairs than those in the interior. We can eliminate the distinction between boundary and interior points by introducing periodic boundary conditions. Formally, we compute distance between vectors x and y in a periodic unit cube as

$$\left(\sum_{i=1}^{d} \min\{(x_i - y_i)^2, (x_i - y_i - 1)^2\} \right)^{\frac{1}{2}}.$$

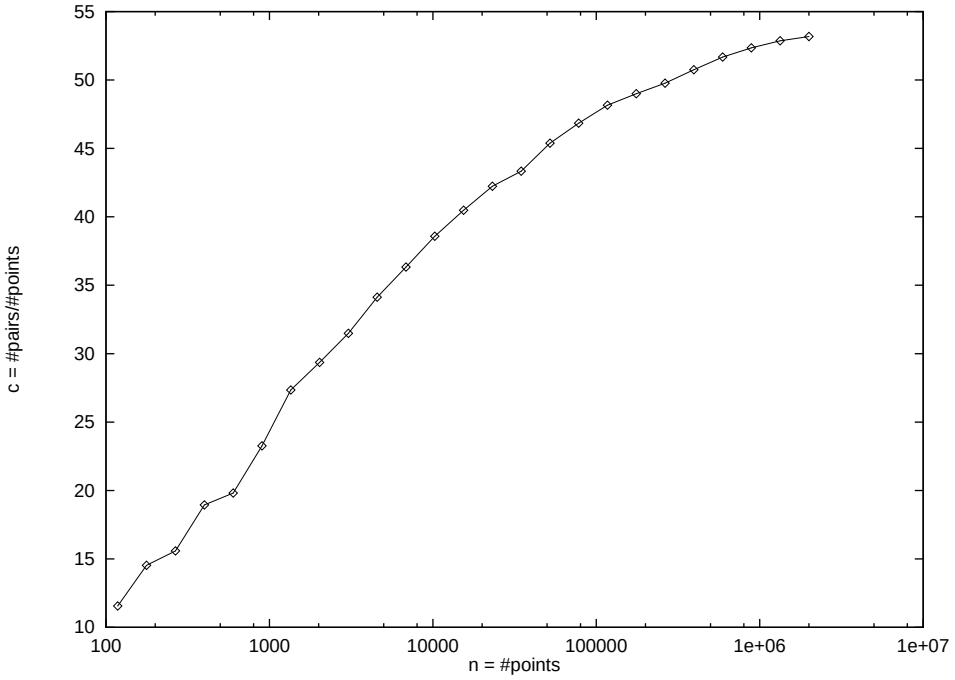


Figure 7.1: Uniform distribution of points in a cube ($d = 3$, $s = 2.0$)

Repeating the preceding test with periodic boundary conditions, we obtained the graph shown in figure 7.2. These data suggest that when $s = 2.0$ and $d = 3$, the number of well-separated pairs in a realization for uniformly distributed points can be conservatively bounded below $60n$. While this is a large constant factor, it appears to be unavoidable due to the geometry. Also, it suggests that our algorithms would nevertheless be preferable to naive quadratic algorithms for values of n in a reasonable range.

Because periodic boundary conditions cause c to approach its limit more rapidly (making it possible to estimate using smaller data sets) we used this distance metric in all subsequent tests.

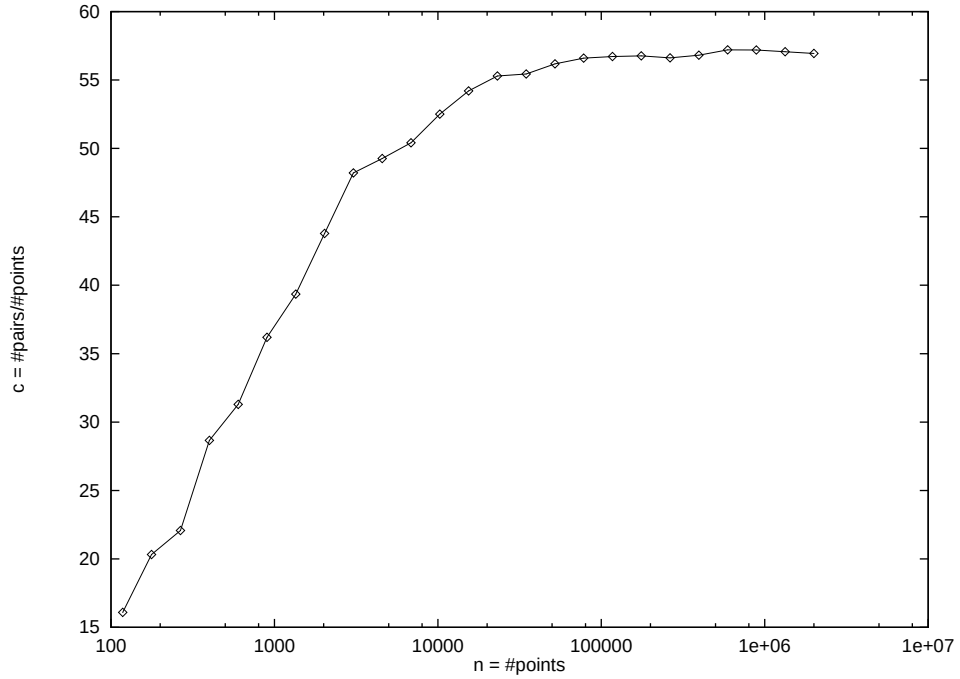


Figure 7.2: Uniform distribution of points in a periodic cube ($d = 3$, $s = 2.0$)

Comparison to Greengard's decomposition

In his development of the fast multipole method in three dimensions, Greengard [38] chooses well-separated pairs based on grid geometry, and in effect uses L_∞ distance as the condition for separation. In particular, a pair of unit cells on a three-dimensional grid are considered well-separated so long as there is at least one coordinate for which their values differ by 3 or more.

As a result, the Euclidean distance between the centers of two well-separated unit cubes may be as little as 3, though it should be noted that many pairs achieve a greater separation. Because the circumscribed sphere of a unit cube has radius $\frac{\sqrt{3}}{2}$, the separation constant by our definition is $2(3 - \sqrt{3})/\sqrt{3}$, which is no more than 1.47.

Greengard proves that each node in his structure is paired with at most

875 other nodes. Compensating for double-counting, his decomposition is therefore bounded to have at most $437.5n$ well-separated pairs. Because the separation he achieves is less than ours in the worst case, and our empirical bound on the number of pairs is significantly lower than $437.5n$, we conclude that there are practical advantages to using Euclidean distance and not L_∞ distance as the criteria for separation. In fact, the only real advantage of pairing cubes based on a fixed grid seems to be simplicity of determining all pairs. In a complete implementation of the fast multipole method, however, the total numerical computation performed for each pair will far outweigh the computation needed to find well-separated pairs using the Euclidean metric. For this reason, we believe a multipole implementation using our decomposition would be preferable to a strict implementation of Greengard's algorithm in three dimensions.

7.2.2 Varying s and d

We have also performed several tests to determine the effect of varying s and d on the pair constant c. The graphs of these tests are shown in figures 7.3 and 7.4. For these, we kept n constant at 10000. This relatively small number of points was chosen because it allowed s and d to be set to higher values than would have been feasible for larger values of n, due to excessive time and space requirements. While this gives an underestimate of the ultimate limit of c, it gives some idea of its value for many point sets of practical significance.

From our theoretical analysis, we know that the bound on c is $(f(d)s)^d$ for some increasing function $f(d)$. At first glance, this seems to be contradicted by the graph shown in figure 7.3. Because the bound on c is superexponential in d, one would expect its curve, when plotted with a logarithmic y axis, to lie always above some straight line. Clearly this is not the case for the curves depicted. Roughly speaking, this occurs because in higher dimensions, the interior of a d-cube is only sparsely packed even for large values of n. A more formal way to understand this is

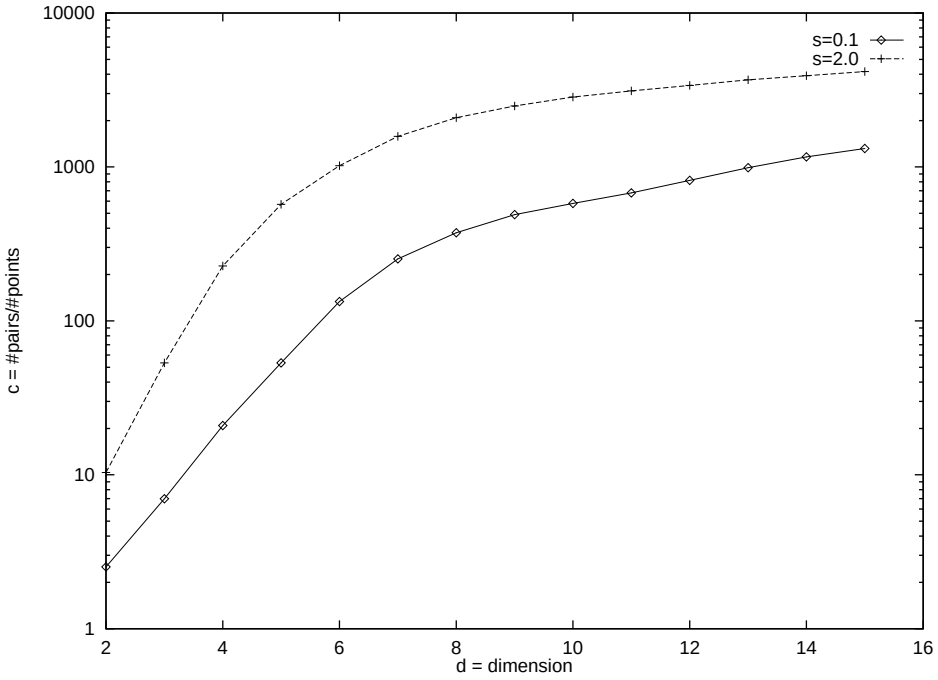


Figure 7.3: The pair constant c for varying d ($n = 10000$)

to recall that the number of well-separated pairs is at most the number of distinct pairs of points, or $\binom{n}{2}$. Hence, for any choice of d and s, the upper limit to c in the graphs presented above cannot exceed $\frac{n-1}{2}$ or 4999.5.

7.3 Tradeoffs and hybrid approaches

In this dissertation, the well-separated pair decomposition was proposed as a general method to solve certain problems posed on the complete Euclidean graph without constructing it explicitly. Typically, we argued in favor of our method by contrasting the quadratic size of the complete graph with the linear size of our decomposition. The situation is not quite so simple, however, if we are interested in optimizing performance in practice. The problem stems from the increased overhead of performing some computation on a pair of well-separated sets rather than a pair of points. For

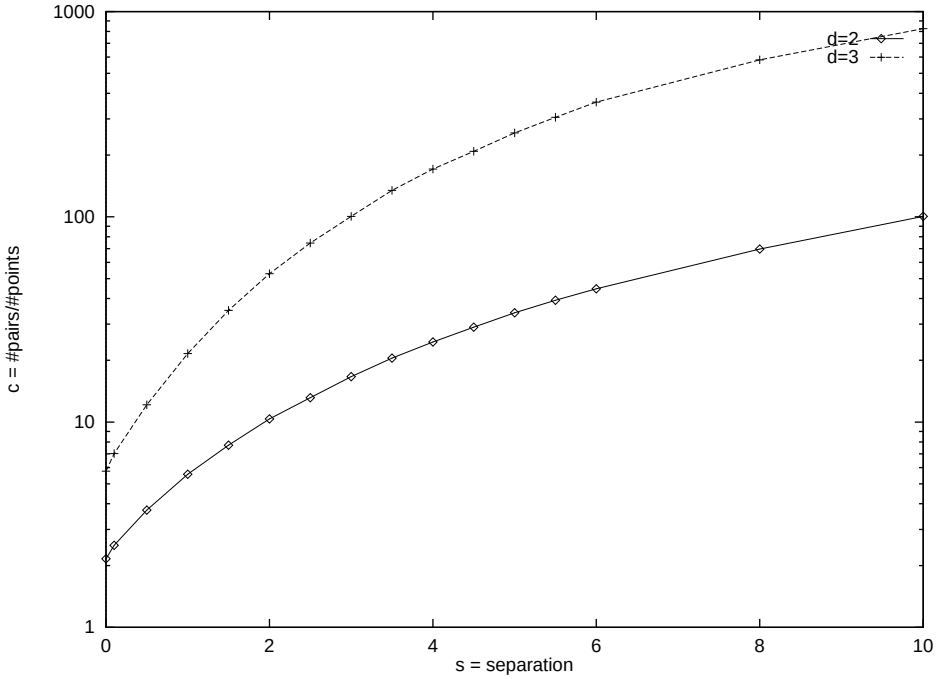


Figure 7.4: The pair constant c for varying s ($n = 10000$)

some applications, most notably the fast multipole method, this overhead is quite high.

It is reasonable to assume that any algorithm that performs an operation on the well-separated pair $\{A, B\}$ could just as easily have performed $|A||B|$ separate operations, one for each pair $\{a, b\}$ such that $a \in A$ and $b \in B$. This observation is made rigorous by the fact, noted earlier, that $\{\{a\}, \{b\}\}$ is itself a well-separated pair. However, for present purposes, we will assume that the cost of treating a pair of points is smaller by a constant factor than the cost of treating a pair of well-separated sets in the most general case. We call this factor the *tradeoff constant* α. When this assumption holds, it is to our advantage to implement a hybrid algorithm that invokes the general well-separated pair computation on the pair $\{A, B\}$ only when $|A||B| > \alpha$, and otherwise performs $|A||B|$ separate operations on the corresponding pairs of points.

In fact, it is standard in implementations of the fast multipole method to make use of a similar tradeoff by constructing a tree in which leaves consist of boxes containing more than n_0 points, where n_0 is some constant chosen to give the greatest practical performance (see, for example, the adaptive algorithm of Greengard [38]). The well-separated pairs obtained from this tree are used for multipole computations, while any pair of points $\{a, b\}$ not covered by a well-separated pair of tree nodes is treated by simply adding the effect of a to the potential at b and vice versa. This computation of point charge influence is in general much faster than any of the primitive operations on truncated multipole expansions.

In this section, we present the results of tests we performed to explore our own formulation of this tradeoff. Because the value of α will vary from application to application, we used an approach that would allow us to predict the best possible speed-up for a hybrid implementation using the well-separated pair decomposition under some assumption of the value of α. To do this, we ran our implementation on test sets composed, as usual, of points uniformly distributed in a cube with periodic boundaries.

In this case, rather than simply counting the total number of pairs, we produced a list of well-separated pairs $\{A, B\}$ sorted by the value of $|A||B|$. Given some α, we used this list to estimate the best speed attainable by a hybrid method, counting only the time for computations on point pairs and well-separated pairs. Experiments with our implementation have supported the standard assumption that this, rather than overhead such as tree construction, is the bottleneck in the fast multipole method. While tree construction has asymptotic complexity $\Omega(n \log n)$, the constant in this bound is small, and the additional $\log n$ factor is overshadowed by the geometric constant in the size of the well-separated realization for practical values of n.

Let $\{\{A_1, B_1\}, \ldots, \{A_m, B_m\}\}$ denote some well-separated realization computed by our implementation. Then, the best complexity attainable by any applica-

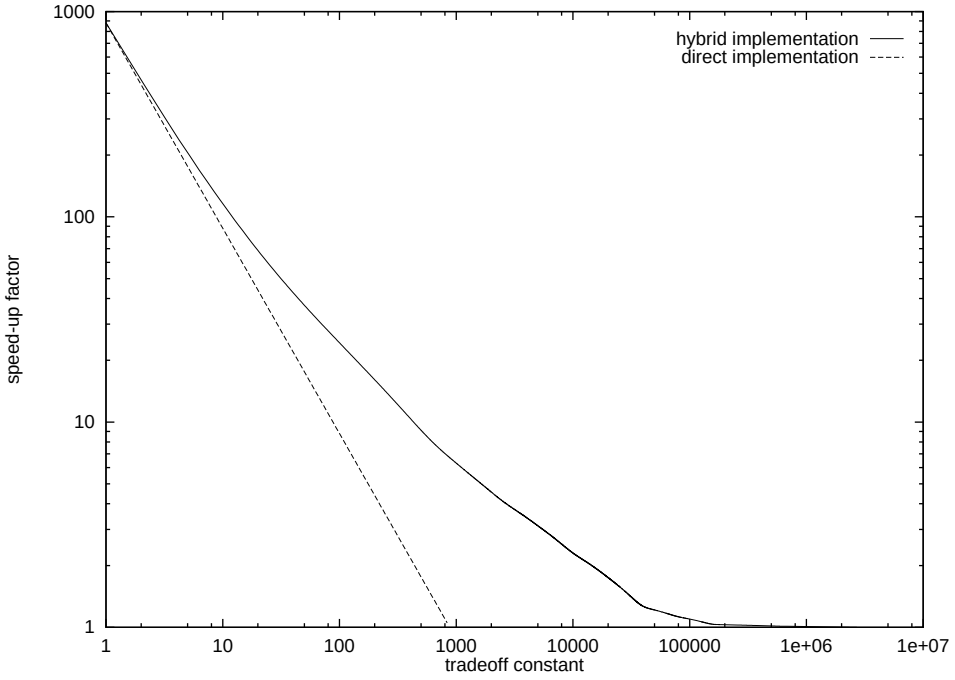


Figure 7.5: Speed-ups in hybrid and direct approaches ($d = 3$, $s = 2.0$, $n = 100000$)

tion that uses this realization is:

$$\sum_{i=1}^{m} \min\{\alpha, |A_i||B_i|\}.$$

Since we are charging only for computations on pairs, the naive algorithm that treats all pairs of distinct points has a cost of $\binom{n}{2}$. Thus, we define the *speed-up factor* for the hybrid algorithm to be the ratio of $\binom{n}{2}$ and the value computed by the above formula.

By the preceding cost metric, it is clear that the hybrid approach cannot do any worse than the direct implementation of any well-separated pair application. Such an implementation has complexity αm, and hence a corresponding speed-up factor of $\binom{n}{2}/(\alpha m)$. Nevertheless, it is not worth the additional complication to implement the hybrid approach unless it gives a significant improvement.

Figure 7.5 shows the comparison between direct and hybrid approaches for

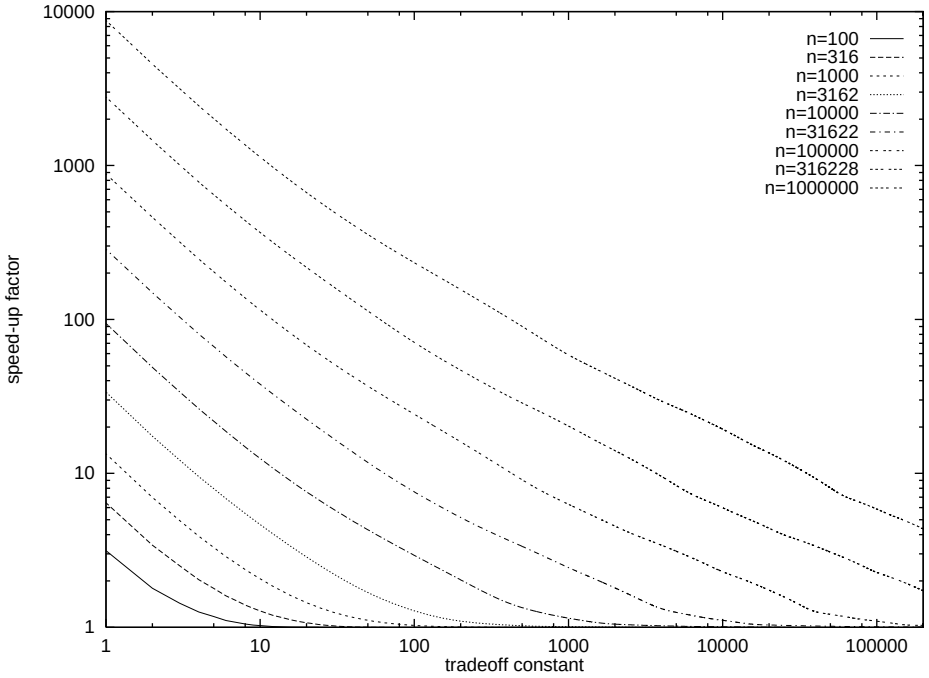


Figure 7.6: Speed-ups for hybrid approach ($d = 3$, $s = 2.0$)

$n = 100000$, $d = 3$, and $s = 2.0$. Note that the hybrid approach extends the range of α for which the well-separated pair decomposition is useful. In this case, the direct implementation has a speed-up factor below 1 (and hence not useful) for $\alpha > 880.5$. For the same value of α, the hybrid implementation has a speed-up factor of 6.67, which is quite significant in practice. In fact, the hybrid algorithm has a speed-up factor of 2 even for α as high as 14350, which is well outside the useful range of the direct implementation.

Figure 7.6 shows the speed-ups that can be obtained under certain assumptions of n and the tradeoff constant α. For this graph, we have fixed $d = 3$ and $s = 2.0$, though the values we are most interested in will depend on the application. Note that the empirical speed-ups determined in this manner seem a bit disappointing given the theoretical promise of reducing quadratic time to linear time. However, even for α in the thousands (which may occur in multipole computation), there are

realistic values of n for which the speed-up is quite significant.

In section 3.2, we noted that there is a tradeoff between s and p, the number of terms in the truncated multipole expansion, when performing a potential computation to a given precision. Because the cost of handling a well-separated pair grows with p, it would be worthwhile to consider a tradeoff function $\alpha(s)$ in carrying out the preceding analysis for the fast multipole method. The exact value of this function is implementation-dependent, and it is therefore impossible to treat in the present context. However, given a particular implementation, one could make a very accurate empirical determination of $\alpha(s)$. This suggests a rigorous approach for optimizing the performance of the fast multipole method using the well-separated pair decomposition.

Chapter 8

Conclusions

In this dissertation, we have developed a data structure that captures a great deal of useful information about the set of interpoint distances of a d-dimensional point set. We have demonstrated its utility by applying it to several important algorithmic problems posed on point sets. While the techniques presented here share some similarity with previous approaches, we consider the explicit definition of the decomposition to be a significant advance in the following ways.

- By removing the well-separated pair decomposition from the context of the algorithms that construct it, one can focus on the geometric properties of well-separated pairs. This simplifies the development of applications, as in the case of all-nearest neighbors, and may lead to a more refined error analysis for approximate algorithms, as in the case of the approximate Euclidean minimum spanning tree.

- By eliminating inessential restrictions on the type of tree to be constructed, one can develop efficient parallel and dynamic algorithms to construct the decomposition. In previous related work, the tree was either defined in terms of a sequential algorithm or else according to a rigid box hierarchy that seems to require use of the floor function. Our flexible characterization has led to efficient parallel and dynamic algorithms using only standard algebraic operations.

- By exploiting the combinatorial structure of the decomposition, one can apply many sophisticated techniques that have been developed for tree-structured computation. These lead to efficient parallel and dynamic algorithms for many of the applications treated in this dissertation.

8.1 Future directions

While many of the algorithms presented here are optimal, several results leave room for improvement. For example, one may ask whether concurrent reads are truly necessary for the optimal parallel algorithm presented in Chapter 5. It seems likely that exclusive reads are sufficient; however, this may rule out the current approach of using simultaneous searches to compute the pairs.

The dynamic algorithms can also be improved. These have time complexity of $O(\log^2 n)$, and it is natural to attempt to reduce this to $O(\log n)$. While this may be possible, it will require a much more refined analysis of the contracted decomposition. There are trees in which any leaf-to-root path has logarithmically many outgoing pairs, and it is unlikely that we can reduce the computation involving any individual pair below $\log n$. Thus, to reduce the complexity below what has been obtained here, it will be necessary to combine some of the work involving different pairs when handling all of the pairs along some path in the tree. Preliminary studies suggest that this may be possible, but the algorithm would be significantly more complicated.

A final area for future work is further study on how these algorithms perform in practice. The packing constants in well-separated pair algorithms are quite large and grow exponentially with increasing dimension. One cannot make any reliable conclusions about the practical significance of such algorithms without empirical testing. In this regard, it may be useful to consider heuristic modifications to the type of tree, especially those that adapt more precisely to a given distribution of points. This could in some cases lead to a reduction in the number of well-separated pairs.

We conjecture that many practical benefits can be realized for three dimen-

sional point sets. In this case the techniques developed for planar point sets no longer suffice, and yet the packing constants remain small enough to promise significant improvement over naive quadratic algorithms. These benefits may remain for somewhat higher values of d. For substantially larger dimensions, the upper bounds on packing constants lose their direct relevance, since they are never attained for realistic point sets. Nevertheless, the algorithms presented here remain applicable and may prove useful. However, the benefits can only be determined experimentally.

Bibliography

[1] S. J. Aarseth, J. R. Gott III, and E. L. Turner. N-body simulations of galaxy clustering. I. Initial conditions and galaxy collapse times. *The Astrophysical Journal*, 228:664–683, 1979.

[2] K. Abrahamson, N. Dadoun, D. A. Kirkpatrick, and T. Przytcka. A simple parallel tree contraction algorithm. *Journal of Algorithms*, 10(2):287–302, 1989.

[3] N. Abramson. *Information Theory and Coding*. McGraw-Hill, 1963.

[4] P. K. Agarwal, H. Edelsbrunner, O. Schwartzkopf, and E. Welzl. Euclidean minimum spanning trees and bichromatic closest pairs. *Discrete and Computational Geometry*, 6:407–422, 1991.

[5] P. K. Agarwal and J. Matoušek. Relative neighborhood graphs in three dimensions. In *Proceedings 3rd Annual Symposium on Discrete Algorithms*, pages 58–65, 1992.

[6] A. Aggarwal, B. Chazelle, L. Guibas, C. O'Dunlaing, and C. Yap. Parallel computational geometry. *Algorithmica*, 3:293–327, 1988.

[7] A. V. Aho, J. E. Hopcroft, and J. D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley Publishing Company, 1974.

[8] A. W. Appel. An efficient program for many-body simulation. *SIAM J. Sci. Stat. Comput.*, 6:85–103, 1985.

[9] S. Arya and D. M. Mount. Approximate nearest-neighbor queries in fixed dimensions. In *Proceedings 4th Annual Symposium on Discrete Algorithms*, pages 271–280, 1993.

[10] S. Arya, D. M. Mount, N. S. Netanyahu, R. Silverman, and A. Wu. An optimal algorithm for approximate nearest-neighbor searching. In *Proceedings 5th Annual Symposium on Discrete Algorithms*, pages 573–582, 1994.

[11] M. J. Atallah and M. T. Goodrich. Efficient parallel solutions to some geometric problems. *Journal of Parallel and Distributed Computing*, 3:492–507, 1986.

[12] F. Aurenhammer. Voronoi diagrams—a survey of a fundamental geometric data structure. *Computing Surveys*, 23(3):345–405, 1991.

[13] J. Barnes and P. Hut. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nature*, 324(4):446–449, 1986.

[14] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the Association for Computing Machinery*, 18:509–517, 1975.

[15] J. L. Bentley. Multidimensional divide-and-conquer. *Communications of the Association for Computing Machinery*, 23(4):214–229, 1980.

[16] C. Berge and A. Ghouila-Houri. *Programming, Games, and Transportation Networks*. John Wiley and Sons, Inc., 1965.

[17] M. Bern, D. Eppstein, and S.-H. Teng. Parallel construction of quadtrees and quality triangulations. In *Algorithms and Data Structures, Third Workshop, WADS '93, Lecture Notes in Computer Science 709*, pages 188–199. Springer-Verlag, 1993.

[18] P. B. Callahan. Optimal parallel all-nearest-neighbors using the well-separated pair decomposition. In *Proceedings 34th Annual Symposium on Foundations of Computer Science*, pages 332–341, 1993.

[19] P. B. Callahan and S. R. Kosaraju. A decomposition of multi-dimensional point-sets with applications to k-nearest-neighbors and n-body potential fields. In *Proceedings 24th Annual ACM Symposium on the Theory of Computing*, pages 546–556, 1992.

[20] P. B. Callahan and S. R. Kosaraju. A decomposition of multi-dimensional point-sets with applications to k-nearest-neighbors and n-body potential fields. Revised Version, 1992.

[21] P. B. Callahan and S. R. Kosaraju. Faster algorithms for some geometric graph problems in higher dimensions. In *Proceedings 4th Annual Symposium on Discrete Algorithms*, pages 291–300, 1993.

[22] P. B. Callahan and S. R. Kosaraju. Algorithms for dynamic closest pair and n-body potential fields. In *Proceedings 6th Annual Symposium on Discrete Algorithms*, 1995.

[23] B. Chandra, G. Das, G. Narasimhan, and J. Soares. New sparseness results on graph spanners. In *8th Annual ACM Symposium on Computational Geometry*, pages 192–201, 1992.

[24] L. P. Chew. There is a planar graph almost as good as the complete graph. In *Proceedings Second Annual Symposium on Computational Geometry*, pages 160–177, 1986.

[25] K. L. Clarkson. Fast algorithms for the all nearest neighbors problem. In *Proceedings 24th Annual Symposium on Foundations of Computer Science*, pages 226–232, 1983.

[26] R. Cole. Parallel merge sort. *SIAM Journal on Computing*, 17:770–785, 1988.

[27] R. Cole and M. T. Goodrich. Optimal parallel algorithms for polygon and point-set problems. In *Proceedings 4th ACM Symposium on Computational Geometry*, pages 201–210, 1988.

[28] R. Cole and M. T. Goodrich. Optimal parallel algorithms for point-set and polygon problems. *Algorithmica*, 7:3–23, 1992.

[29] R. Cole and U. Vishkin. The accelerated centroid decomposition technique for optimal parallel tree evaluation in logarithmic time. *Algorithmica*, 3:329–346, 1988.

[30] D. Dobkin and R. Lipton. On the complexity of computations under varying sets of primitives. *Journal of Computer and System Sciences*, 18:86–91, 1979.

[31] D. B. Dobkin, S. J. Friedman, and K. J. Supowit. Delaunay graphs are almost as good as complete graphs. In *Proceedings 28th Annual Symposium on Foundations of Computer Science*, pages 20–26, 1987.

[32] G. N. Frederickson. A data structure for dynamically maintaining rooted trees. In *Proceedings 4th Annual Symposium on Discrete Algorithms*, pages 175–184, 1993.

[33] A. M. Frieze, G. L. Miller, and S.-H. Teng. Separator based parallel divide and conquer in computational geometry. In *Proceedings 4th Annual Symposium Parallel Algorithms and Architectures*, pages 420–429, 1992.

[34] H. Gazit, G. L. Miller, and S.-H. Teng. Optimal tree contraction in the EREW model. In S. K. Tewsburg, B. W. Dickinson, and S. C. Schwartz, editors, *Concurrent Computations*, pages 139–155. Plenum Publishing, 1988.

[35] M. Golin, R. Raman, C. Schwarz, and M. Smid. Randomized data structures for the dynamic closest pair problem. In *Proceedings 4th Annual Symposium on Discrete Algorithms*, pages 301–310, 1993.

[36] L. Greengard and W. D. Gropp. A parallel version of the fast multipole method. *Computers Math. Applic.*, 20(7):63–71, 1990.

[37] L. Greengard and V. Rokhlin. A fast algorithm for particle simulations. *Journal of Computational Physics*, 73:325–348, 1987.

[38] L. F. Greengard. *The Rapid Evaluation of Potential Fields in Particle Systems.* The MIT Press, 1988.

[39] C. Hafner and N. Kuster. Computations of electromagnetic fields by the multiple multipole method (generalized multipole technique). *Radio Science*, 26(1):291–7, 1991.

[40] L. Hernquist. Hierarchical n-body methods. *Computer Physics Communications*, 48:107–115, 1988.

[41] S. Kapoor and M. Smid. New techniques for exact and approximate dynamic closest-point problems. In *10th Annual ACM Symposium on Computational Geometry*, pages 165–174, 1994.

[42] R. M. Karp and V. Ramachandran. A survey of parallel algorithms for shared-memory machines. Report No. UCB/CSD 88/408, Computer Science Division (EECS), University of California, Berkeley, 1988.

[43] J. Katzenelson. Computational structure of the N-body problem. *SIAM Journal on Scientific and Statistical Computing*, 10(4):787–815, July 1989.

[44] M. Keil. Approximating the complete Euclidean graph. In *Proceedings First Scandinavian Workshop on Algorithm Theory*, pages 208–213, 1988.

[45] S. R. Kosaraju and A. L. Delcher. Optimal parallel evaluation of tree-structured computations by raking. In J. Reif, editor, *VLSI Algorithms and Architectures: Proceedings of the 3rd Aegean Workshop on Computing*, pages 101–110. Springer-Verlag, 1988.

[46] S. C. Kwan and W. L. Ruzzo. Adaptive parallel algorithms for finding minimum spanning trees. In *Proceedings 1984 International Parallel Processing Conference*, pages 439–443, 1984.

[47] H.-P. Lenhof and M. Smid. Enumerating the k closest pairs optimally. In *Proceedings 33rd Annual Symposium on Foundations of Computer Science*, pages 380–386, 1992.

[48] G. L. Miller, S.-H. Teng, and S. A. Vavasis. A unified geometric approach to graph separators. In *Proceedings 32nd Annual Symposium on Foundations of Computer S cience*, pages 538–547, 1991.

[49] R. H. Miller and K. H. Prendergast. Stellar dynamics in a discrete phase space. *The Astrophysical Journal*, 151:699–709, 1968.

[50] R. H. Miller, K. H. Prendergast, and W. J. Quirk. Numerical experiments on spiral structure. *The Astrophysical Journal*, 161:903–916, 1970.

[51] V. Y. Pan, J. H. Reif, and S. R. Tate. The power of combining the techniques of algebraic and numerical computing: Improved approximate multipoint polynomial evaluation and improved multipole algorithms. In *Proceedings 32nd Annual Symposium on Foundations of Computer S cience*, pages 703–713, 1992.

[52] F. P. Preparata and M. I. Shamos. *Computational Geometry: An Introduction.* Springer-Verlag, 1985.

[53] M. O. Rabin. Proving simultaneous positivity of linear forms. *Journal of Computer and System Sciences*, 6:639–650, 1972.

[54] J. H. Reif and S. R. Tate. The complexity of n-body simulation. (draft), Computer Science Dept., Duke University, 1992.

[55] E. M. Reingold. On the optimality of some set algorithms. *Journal of the Association for Computing Machinery*, 19:649–659, 1972.

[56] V. Rokhlin. Rapid solution of integral equations of classical potential theory. *Journal of Computational Physics*, 60:187–207, 1985.

[57] J. Ruppert and R. Seidel. Approximating the d-dimensional complete Euclidean graph. In *Third Canadian Conference on Computational Geometry*, pages 207–210, 1991.

[58] J. S. Salowe. Constructing multidimensional spanner graphs. *International Journal of Computional Geometry and Applications*, 1(2):99–107, 1991.

[59] J. S. Salowe. Construction of multidimensional spanner graphs with applications to minimum spanning trees. In *7th Annual ACM Symposium on Computational Geometry*, pages 256–261, 1991.

[60] J. S. Salowe. Shallow interdistance enumeration and interdistance enumeration. In *Proceedings of WADS '91*, pages 117–128, 1991.

[61] H. Samet. The quadtree and related hierarchical data structures. *Computing Surveys*, 16(2):187–260, 1984.

[62] K. E. Schmidt and M. A. Lee. Implementing the fast multipole multipole method in three dimensions. *J. of Stat. Physics*, 63(5–6):1223–35, June 1991.

[63] C. Schwarz, M. Smid, and J. Snoeyink. An optimal algorithm for the on-line closest pair problem. In *8th Annual ACM Symposium on Computational Geometry*, pages 330–336, 1992.

[64] M. I. Shamos and D. Hoey. Closest-point problems. In *Proceedings of the 16th Annual IEEE Symposium on Foundations of Computer Science*, pages 151–162, 1975.

[65] R. E. Tarjan and U. Vishkin. Finding biconnected components and computing tree functions in logarithmic parallel time. *SIAM Journal on Computing*, 14:862–874, 1985.

[66] P. M. Vaidya. A fast approximation algorithm for minimum spanning trees in k-dimensional space. In *Proceedings 25th Annual Symposium on Foundations of Computer S cience*, pages 403–407, 1984.

[67] P. M. Vaidya. An optimal algorithm for the all-nearest-neighbors problem. In *Proceedings 27th Annual Symposium on Foundations of Computer Science*, pages 117–122, 1986.

[68] P. M. Vaidya. Minimum spanning trees in k-dimensional space. *SIAM Journal on Computing*, 17:572–582, 1988.

[69] P. M. Vaidya. An $O(n \log n)$ algorithm for the all-nearest-neighbors problem. *Discrete and Computational Geometry*, 4:101–115, 1989.

[70] P. M. Vaidya. A sparse graph almost as good as the complete graph on points in k dimensions. *Discrete and Computational Geometry*, 6:369–381, 1991.

[71] A. C. Yao. On constructing minimum spanning trees in k-dimensional space and related problems. *SIAM Journal on Computing*, 11:721–736, 1982.

[72] F. Zhao and S. L. Johnsson. The parallel multipole method on the Connection Machine. *SIAM Journal on Scientific and Statistical Computing*, 12(6):1420–1437, 1991.

Vita

Paul Callahan was born August 5, 1965 in the suburbs of Philadelphia, where he lived his entire life before going to college. Not much is remembered from that time, save that it involved a lot of open space and wild, unfettered greenery, most of which is no longer extant.

He spent the years from 1983 to 1987 in State College, Pennsylvania, earning his B.S. from Penn State University with a major in Computer Science, and minors in Math and Physics. Reluctant to move away so soon from Pennsylvania's answer to Shangri-la, albeit a gray and drizzly one, he continued to live in State College from 1987 to 1989, while earning his M.S. in Computer Science.

In 1989, he moved to Baltimore to begin a doctoral program in Computer Science at Johns Hopkins University, thus hinting at a possible trend in his professional interests. As with most of those to whom fate decrees an extended stay in Baltimore, he eventually learned to appreciate the dismal charm of the place. During his stay, the quality of coffee improved markedly, with an explosion of so many trendy cafes that he almost learned to hate espresso (almost). Early in 1995, he defended his dissertation, leaving him with no compelling reason to remain in Baltimore. The experience has left him with much to puzzle over, the greatest mystery of all being why "the city that reads" contains so few worthwhile bookstores.